

```

    flipCrop,
    isFullCrop,
    naturalSize,
    normalizeCrop,
    quarterTurn,
    type CropHandle,
  } from "@lib/whiteboard/imageEdit";
import { useResolvedImageSrc } from "../ElementRenderer";
import { cn } from "@lib/utls";

type Tab = "crop" | "look" | "shape";

/** Ratios as the user SEES them (width:height on screen). null = free. */
const RATIOS: Array<{ id: string; label: string; r: number | null | "original" }> = [
  { id: "free", label: "Free", r: null },
  { id: "original", label: "Original", r: "original" },
  { id: "1:1", label: "1:1", r: 1 },
  { id: "4:3", label: "4:3", r: 4 / 3 },
  { id: "3:4", label: "3:4", r: 3 / 4 },
  { id: "16:9", label: "16:9", r: 16 / 9 },
];

const BORDER_COLORS = ["#18181b", "#ffffff", "#6366f1", "#f43f5e", "#f59e0b", "#10b981"];

const HANDLES: Array<{ h: CropHandle; style: React.CSSProperties; cursor: string }> = [
  { h: "nw", style: { left: 0, top: 0 }, cursor: "nwse-resize" },
  { h: "ne", style: { left: "100%", top: 0 }, cursor: "nesw-resize" },
  { h: "sw", style: { left: 0, top: "100%" }, cursor: "nesw-resize" },
  { h: "se", style: { left: "100%", top: "100%" }, cursor: "nwse-resize" },
  { h: "n", style: { left: "50%", top: 0 }, cursor: "ns-resize" },
  { h: "s", style: { left: "50%", top: "100%" }, cursor: "ns-resize" },
  { h: "w", style: { left: 0, top: "50%" }, cursor: "ew-resize" },
  { h: "e", style: { left: "100%", top: "50%" }, cursor: "ew-resize" },
];

export function ImageStudio() {
  const id = useWhiteboard((s) => s.imageStudioId);
  const board = useActiveBoard();
  const el = React.useMemo(() => {
    if (!id || !board) return null;
    const found = getBoardElements(board).find((e) => e.id === id);
    return found && found.type === "image" ? (found as ImageElement) : null;
    // Only the id picks the picture: live edits elsewhere must not reset the draft.
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [id]);
  if (!id || !el || typeof document === "undefined") return null;
  // Portalled to <body> so it covers the sidebar and every floating bar.
  return createPortal(<Studio key={id} el={el} />, document.body);
}

function Studio({ el }: { el: ImageElement }) {
  const close = () => useWhiteboard.getState().setImageStudioId(null);
  const href = useResolvedImageSrc(el.src);
  const { nw, nh } = naturalSize(el);

  const [tab, setTab] = React.useState<Tab>("crop");
  const [crop, setCrop] = React.useState<ImageCrop>(normalizeCrop(el.crop));
  const [flipX, setFlipX] = React.useState(!el.flipX);
  const [flipY, setFlipY] = React.useState(!el.flipY);
  const [turns, setTurns] = React.useState(0); // extra quarter turns, clockwise
  const [ratio, setRatio] = React.useState<string>("free");
  const [filter, setFilter] = React.useState<ImageFilter>(el.filter ?? "none");
  const [frame, setFrame] = React.useState<ImageFrame>(el.frame ?? "square");
  const [borderWidth, setBorderWidth] = React.useState(el.borderWidth ?? 0);
  const [borderColor, setBorderColor] = React.useState(el.borderColor ?? "#18181b");

  // ---- stage size -----
  const stageRef = React.useRef<HTMLDivElement>(null);
  const [stage, setStage] = React.useState({ w: 600, h: 400 });
  React.useLayoutEffect(() => {
    const node = stageRef.current;
    if (!node) return;
    const measure = () => setStage({ w: node.clientWidth, h: node.clientHeight });
    measure();
    const ro = new ResizeObserver(measure);
    ro.observe(node);
    return () => ro.disconnect();
  }, []);

```

```

const odd = turns % 2 === 1;
// The picture fits the stage AS TURNED: a sideways photo swaps its sides.
const pad = 28;
const fit = Math.min((stage.w - pad * 2) / (odd ? nh : nw), (stage.h - pad * 2) / (odd ? nw : nh));
const dispW = Math.max(40, nw * fit);
const dispH = Math.max(40, nh * fit);

/** A screen ratio as a crop-fraction ratio (1 = the picture's own shape). */
const fractionAspect = React.useCallback(
  (id: string): number | null => {
    const r = RATIOS.find((x) => x.id === id)?.r ?? null;
    if (r === null) return null;
    if (r === "original") return 1;
    const px = odd ? 1 / r : r; // on screen w:h is picture h:w when turned sideways
    return (px * nh) / nw;
  },
  [odd, nw, nh]
);

const pickRatio = (id: string) => {
  setRatio(id);
  const a = fractionAspect(id);
  if (a) setCrop((c) => (isFullCrop(c) ? centeredAspectCrop(a) : fitAspect(c, a)));
};

// ---- dragging the crop -----
const drag = React.useRef<{ handle: CropHandle; x: number; y: number; start: ImageCrop; pid: number } | null>(null);
const onHandleDown = (handle: CropHandle) => (e: React.PointerEvent) => {
  e.preventDefault();
  e.stopPropagation();
  (e.currentTarget as HTMLElement).setPointerCapture(e.pointerId);
  drag.current = { handle, x: e.clientX, y: e.clientY, start: crop, pid: e.pointerId };
};
const onMove = (e: React.PointerEvent) => {
  const d = drag.current;
  if (!d || d.pid !== e.pointerId) return;
  // Screen movement -> picture movement: undo the stage's quarter turns.
  const sx = e.clientX - d.x;
  const sy = e.clientY - d.y;
  const t = (-turns * Math.PI) / 2;
  const cx = sx * Math.cos(t) - sy * Math.sin(t);
  const cy = sx * Math.sin(t) + sy * Math.cos(t);
  setCrop(dragCrop(d.start, d.handle, cx / dispW, cy / dispH, fractionAspect(ratio)));
};
const onUp = (e: React.PointerEvent) => {
  if (drag.current?.pid === e.pointerId) drag.current = null;
};

const doFlip = (axis: "x" | "y") => {
  if (axis === "x") setFlipX((v) => !v);
  else setFlipY((v) => !v);
  setCrop((c) => flipCrop(c, axis));
};
const turn = (dir: 1 | -1) => setTurns((t) => (t + dir + 4) % 4);

const reset = () => {
  setCrop({ ...FULL_CROP });
  setFlipX(false);
  setFlipY(false);
  setTurns(0);
  setRatio("free");
  setFilter("none");
  setFrame("square");
  setBorderWidth(0);
};

const apply = () => {
  const store = useWhiteboard.getState();
  const c = normalizeCrop(crop);
  const box = boxForCrop(el, c);
  let rotation = el.rotation || 0;
  for (let i = 0; i < turns; i++) rotation = quarterTurn(rotation, 1);
  store.commitHistory();
  store.updateElement(
    el.id,
    {

```

```

    ...box,
    rotation,
    crop: isFullCrop(c) ? undefined : c,
    flipX: flipX || undefined,
    flipY: flipY || undefined,
    filter: filter === "none" ? undefined : filter,
    frame: frame === "square" ? undefined : frame,
    borderWidth: borderWidth || undefined,
    borderColor: borderWidth ? borderColor : undefined,
  } as Partial<ImageElement>,
  false
);
close();
};

React.useEffect(() => {
  const onKey = (e: KeyboardEvent) => {
    if (e.key === "Escape") {
      e.stopPropagation();
      close();
    } else if (e.key === "Enter") {
      e.preventDefault();
      apply();
    }
  };
  window.addEventListener("keydown", onKey, true);
  return () => window.removeEventListener("keydown", onKey, true);
});

const css = filterCss(filter);
const flipCss = `scale(${flipX ? -1 : 1}, ${flipY ? -1 : 1})`;
const cropStyle: React.CSSProperties = {
  left: crop.x * dispW,
  top: crop.y * dispH,
  width: crop.w * dispW,
  height: crop.h * dispH,
  borderRadius: frame === "circle" ? "50%" : frame === "rounded" ? Math.min(crop.w * dispW, crop.h * dispH)
    * 0.08 : 0,
  boxShadow: "0 0 0 9999px rgba(9, 9, 11, 0.62)",
  outline: borderWidth && tab === "shape" ? `${Math.max(1, borderWidth * fit)}px solid ${borderColor}` : unde
    fined,
};

const TABS: Array<{ id: Tab; label: string; icon: typeof Crop }> = [
  { id: "crop", label: "Crop", icon: Crop },
  { id: "look", label: "Look", icon: Palette },
  { id: "shape", label: "Shape", icon: Shapes },
];

return (
  <div
    className="fixed inset-0 z-[80] flex flex-col bg-zinc-950/95 text-zinc-100 backdrop-blur-md animate-in fade-in"
    onPointerMove={onMove}
    onPointerUp={onUp}
    onPointerCancel={onUp}
    role="dialog"
    aria-label="Edit picture"
  >
    { /* Top bar */ }
    <div className="flex items-center justify-between gap-2 px-3 py-2.5 sm:px-5">
      <button onClick={close} className="flex items-center gap-1.5 rounded-full px-3 py-1.5 text-sm text-zinc-300 hover:bg-white/10">
        <X size={16} /> Cancel
      </button>
      <div className="text-sm font-semibold tracking-wide">Edit picture</div>
      <button
        onClick={apply}
        className="flex items-center gap-1.5 rounded-full bg-indigo-500 px-4 py-1.5 text-sm font-semibold text-white shadow-lg shadow-indigo-500/30 hover:bg-indigo-400"
      >
        <Check size={16} /> Done
      </button>
    </div>

    { /* Stage */ }
    <div ref={stageRef} className="relative min-h-0 flex-1 overflow-hidden">
      <div

```

```

class="absolute left-1/2 top-1/2 transition-transform duration-200"
style={{ width: dispW, height: dispH, transform: `translate(-50%, -50%) rotate(${turns * 90}deg)` }}
>
<div className="absolute inset-0 overflow-hidden">
  {href ? (
    // eslint-disable-next-line @next/next/no-img-element
    <img
      src={href}
      alt=""
      draggable={false}
      className="h-full w-full select-none"
      style={{ transform: flipCss, filter: css || undefined }}
    />
  ) : (
    <div className="h-full w-full animate-pulse bg-zinc-800" />
  )}
</div>
{ /* The kept area; everything outside is dimmed by its shadow. */ }
<div
  className={cn("absolute touch-none", tab === "crop" ? "cursor-move" : "pointer-events-none")}
  style={cropStyle}
  onPointerDown={tab === "crop" ? onHandleDown("move") : undefined}
>
  {tab === "crop" && (
    <
      { /* thirds grid */ }
      <div className="pointer-events-none absolute inset-0 border border-white/90">
        <div className="absolute inset-y-0 left-1/3 w-px bg-white/35" />
        <div className="absolute inset-y-0 left-2/3 w-px bg-white/35" />
        <div className="absolute inset-x-0 top-1/3 h-px bg-white/35" />
        <div className="absolute inset-x-0 top-2/3 h-px bg-white/35" />
      </div>
      {HANDLES.map(({ h, style, cursor }) => (
        <div
          key={h}
          onPointerDown={onHandleDown(h)}
          className="absolute flex h-9 w-9 -translate-x-1/2 -translate-y-1/2 items-center justify-center touch-none"
          style={{ ...style, cursor }}
        >
          <span
            className={cn(
              "block rounded-full bg-white shadow-[0_0_1.5px_rgba(99,102,241,0.9)]",
              h.length === 2 ? "h-3.5 w-3.5" : h === "n" || h === "s" ? "h-1.5 w-6" : "h-6 w-1.5"
            )}
          />
        </div>
      )]}
    </>
  )}
</div>
</div>
{ /* Tools */ }
<div className="border-t border-white/10 bg-zinc-900/80 px-3 pb-4 pt-3 sm:px-5">
  <div className="mx-auto max-w-xl">
    {tab === "crop" && (
      <div className="flex flex-col gap-3">
        <div className="flex gap-1.5 overflow-x-auto pb-0.5">
          {RATIOS.map((r) => (
            <Chip key={r.id} active={ratio === r.id} onClick={() => pickRatio(r.id)}>
              {r.label}
            </Chip>
          ))}
        </div>
        <div className="flex items-center justify-center gap-1">
          <IconBtn label="Turn left" onClick={() => turn(-1)} icon={RotateCcw} />
          <IconBtn label="Turn right" onClick={() => turn(1)} icon={RotateCw} />
          <span className="mx-2 h-6 w-px bg-white/15" />
          <IconBtn label="Mirror" onClick={() => doFlip("x")} icon={FlipHorizontal2} active={flipX} />
          <IconBtn label="Upside down" onClick={() => doFlip("y")} icon={FlipVertical2} active={flipY} />
          <span className="mx-2 h-6 w-px bg-white/15" />
          <IconBtn label="Reset all" onClick={reset} icon={Undo2} />
        </div>
      </div>
    )}
  </div>
}
}

```

```

{tab === "look" && (
  <div className="flex gap-2.5 overflow-x-auto pb-1">
    {IMAGE_FILTERS.map((f) => (
      <button
        key={f.id}
        onClick={() => setFilter(f.id)}
        title={f.hint}
        className="flex w-[72px] shrink-0 flex-col items-center gap-1.5"
      >
        <span
          className={cn(
            "block h-16 w-16 overflow-hidden rounded-2xl ring-2 transition",
            filter === f.id ? "ring-indigo-400" : "ring-transparent hover:ring-white/30"
          )}
        >
          {href && (
            // eslint-disable-next-line @next/next/no-img-element
            <img src={href} alt="" className="h-full w-full object-cover" style={{ filter: f.css ||
            undefined, transform: flipCss }} draggable={false} />
          )}
        </span>
        <span className={cn("text-[11px]", filter === f.id ? "font-semibold text-white" : "text-zinc
        -400")}>{f.label}</span>
      </button>
    ))}
  </div>
)}

{tab === "shape" && (
  <div className="flex flex-col gap-3">
    <div className="flex justify-center gap-2">
      {IMAGE_FRAMES.map((f) => (
        <button
          key={f.id}
          onClick={() => setFrame(f.id)}
          className={cn(
            "flex w-20 flex-col items-center gap-1.5 rounded-2xl py-2 transition",
            frame === f.id ? "bg-indigo-500/20 text-white" : "text-zinc-400 hover:bg-white/5"
          )}
        >
          <span
            className={cn(
              "block h-8 w-8 border-2",
              frame === f.id ? "border-indigo-300" : "border-zinc-500",
              f.id === "circle" ? "rounded-full" : f.id === "rounded" ? "rounded-lg" : "rounded-none"
            )}
          />
          <span className="text-[11px]">{f.label}</span>
        </button>
      ))}
    </div>
    <div className="flex flex-wrap items-center justify-center gap-2">
      <span className="text-[11px] text-zinc-400">Border</span>
      {[0, 2, 5].map((w) => (
        <Chip key={w} active={borderWidth === w} onClick={() => setBorderWidth(w)}>
          {w === 0 ? "None" : w === 2 ? "Thin" : "Thick"}
        </Chip>
      ))}
      {borderWidth > 0 &&
        BORDER_COLORS.map((c) => (
          <button
            key={c}
            onClick={() => setBorderColor(c)}
            aria-label={`Border colour ${c}`}
            className={cn("h-6 w-6 rounded-full border border-white/25", borderColor === c && "ring-
            2 ring-indigo-400 ring-offset-2 ring-offset-zinc-900")}
            style={{ background: c }}
          />
        ))}
    </div>
  </div>
)}

<div className="mx-auto mt-3 grid max-w-xs grid-cols-3 gap-1 rounded-2xl bg-white/5 p-1">
  {TABS.map(({ id, label, icon: Icon }) => (
    <button

```

```

        key={id}
        onClick={() => setTab(id)}
        className={cn(
          "flex items-center justify-center gap-1.5 rounded-xl py-2 text-[13px] font-medium transition",
          tab === id ? "bg-white text-zinc-900 shadow" : "text-zinc-400 hover:text-white"
        )}
      >
        <Icon size={15} /> {label}
      </button>
    )}
  </div>
</div>
</div>
</div>
);
}

```

```

function Chip({ active, onClick, children }: { active: boolean; onClick: () => void; children: React.ReactNode }) {
  return (
    <button
      onClick={onClick}
      className={cn(
        "shrink-0 rounded-full px-3.5 py-1.5 text-[12px] font-medium transition",
        active ? "bg-white text-zinc-900" : "bg-white/8 text-zinc-300 hover:bg-white/15"
      )}
    >
      {children}
    </button>
  );
}

```

```

function IconBtn({
  label,
  onClick,
  icon: Icon,
  active,
}: {
  label: string;
  onClick: () => void;
  icon: typeof Crop;
  active?: boolean;
}) {
  return (
    <button
      onClick={onClick}
      title={label}
      aria-label={label}
      className={cn(
        "flex h-10 w-10 items-center justify-center rounded-full transition",
        active ? "bg-indigo-500/25 text-indigo-200" : "text-zinc-300 hover:bg-white/10"
      )}
    >
      <Icon size={18} />
    </button>
  );
}

```

```

=====
// === END FILE: src/components/whiteboard/ImageStudio.tsx ===
=====

```

```

=====
// === START FILE: src/components/whiteboard/InlineTextEditor.tsx ===
// File 57 of 194 | Lines: 1577 | Size: 61903 bytes
=====

```

```

"use client";

/**
 * Inline Text & Markdown Editor (v5 — Professional Grade)
 * =====
 * The unified, professional inline text editor for ProNotes.
 *
 * Core Features:
 * - Live Voice Dictation ("Record to Text"): Real-time Speech-to-Text streaming

```

```

*   directly into the note with pulsing audio waveform and listening status.
* - Intelligent List Continuation: Pressing Enter on ` [ ]`, ` - `, or `1. `
*   automatically continues the list, and pressing Enter on an empty prefix
*   exits list mode cleanly.
* - Rich Typography & Formatting: Headings (H1/H2/H3), Bold, Italic, Underline,
*   Strikethrough, Code, Highlighting, Blockquotes, Checklists, and LaTeX Math.
* - Card / Callout Mode: Toggle between canvas transparent text and styled
*   cards with background tints, borders, and rounded corners.
* - Smart Text Utilities: Case transformations (UPPERCASE, lowercase, Title Case)
*   and real-time word/character count.
* - Touch & Stylus Ergonomics: Clamped to viewport edges, generous touch targets,
*   glassmorphic design, and instant tactile feedback.
*/

import React, { useEffect, useLayoutEffect, useRef, useState, useCallback, useMemo } from "react";
import { useWhiteboard } from "@store/whiteboard";
import { getBoardElements } from "@lib/whiteboard/notebook";
import { worldToScreen, type Viewport } from "@lib/whiteboard/geometry";
import {
  FONT_FAMILIES,
  PEN_COLORS,
  STICKY_COLORS,
  CARD_BACKGROUNDS,
  setToolInk,
  type LineDir,
  type StickyColor,
  type StickyElement,
  type StyleState,
  type TextAlign,
  type TextElement,
} from "@lib/whiteboard/types";
import {
  detectLineDir,
  getNextLinePrefix,
  prefixSelectedLines,
  resolveLineDir,
  selectedLineRange,
  setLineDirs,
  spliceFormatMarkers,
} from "@lib/whiteboard/textParser";
import {
  VoiceDictationController,
  spliceDictationChunk,
  placeInterim,
  stripInterim,
  SUPPORTED_DICTATION_LANGS,
  resolveDictationLang,
  type DictationRange,
  type SpeechStatus,
} from "@lib/whiteboard/speechRecognition";
import {
  Bold,
  Italic,
  Underline,
  Strikethrough,
  Code,
  Heading1,
  Heading2,
  Heading3,
  List,
  ListOrdered,
  CheckSquare,
  Quote,
  AlignLeft,
  AlignCenter,
  AlignRight,
  Link2,
  Check,
  ChevronDown,
  MoreHorizontal,
  ArrowUpToLine,
  ArrowDownToLine,
  Copy,
  Lock,
  Trash2,
  Mic,
  MicOff,
  Languages,

```

```

    Square,
    Sparkles,
    AlertCircle,
    Loader2,
    X,
  } from "lucide-react";
import { Popover, PopoverContent, PopoverTrigger } from "@components/ui/popover";
import { Tooltip, TooltipContent, TooltipProvider, TooltipTrigger } from "@components/ui/tooltip";
import { StepButton } from "../StepperControl";
import { cn } from "@lib/utils";

const MIN_FONT = 10;
const MAX_FONT = 96;
const SIZE_PRESETS = [12, 14, 18, 24, 32, 40, 54, 72];
const EDGE = 8;
const GAP = 8;

export function InlineTextEditor({ viewport }: { viewport: Viewport }) {
  const editingId = useWhiteboard((s) => s.editingId);
  const setEditingId = useWhiteboard((s) => s.setEditingId);
  const camera = useWhiteboard((s) => s.camera);
  const activeBoardId = useWhiteboard((s) => s.activeBoardId);
  const boards = useWhiteboard((s) => s.boards);
  const updateElement = useWhiteboard((s) => s.updateElement);
  const deleteElements = useWhiteboard((s) => s.deleteElements);
  const duplicateElements = useWhiteboard((s) => s.duplicateElements);
  const toggleLock = useWhiteboard((s) => s.toggleLock);
  const bringToFront = useWhiteboard((s) => s.bringToFront);
  const sendToBack = useWhiteboard((s) => s.sendToBack);
  const commitHistory = useWhiteboard((s) => s.commitHistory);
  const setStyle = useWhiteboard((s) => s.setStyle);

  const activeBoard = useMemo(() => boards.find((b) => b.id === activeBoardId), [boards, activeBoardId]);
  const activeElement = useMemo(() => {
    if (!editingId || !activeBoard) return null;
    return getBoardElements(activeBoard).find((el) => el.id === editingId) || null;
  }, [editingId, activeBoard]);

  const [text, setText] = useState("");
  // Live selection, so line tools (direction, and the bar indicator) follow
  // the cursor without waiting for a keystroke.
  const [selRange, setSelRange] = useState({ start: 0, end: 0 });
  const [wikiQuery, setWikiQuery] = useState<string | null>(null);
  const [wikiCursorPos, setWikiCursorPos] = useState<number>(0);
  const [selectedWikiIndex, setSelectedWikiIndex] = useState(0);
  const [barSize, setBarSize] = useState({ width: 0, height: 44 });

  // Voice dictation state
  const [dictating, setDictating] = useState(false);
  const [speechStatus, setSpeechStatus] = useState<SpeechStatus>("idle");
  const [dictationMessage, setDictationMessage] = useState<string | null>(null);
  const [interimText, setInterimText] = useState("");
  const dictationRef = useRef<VoiceDictationController | null>(null);
  const dictationDismissTimer = useRef<NodeJS.Timeout | null>(null);
  const dictationLang = useWhiteboard((s) => s.dictationLang);
  const setDictationLang = useWhiteboard((s) => s.setDictationLang);
  const pickDictationLang = useCallback(
    (id: string) => {
      const resolved = resolveDictationLang(id);
      setDictationLang(resolved);
      // Live-switch when the engine allows it; otherwise the next
      // Dictate press starts in the new language.
      dictationRef.current?.setLang(resolved);
    },
    [setDictationLang]
  );

  const textareaRef = useRef<HTMLTextAreaElement>(null);
  const boxRef = useRef<HTMLDivElement>(null);
  const barRef = useRef<HTMLDivElement>(null);
  const sessionCommittedRef = useRef(false);

  // Sync state when entering edit mode on an element
  useEffect(() => {
    sessionCommittedRef.current = false;
    setDictating(false);
    setInterimText("");
    setDictationMessage(null);
  });

```

```

lastDictationRef.current = null;
interimRangeRef.current = null;
if (dictationDismissTimer.current) {
  clearTimeout(dictationDismissTimer.current);
  dictationDismissTimer.current = null;
}
// abort, not stop: stop() writes the words still being heard, and by now
// this callback points at the NEW box. Leaving a box (exitEditing) stops
// properly first, so nothing said there is lost.
if (dictationRef.current) {
  dictationRef.current.abort();
}

if (activeElement && (activeElement.type === "text" || activeElement.type === "sticky")) {
  const elText = (activeElement as TextElement | StickyElement).text || "";
  setText(elText);
  setWikiQuery(null);

  requestAnimationFrame(() => {
    if (textareaRef.current) {
      textareaRef.current.focus({ preventScroll: true });
      const len = textareaRef.current.value.length;
      textareaRef.current.setSelectionRange(len, len);
      adjustHeight();
    }
  });
}
// eslint-disable-next-line react-hooks/exhaustive-deps
}, [editingId]);

// Clean up voice dictation when component unmounts
useEffect(() => {
  return () => {
    dictationRef.current?.abort();
    if (dictationDismissTimer.current) {
      clearTimeout(dictationDismissTimer.current);
    }
  };
}, []);

const exitEditing = useCallback(() => {
  if (dictationDismissTimer.current) {
    clearTimeout(dictationDismissTimer.current);
    dictationDismissTimer.current = null;
  }
  if (dictating) {
    dictationRef.current?.stop();
    setDictating(false);
    setInterimText("");
  }
  lastDictationRef.current = null;
  setDictationMessage(null);
  setEditingId(null);
}, [dictating, setEditingId]);

// End edit on pointer-down outside
useEffect(() => {
  if (!editingId) return;
  const onDown = (e: PointerEvent) => {
    const t = e.target as Element | null;
    if (!t) return;
    if (boxRef.current?.contains(t)) return;
    if (barRef.current?.contains(t)) return;
    if (t.closest?("[data-radix-popover-content-wrapper]")) return;
    exitEditing();
  };
  document.addEventListener("pointerdown", onDown, true);
  return () => document.removeEventListener("pointerdown", onDown, true);
}, [editingId, exitEditing]);

const mutateElement = useCallback(
  (patch: Partial<TextElement> | Partial<StickyElement>) => {
    if (!activeElement) return;
    if (!sessionCommittedRef.current) {
      commitHistory();
      sessionCommittedRef.current = true;
    }
    updateElement(activeElement.id, patch as never, false);
  },

```

```

    },
    [activeElement, commitHistory, updateElement]
  );

const applyStyle = useCallback(
  (patch: { fontFamily?: string; fontSize?: number; color?: string }) => {
    if (!activeElement) return;
    mutateElement(patch as Partial<TextElement>);
    const current = useWhiteboard.getState().style;
    const next: Partial<StyleState> = {};
    if (activeElement.type === "text") {
      if (patch.fontFamily) next.fontFamily = patch.fontFamily;
      Object.assign(next, setToolLink(current, "text", { color: patch.color, size: patch.fontSize }));
    } else if (patch.fontSize !== undefined) {
      Object.assign(next, setToolLink(current, "sticky", { size: patch.fontSize }));
    }
    if (Object.keys(next).length > 0) setStyle(next);
  },
  [activeElement, mutateElement, setStyle]
);

const adjustHeight = useCallback(() => {
  const ta = textareaRef.current;
  if (!ta || !activeElement) return;
  ta.style.height = "auto";
  const newHeight = Math.max(ta.scrollHeight / camera.zoom, 40);
  if ("height" in activeElement && typeof activeElement.height === "number" && Math.abs(activeElement.height - newHeight) > 4) {
    mutateElement({ height: newHeight } as Partial<TextElement>);
  }
}, [activeElement, camera.zoom, mutateElement]);

useLayoutEffect(() => {
  const bar = barRef.current;
  if (!bar) return;
  const measure = () => setBarSize({ width: bar.offsetWidth, height: bar.offsetHeight });
  measure();
  const ro = new ResizeObserver(measure);
  ro.observe(bar);
  return () => ro.disconnect();
}, [editingId]);

// Where a dictated phrase goes. The controller is created once and outlives
// the box it was started in, so its callback must not close over
// `activeElement`: it did, and dictating into a second box wrote that box's
// whole text into the FIRST one, replacing what was there. This ref always
// points at the box being edited right now.
// Where the last dictated chunk landed. A cumulative re-send from the
// engine ("how" -> "how to") revises this range in place instead of
// appending a second copy. Any manual typing clears it (see
// handleTextChange), and a moved cursor makes the splice fall back to an
// append – so user edits are never eaten by a late revision.
const lastDictationRef = useRef<DictationRange | null>(null);
// The words being heard right now, already written into the text so they
// appear AS you speak (they used to wait, hidden in a small pill, until the
// whole phrase was final). Each interim replaces this range; the final
// replaces it for good.
const interimRangeRef = useRef<DictationRange | null>(null);

/** Writes dictated text without grabbing focus. */
const applyDictatedText = (ta: HTMLTextAreaElement, next: string, cursor: number) => {
  setText(next);
  mutateElement({ text: next });
  // Focusing on every phrase popped the tablet keyboard up again and again
  // while you were talking. The caret moves; focus stays where it is.
  const hadFocus = typeof document !== "undefined" && document.activeElement === ta;
  requestAnimationFrame(() => {
    if (hadFocus) ta.focus({ preventScroll: true });
    ta.setSelectionRange(cursor, cursor);
    adjustHeight();
  });
};

const showInterimRef = useRef<(interim: string) => void>(() => {});
showInterimRef.current = (interim: string) => {
  setInterimText(interim);
  const ta = textareaRef.current;
  if (!ta || !activeElement) return;

```

```

const prev = interimRangeRef.current;
if (!prev && !interim.trim()) return;
const out = placeInterim(ta.value, ta.selectionStart, ta.selectionEnd, prev, interim);
interimRangeRef.current = out.range;
applyDictatedText(ta, out.text, out.cursor);
};

const insertDictationRef = useRef<(chunk: string, opts?: { revisesPrevious: boolean }) => void>(() => {});
insertDictationRef.current = (finalChunk: string, opts?: { revisesPrevious: boolean }) => {
  setInterimText("");
  const ta = textareaRef.current;
  if (!ta || !activeElement) return;
  const { value, selStart, selEnd } = stripInterim(ta.value, ta.selectionStart, ta.selectionEnd, interimRangeRef.current);
  interimRangeRef.current = null;
  const out = spliceDictationChunk(
    value,
    selStart,
    selEnd,
    finalChunk,
    lastDictationRef.current,
    opts?.revisesPrevious ?? false
  );
  lastDictationRef.current = out.range;
  applyDictatedText(ta, out.text, out.cursor);
};

// Voice Dictation Toggle
const toggleDictation = useCallback(async () => {
  if (dictationDismissTimer.current) {
    clearTimeout(dictationDismissTimer.current);
    dictationDismissTimer.current = null;
  }

  if (dictating) {
    dictationRef.current?.stop();
    setDictating(false);
    setInterimText("");
    setSpeechStatus("idle");
  } else {
    setDictationMessage(null);
    if (!dictationRef.current) {
      dictationRef.current = new VoiceDictationController({
        lang: useWhiteboard.getState().dictationLang,
        onInterim: (interim) => showInterimRef.current(interim),
        onFinal: (finalChunk, opts) => insertDictationRef.current(finalChunk, opts),
        onError: (err) => {
          console.warn("Speech dictation note:", err);
          setDictationMessage(err);
          if (dictationDismissTimer.current) clearTimeout(dictationDismissTimer.current);
          dictationDismissTimer.current = setTimeout(() => {
            setDictationMessage(null);
          }, 8000);
        },
      });
    }
    onStatusChange: (status) => {
      setSpeechStatus(status);
      if (status === "idle" || status === "error" || status === "unsupported") {
        setDictating(false);
        setInterimText("");
        if (status === "unsupported" && dictationRef.current) {
          const errMsg = dictationRef.current.getErrorMessage();
          if (errMsg) {
            setDictationMessage(errMsg);
            if (dictationDismissTimer.current) clearTimeout(dictationDismissTimer.current);
            dictationDismissTimer.current = setTimeout(() => {
              setDictationMessage(null);
            }, 8000);
          }
        }
      }
    }
  } else if (status === "listening" || status === "requesting-permission") {
    setDictating(true);
    setDictationMessage(null);
  }
}, []);
});

setDictating(true);

```

```

    setSpeechStatus("requesting-permission");
    const started = await dictationRef.current.start();
    if (!started) {
      setDictating(false);
      const errMsg = dictationRef.current.getErrorMessage();
      if (errMsg) {
        setDictationMessage(errMsg);
        if (dictationDismissTimer.current) clearTimeout(dictationDismissTimer.current);
        dictationDismissTimer.current = setTimeout(() => {
          setDictationMessage(null);
        }, 8000);
      }
    }
  }, [dictating]);

// WikiLink matches
const matchingBoards = useMemo(() => {
  if (wikiQuery === null) return [];
  const q = wikiQuery.toLowerCase();
  return boards.filter((b) => b.id !== activeBoardId && b.name.toLowerCase().includes(q)).slice(0, 5);
}, [wikiQuery, boards, activeBoardId]);

// Typing & WikiLink detection
const handleTextChange = (e: React.ChangeEvent<HTMLTextAreaElement>) => {
  const newText = e.target.value;
  // Manual typing invalidates the dictation revision anchor: a late
  // cumulative final must append now, never rewrite what was just typed.
  lastDictationRef.current = null;
  interimRangeRef.current = null;
  setSelRange({ start: e.target.selectionStart || 0, end: e.target.selectionEnd || 0 });
  setText(newText);
  const pos = e.target.selectionStart || 0;

  const textBeforeCursor = newText.slice(0, pos);
  const lastOpen = textBeforeCursor.lastIndexOf("[");
  const lastClose = textBeforeCursor.lastIndexOf("]");

  if (lastOpen !== -1 && lastOpen > lastClose) {
    const query = textBeforeCursor.slice(lastOpen + 2);
    if (!query.includes("\n") && query.length < 30) {
      setWikiQuery(query);
      setWikiCursorPos(lastOpen);
      setSelectedWikiIndex(0);
    } else {
      setWikiQuery(null);
    }
  } else {
    setWikiQuery(null);
  }

  if (activeElement) {
    // Deleted lines must not leave direction overrides pointing past them.
    const dirs = (activeElement as TextElement).lineDirs;
    const lineCount = newText.split("\n").length;
    const patch: Partial<TextElement> = { text: newText };
    if (dirs && dirs.length > lineCount) {
      const trimmed = dirs.slice(0, lineCount);
      while (trimmed.length > 0 && trimmed[trimmed.length - 1] === "auto") trimmed.pop();
      patch.lineDirs = trimmed;
    }
    mutateElement(patch);
    adjustHeight();
  }
};

const insertWikiLink = (boardName: string) => {
  // The LIVE textarea value, not React state: state can lag the last
  // keystroke, and slicing a stale string at a live cursor offset mangles
  // the line when typing fast and hitting Enter immediately.
  const ta = textareaRef.current;
  if (!ta || wikiQuery === null) return;
  const live = ta.value;
  const afterPos = ta.selectionStart || live.length;
  const before = live.slice(0, wikiCursorPos);
  const after = live.slice(afterPos);
  const replacement = `[[${boardName}]]`;
  const updated = before + replacement + after;

```

```

    setText(updated);
    setWikiQuery(null);
    if (activeElement) mutateElement({ text: updated });

    requestAnimationFrame(() => {
      if (textareaRef.current) {
        textareaRef.current.focus();
        const newPos = before.length + replacement.length;
        textareaRef.current.setSelectionRange(newPos, newPos);
        adjustHeight();
      }
    });
  };

  // Formatting actions
  const applyFormat = (prefix: string, suffix = "") => {
    const ta = textareaRef.current;
    if (!ta || !activeElement) return;
    const start = ta.selectionStart;
    const end = ta.selectionEnd;
    // Empty markers with the cursor parked between them (see
    // spliceFormatMarkers): no placeholder word to delete first.
    const { updated, cursor } = spliceFormatMarkers(text, start, end, prefix, suffix);
    setText(updated);
    mutateElement({ text: updated });

    requestAnimationFrame(() => {
      ta.focus();
      ta.setSelectionRange(cursor, cursor);
      adjustHeight();
    });
  };

  // Line prefix formatting (e.g. #, ##, - [ ], - ): every line the selection
  // touches (see prefixSelectedLines), so multi-line selections format whole.
  const applyLinePrefix = (prefix: string) => {
    const ta = textareaRef.current;
    if (!ta || !activeElement) return;
    const { updated, cursor } = prefixSelectedLines(text, ta.selectionStart, ta.selectionEnd, prefix);
    setText(updated);
    mutateElement({ text: updated });

    requestAnimationFrame(() => {
      ta.focus();
      ta.setSelectionRange(cursor, cursor);
      adjustHeight();
    });
  };

  // Case transforms
  const transformCase = (mode: "upper" | "lower" | "title" | "clean") => {
    const ta = textareaRef.current;
    if (!ta || !activeElement) return;
    const start = ta.selectionStart;
    const end = ta.selectionEnd;
    const isSelection = start !== end;
    const target = isSelection ? text.slice(start, end) : text;

    let transformed = target;
    if (mode === "upper") {
      transformed = target.toUpperCase();
    } else if (mode === "lower") {
      transformed = target.toLowerCase();
    } else if (mode === "title") {
      transformed = target.replace(/\b\w+/g, (txt) => txt.charAt(0).toUpperCase() + txt.slice(1).toLowerCase());
    } else if (mode === "clean") {
      transformed = target.replace(/[ \t]+/g, " ").replace(/\n{3}/g, "\n\n").trim();
    }

    const updated = isSelection
      ? text.slice(0, start) + transformed + text.slice(end)
      : transformed;

    setText(updated);
    mutateElement({ text: updated });

    requestAnimationFrame(() => {

```

```

    ta.focus();
    if (isSelection) {
      ta.setSelectionRange(start, start + transformed.length);
    }
    adjustHeight();
  });
};

const handleKeyDown = (e: React.KeyboardEvent<HTMLTextAreaElement>) => {
  // A word still being composed (the underlined word on Samsung / Gboard,
  // or an IME candidate) belongs to the keyboard. Rewriting the text here
  // from React state - which has not seen that word yet - deleted it.
  if (e.nativeEvent.isComposing) return;

  // 1. WikiLink Autocomplete navigation
  if (wikiQuery !== null && matchingBoards.length > 0) {
    if (e.key === "ArrowDown") {
      e.preventDefault();
      setSelectedWikiIndex((i) => (i + 1) % matchingBoards.length);
      return;
    }
    if (e.key === "ArrowUp") {
      e.preventDefault();
      setSelectedWikiIndex((i) => (i - 1 + matchingBoards.length) % matchingBoards.length);
      return;
    }
    if (e.key === "Enter" || e.key === "Tab") {
      e.preventDefault();
      // The list can shrink under a stale highlight; fall back to first.
      const pick = matchingBoards[selectedWikiIndex] ?? matchingBoards[0];
      if (pick) insertWikiLink(pick.name);
      return;
    }
    if (e.key === "Escape") {
      e.preventDefault();
      setWikiQuery(null);
      return;
    }
  }

  // 2. Intelligent List Continuation on Enter
  if (e.key === "Enter" && !e.shiftKey) {
    const ta = textareaRef.current;
    if (ta) {
      // The textarea, not React state: state can lag the last keystroke.
      const text = ta.value;
      const pos = ta.selectionStart;
      const textBefore = text.slice(0, pos);
      const lastNewline = textBefore.lastIndexOf("\n");
      const currentLine = textBefore.slice(lastNewline + 1);

      const { prefix, shouldExit } = getNextLinePrefix(currentLine);

      if (shouldExit) {
        // Erase empty prefix and leave clean empty line
        e.preventDefault();
        const lineStart = lastNewline + 1;
        const updated = text.slice(0, lineStart) + text.slice(pos);
        setText(updated);
        mutateElement({ text: updated });
        requestAnimationFrame(() => {
          ta.focus();
          ta.setSelectionRange(lineStart, lineStart);
          adjustHeight();
        });
        return;
      }

      if (prefix) {
        // Continue checklist, bullet, or numbered item
        e.preventDefault();
        const textAfter = text.slice(pos);
        const insertion = `\n${prefix}`;
        const updated = textBefore + insertion + textAfter;
        setText(updated);
        mutateElement({ text: updated });
        requestAnimationFrame(() => {
          ta.focus();

```

```
        const newPos = pos + insertion.length;
        ta.setSelectionRange(newPos, newPos);
        adjustHeight();
    });
    return;
}
}
}

// 3. Tab Indent / Shift+Tab Outdent
if (e.key === "Tab") {
    e.preventDefault();
    const ta = textareaRef.current;
    if (ta) {
        const start = ta.selectionStart;
        const end = ta.selectionEnd;
        if (e.shiftKey) {
            const textBefore = text.slice(0, start);
            const lastNewline = textBefore.lastIndexOf("\n");
            const lineStart = lastNewline + 1;
            if (text.slice(lineStart, lineStart + 2) === "  ") {
                const updated = text.slice(0, lineStart) + text.slice(lineStart + 2);
                setText(updated);
                mutateElement({ text: updated });
                requestAnimationFrame(() => {
                    ta.setSelectionRange(Math.max(lineStart, start - 2), Math.max(lineStart, end - 2));
                });
            }
        } else {
            const updated = text.slice(0, start) + "  " + text.slice(end);
            setText(updated);
            mutateElement({ text: updated });
            requestAnimationFrame(() => {
                ta.setSelectionRange(start + 2, start + 2);
            });
        }
        return;
    }
}

// 4. Keyboard Shortcuts
if (e.ctrlKey || e.metaKey) {
    if (e.shiftKey && e.code === "KeyR") {
        e.preventDefault();
        e.stopPropagation();
        toggleDictation();
        return;
    }
    if (e.shiftKey && e.code === "KeyL") {
        e.preventDefault();
        e.stopPropagation();
        applyLinePrefix("- [ ] ");
        return;
    }
    if (e.shiftKey && e.code === "KeyM") {
        e.preventDefault();
        e.stopPropagation();
        applyFormat("$", "$");
        return;
    }
    if (e.shiftKey && (e.code === "KeyX" || e.code === "KeyS")) {
        e.preventDefault();
        e.stopPropagation();
        applyFormat("~~", "~~");
        return;
    }
    if (e.code === "KeyB") {
        e.preventDefault();
        e.stopPropagation();
        applyFormat("***", "***");
        return;
    }
    if (e.code === "KeyI") {
        e.preventDefault();
        e.stopPropagation();
        applyFormat("*", "*");
        return;
    }
}
```

```

    if (e.code === "KeyU") {
      e.preventDefault();
      e.stopPropagation();
      applyFormat("<u>", "</u>");
      return;
    }
    if (e.code === "KeyK") {
      e.preventDefault();
      e.stopPropagation();
      applyFormat("[[", "]]");
      return;
    }
  }

  if (e.key === "Escape") {
    exitEditing();
  }
};

const returnFocus = (e: Event) => {
  e.preventDefault();
  textareaRef.current?.focus({ preventScroll: true });
};

if (!activeElement || (activeElement.type !== "text" && activeElement.type !== "sticky")) {
  return null;
}

const isSticky = activeElement.type === "sticky";
const textEl = activeElement as TextElement;
const stickyEl = activeElement as StickyElement;
const fontSize = isSticky ? stickyEl.fontSize : textEl.fontSize;
const align = (isSticky ? stickyEl.align : textEl.align) || "left";
// Per-line writing direction: the override on the cursor's line, if any,
// else what auto-detection reads from that line's own words.
const lineDirs = (isSticky ? stickyEl.lineDirs : textEl.lineDirs) ?? [];
const cursorLine = selectedLineRange(text, selRange.start, selRange.end).first;
const cursorLineDir = resolveLineDir(lineDirs, cursorLine);
const cursorLineText = text.split("\n")[cursorLine] ?? "";
const effectiveLineDir = cursorLineDir === "auto" ? detectLineDir(cursorLineText) : cursorLineDir;
const applyLineDir = (dir: LineDir) => {
  const next = setLineDirs(lineDirs, text, selRange.start, selRange.end, dir);
  mutateElement({ lineDirs: next.length > 0 ? next : undefined });
};
const opacity = activeElement.opacity ?? 1;
const id = activeElement.id;

// Card mode state
const isCard = !isSticky && !(textEl.cardMode || (textEl.backgroundColor && textEl.backgroundColor !== "transparent"));

// Word & character stats
const wordCount = text.trim() ? text.trim().split(/\s+/).length : 0;
const charCount = text.length;

// Screen coordinates
const screenPos = worldToScreen(activeElement.x, activeElement.y, camera, viewport);
const screenWidth = activeElement.width * camera.zoom;
const screenHeight = Math.max(activeElement.height * camera.zoom, 40);
const stickyStyle = isSticky ? STICKY_COLORS[stickyEl.color] : null;

// Bar placement
const barW = Math.min(barSize.width || 0, viewport.width - EDGE * 2);
const barLeft = Math.max(EDGE, Math.min(screenPos.x, viewport.width - barW - EDGE));
const above = screenPos.y - barSize.height - GAP;
const below = screenPos.y + screenHeight + GAP;
const barTop =
  above >= EDGE
    ? above
    : below + barSize.height <= viewport.height - EDGE
    ? below
    : EDGE;

const iconBtn =
  "flex h-8 min-w-8 shrink-0 items-center justify-center rounded-lg px-1.5 text-muted-foreground transition-colors hover:bg-muted hover:text-foreground";
const divider = <span className="mx-0.5 h-5 w-px shrink-0 bg-border/70" aria-hidden />;

```

```

return (
  <TooltipProvider delayDuration={250}>
    { /* 1. Main Floating Formatting Bar */ }
    <div
      ref={barRef}
      data-wb-text-bar
      className="absolute z-50 flex items-center gap-0.5 overflow-x-auto rounded-2xl border border-border/80
        bg-white/95 p-1 text-xs text-foreground shadow-2xl backdrop-blur-xl select-none dark:bg-zinc-900/
        95"
      style={{ left: barLeft, top: barTop, maxWidth: viewport.width - EDGE * 2 }}
      onMouseDown={(e) => e.preventDefault()}
      onPointerDown={(e) => e.stopPropagation()}
    >
      { /* Font family */ }
      { !isSticky && (
        <Popover>
          <PopoverTrigger asChild>
            <button className={cn(iconBtn, "gap-1 px-2 font-medium text-foreground")} title="Font family">
              <span className="max-w-[70px] truncate">
                {FONT_FAMILIES.find((f) => f.value === textEl.fontFamily)?.label.split(" ")[0] || "Font"}
              </span>
              <ChevronDown size={11} className="opacity-60" />
            </button>
          </PopoverTrigger>
          <PopoverContent className="w-48 p-1 text-xs" align="start" onCloseAutoFocus={returnFocus}>
            {FONT_FAMILIES.map((ff) => (
              <button
                key={ff.id}
                onClick={() => applyStyle({ fontFamily: ff.value })}
                className="flex w-full items-center justify-between rounded px-2.5 py-1.5 text-left hover:bg
                -muted"
                style={{ fontFamily: ff.value }}
              >
                <span>{ff.label}</span>
                {textEl.fontFamily === ff.value && <Check size={12} className="text-primary" />}
              </button>
            ))}
          </PopoverContent>
        </Popover>
      )}

      { /* Font size stepper */ }
      <StepButton
        direction={-1}
        value={fontSize}
        min={MIN_FONT}
        max={MAX_FONT}
        onChange={(v) => applyStyle({ fontSize: v })}
        className="h-8 w-7 shrink-0"
      />

      <Popover>
        <PopoverTrigger asChild>
          <button className={cn(iconBtn, "px-1 font-semibold tabular-nums text-foreground")} title="Font siz
            e">
            {fontSize}px
          </button>
        </PopoverTrigger>
        <PopoverContent side="top" className="w-56 space-y-2.5 p-3" onCloseAutoFocus={returnFocus}>
          <input
            type="range"
            min={MIN_FONT}
            max={MAX_FONT}
            step={1}
            value={fontSize}
            aria-label="Font size"
            onChange={(e) => applyStyle({ fontSize: Number(e.target.value) })}
            className="h-2 w-full cursor-pointer appearance-none rounded-lg bg-muted accent-primary"
          />
          <div className="grid grid-cols-4 gap-1">
            {SIZE_PRESETS.map((sz) => (
              <button
                key={sz}
                onClick={() => applyStyle({ fontSize: sz })}
                className={cn(
                  "rounded border py-1.5 text-[11px] transition-colors",
                  fontSize === sz
                    ? "border-primary bg-primary font-semibold text-primary-foreground"
                    : "border-border text-muted-foreground hover:bg-muted"
                )}
              />
            ))}
          </div>
        </PopoverContent>
      </Popover>
    </div>
  </TooltipProvider>
)

```

```

    })
    >
    {sz}
  </button>
  )))
</div>
</PopoverContent>
</Popover>
<StepButton
  direction={1}
  value={fontSize}
  min={MIN_FONT}
  max={MAX_FONT}
  onChange={(v) => applyStyle({ fontSize: v })}
  className="h-8 w-7 shrink-0"
/>

{divider}

{/* Typography: Bold, Italic, Underline, Strikethrough, Code */}
<Tooltip>
  <TooltipTrigger asChild>
    <button onClick={() => applyFormat("****", "****")} className={iconBtn} aria-label="Bold (Ctrl+B)">
      <Bold size={14} />
    </button>
  </TooltipTrigger>
  <TooltipContent side="top">Bold (Ctrl+B)</TooltipContent>
</Tooltip>

<Tooltip>
  <TooltipTrigger asChild>
    <button onClick={() => applyFormat("~~", "~~")} className={iconBtn} aria-label="Italic (Ctrl+I)">
      <Italic size={14} />
    </button>
  </TooltipTrigger>
  <TooltipContent side="top">Italic (Ctrl+I)</TooltipContent>
</Tooltip>

<Tooltip>
  <TooltipTrigger asChild>
    <button onClick={() => applyFormat("<u>", "</u>")} className={iconBtn} aria-label="Underline (Ctrl+U)">
      <Underline size={14} />
    </button>
  </TooltipTrigger>
  <TooltipContent side="top">Underline (Ctrl+U)</TooltipContent>
</Tooltip>

<Tooltip>
  <TooltipTrigger asChild>
    <button onClick={() => applyFormat("~~~", "~~~")} className={iconBtn} aria-label="Strikethrough">
      <Strikethrough size={14} />
    </button>
  </TooltipTrigger>
  <TooltipContent side="top">Strikethrough</TooltipContent>
</Tooltip>

<Tooltip>
  <TooltipTrigger asChild>
    <button onClick={() => applyFormat("`", "`")} className={iconBtn} aria-label="Inline Code">
      <Code size={14} />
    </button>
  </TooltipTrigger>
  <TooltipContent side="top">Inline Code</TooltipContent>
</Tooltip>

{divider}

{/* Headings Popover */}
<Popover>
  <PopoverTrigger asChild>
    <button className={cn(iconBtn, "gap-0.5 px-1.5")} title="Heading Style">
      <Heading1 size={14} />
      <ChevronDown size={10} className="opacity-60" />
    </button>
  </PopoverTrigger>
  <PopoverContent className="w-40 p-1 text-xs" align="start" onCloseAutoFocus={returnFocus}>
    <button

```

```

      onClick={() => applyLinePrefix("# ")}
      className="flex w-full items-center gap-2 rounded px-2 py-1.5 hover:bg-muted font-bold text-base
    "
  >
    <Heading1 size={16} /> Heading 1
  </button>
  <button
    onClick={() => applyLinePrefix("## ")}
    className="flex w-full items-center gap-2 rounded px-2 py-1.5 hover:bg-muted font-semibold text-sm"
  >
    <Heading2 size={15} /> Heading 2
  </button>
  <button
    onClick={() => applyLinePrefix("### ")}
    className="flex w-full items-center gap-2 rounded px-2 py-1.5 hover:bg-muted font-medium text-xs
    "
  >
    <Heading3 size={14} /> Heading 3
  </button>
</PopoverContent>
</Popover>

{/* Lists & Tasks Popover */}
<Popover>
  <PopoverTrigger asChild>
    <button className={cn(iconBtn, "gap-0.5 px-1.5")} title="Lists & Tasks">
      <CheckSquare size={14} />
      <ChevronDown size={10} className="opacity-60" />
    </button>
  </PopoverTrigger>
  <PopoverContent className="w-44 p-1 text-xs" align="start" onCloseAutoFocus={returnFocus}>
    <button
      onClick={() => applyLinePrefix("- [ ] ")}
      className="flex w-full items-center gap-2 rounded px-2 py-1.5 hover:bg-muted"
    >
      <CheckSquare size={14} className="text-primary" /> Task Checklist
    </button>
    <button
      onClick={() => applyLinePrefix("- ")}
      className="flex w-full items-center gap-2 rounded px-2 py-1.5 hover:bg-muted"
    >
      <List size={14} /> Bullet List
    </button>
    <button
      onClick={() => applyLinePrefix("1. ")}
      className="flex w-full items-center gap-2 rounded px-2 py-1.5 hover:bg-muted"
    >
      <ListOrdered size={14} /> Numbered List
    </button>
    <button
      onClick={() => applyLinePrefix("> ")}
      className="flex w-full items-center gap-2 rounded px-2 py-1.5 hover:bg-muted"
    >
      <Quote size={14} /> Blockquote
    </button>
  </PopoverContent>
</Popover>

{divider}

{/* SMART TOOLS: Voice Dictation (Record to Text), WikiLink, LaTeX Math */}
<Popover>
  <PopoverTrigger asChild>
    <button
      className={cn(iconBtn, "gap-0.5 px-1.5 font-semibold")}
      title="Dictation language"
      aria-label={`Dictation language: ${dictationLang}`}
    >
      <Languages size={14} />
      <span className="text-[10px] leading-none">{dictationLang.split("-")[0]?.toUpperCase() ?? "EN"}<
        /span>
      <ChevronDown size={10} className="opacity-60" />
    </button>
  </PopoverTrigger>
  <PopoverContent className="w-44 p-1 text-xs" align="start" onCloseAutoFocus={returnFocus}>
    <div className="px-2 py-1 text-[10px] font-semibold uppercase tracking-wider text-muted-foreground
    ">

```

```

    Dictation language
  </div>
  {SUPPORTED_DICTATION_LANGS.map((l) => {
    const active = resolveDictationLang(dictationLang) === l.id;
    return (
      <button
        key={l.id}
        onClick={() => pickDictationLang(l.id)}
        className={cn(
          "flex w-full items-center gap-2 rounded px-2 py-1.5 hover:bg-muted",
          active && "font-semibold text-primary"
        )}
        aria-pressed={active}
      >
        <span className="flex-1 text-left">{l.label}</span>
        {active && <Check size={12} />}
      </button>
    );
  })}
</PopoverContent>
</Popover>

<Tooltip>
  <TooltipTrigger asChild>
    <button
      onClick={toggleDictation}
      className={cn(
        "flex h-8 shrink-0 items-center justify-center rounded-lg px-2 transition-all font-medium text-
        -xs gap-1",
        dictating
          ? "bg-red-500 text-white shadow-md animate-pulse hover:bg-red-600"
          : "text-muted-foreground hover:bg-muted hover:text-foreground"
      )}
      aria-label={dictating ? "Stop Voice Dictation" : "Record to Text (Voice Dictation)"}
    >
      {dictating ? (
        speechStatus === "requesting-permission" ? (
          <Loader2 size={14} className="animate-spin" />
        ) : (
          <MicOff size={14} className="animate-pulse" />
        )
      ) : (
        <Mic size={14} />
      )}
      <span className="hidden sm:inline">
        {dictating
          ? speechStatus === "requesting-permission"
            ? "Allow Mic..."
            : "Listening..."
          : "Dictate"}
      </span>
    </button>
  </TooltipTrigger>
  <TooltipContent side="top">
    {dictating
      ? speechStatus === "requesting-permission"
        ? "Requesting microphone permission..."
        : "Listening – Click to stop (Ctrl+Shift+R)"
      : "Record to Text – Speak into note (Ctrl+Shift+R)"}
  </TooltipContent>
</Tooltip>

<Tooltip>
  <TooltipTrigger asChild>
    <button
      onClick={() => applyFormat("[", " ")}
      className={cn(iconBtn, "text-primary hover:text-primary")}
      aria-label="Insert [[WikiLink]] (Ctrl+K)"
    >
      <Link2 size={14} />
    </button>
  </TooltipTrigger>
  <TooltipContent side="top">Insert [[WikiLink]] (Ctrl+K)</TooltipContent>
</Tooltip>

<Tooltip>
  <TooltipTrigger asChild>
    <button

```

```

      onClick={() => applyFormat("$", "$")}
      className={iconBtn}
      aria-label="LaTeX Math Formula"
    >
      <span className="font-serif italic font-semibold text-xs">fx</span>
    </button>
  </TooltipTrigger>
  <TooltipContent side="top">LaTeX Math ($...$)</TooltipContent>
</Tooltip>

{/* Card Mode & Styling */}
{!isSticky && (
  <Popover>
    <PopoverTrigger asChild>
      <button
        className={cn(
          iconBtn,
          "gap-0.5 px-1.5",
          isCard && "bg-primary/10 text-primary font-medium"
        )}
        title="Card box styling & background"
      >
        <Square size={14} />
        <ChevronDown size={10} className="opacity-60" />
      </button>
    </PopoverTrigger>
    <PopoverContent className="w-56 space-y-3 p-3 text-xs" align="center" onCloseAutoFocus={returnFocus}>
      <div className="flex items-center justify-between">
        <span className="font-medium text-foreground">Card Box Mode</span>
        <button
          onClick={() => {
            const nextMode = !isCard;
            mutateElement({
              cardMode: nextMode,
              backgroundColor: nextMode ? textEl.backgroundColor || "#ffffff" : "transparent",
              borderColor: nextMode ? textEl.borderColor || "#e2e8f0" : "transparent",
              borderRadius: nextMode ? textEl.borderRadius || 8 : 0,
              padding: nextMode ? textEl.padding || 12 : 4,
            });
          }}
          className={cn(
            "h-5 w-9 rounded-full transition-colors relative",
            isCard ? "bg-primary" : "bg-muted"
          )}
        >
          <span
            className={cn(
              "absolute top-0.5 h-4 w-4 rounded-full bg-white transition-transform shadow-sm",
              isCard ? "right-0.5" : "left-0.5"
            )}
          />
        </button>
      </div>

      {isCard && (
        <>
          <div className="space-y-1.5">
            <span className="text-[10px] font-semibold text-muted-foreground uppercase">Card Theme</span>
          </div>
          <div className="grid grid-cols-4 gap-1.5">
            {CARD_BACKGROUNDDS.filter((c) => c.id !== "transparent").map((c) => (
              <button
                key={c.id}
                onClick={() => {
                  mutateElement({
                    backgroundColor: c.bg,
                    borderColor: c.border,
                  });
                }}
                className={cn(
                  "h-7 rounded-md border text-[10px] font-medium flex items-center justify-center transition-all",
                  textEl.backgroundColor === c.bg
                    ? "ring-2 ring-primary ring-offset-1 border-primary"
                    : "border-border hover:opacity-90"
                )}
                style={{ backgroundColor: c.bg, color: c.id === "dark" ? "#ffffff" : "#1e293b" }}
              >

```

```

        title={c.label}
      >
        {c.label.split(" ")[0]}
      </button>
    )}}
  </div>
</div>

<div className="space-y-1">
  <span className="text-[10px] font-semibold text-muted-foreground uppercase">Corners</span>
  <div className="flex gap-1.5">
    {[
      { label: "Sharp", radius: 2 },
      { label: "Smooth", radius: 8 },
      { label: "Rounded", radius: 14 },
    ].map((r) => (
      <button
        key={r.label}
        onClick={() => mutateElement({ borderRadius: r.radius })}
        className={cn(
          "flex-1 rounded border py-1 text-[10px] transition-colors",
          (textEl.borderRadius || 8) === r.radius
            ? "bg-primary text-primary-foreground border-primary"
            : "border-border hover:bg-muted"
        )}
      >
        {r.label}
      </button>
    )]}
  </div>
</div>
</>
  )}
</PopoverContent>
</Popover>
)}

{divider}

{/* Alignment */}
{(["left", "center", "right"] as TextAlign[]).map((a) => (
  <button
    key={a}
    onClick={() => mutateElement({ align: a })}
    className={cn(iconBtn, align === a && "bg-muted text-primary")}
    title={`Align ${a}`}
    aria-label={`Align ${a}`}
  >
    {a === "left" && <AlignLeft size={14} />}
    {a === "center" && <AlignCenter size={14} />}
    {a === "right" && <AlignRight size={14} />}
  </button>
)}

{/* Line direction: one line RTL Hebrew, the next LTR English */}
{(["rtl", "auto", "ltr"] as LineDir[]).map((d) => (
  <button
    key={d}
    onClick={() => applyLineDir(d)}
    className={cn(iconBtn, "px-2 text-[11px] font-semibold", cursorLineDir === d && "bg-muted text-primary")}
    title={
      d === "auto"
        ? `Direction: auto (this line reads ${effectiveLineDir.toUpperCase()})`
        : `Force this line ${d.toUpperCase()}`
    }
    aria-label={d === "auto" ? "Line direction auto" : `Force line ${d.toUpperCase()}`}
    aria-pressed={cursorLineDir === d}
  >
    {d === "auto" ? "Auto" : d.toUpperCase()}
  </button>
)}

{divider}

{/* Text Color & Opacity */}
<Popover>
  <PopoverTrigger asChild>

```

```

<button className={cn(iconBtn, "gap-1")} title="Text colour and opacity">
  <span
    className="h-4 w-4 rounded-full border border-black/15 shadow-sm dark:border-white/20"
    style={{ backgroundColor: isSticky ? stickyStyle?.bg : textEl.color, opacity }}
  />
  <ChevronDown size={10} className="opacity-60" />
</button>
</PopoverTrigger>
<PopoverContent className="w-60 space-y-3 p-3" onCloseAutoFocus={returnFocus}>
  <div className="text-[10px] font-semibold text-muted-foreground uppercase tracking-wider">
    {isSticky ? "Note Colour" : "Ink Colour"}
  </div>
  <div className="grid grid-cols-6 gap-2">
    {isSticky
      ? Object.entries(STICKY_COLORS).map(([key, c]) => (
        <button
          key={key}
          onClick={() => mutateElement({ color: key as StickyColor })}
          title={c.label}
          className={cn(
            "h-7 w-7 rounded-full border",
            stickyEl.color === key ? "ring-2 ring-primary ring-offset-1" : "border-border"
          )}
          style={{ backgroundColor: c.bg }}
        />
      ))
      : PEN_COLORS.map((c) => (
        <button
          key={c}
          onClick={() => applyStyle({ color: c })}
          title={c}
          className={cn(
            "h-7 w-7 rounded-full border border-black/10 transition-transform hover:scale-105",
            textEl.color?.toLowerCase() === c.toLowerCase() && "ring-2 ring-primary ring-offset-1"
          )}
          style={{ backgroundColor: c }}
        />
      ))
    }
  </div>

  {!isSticky && (
    <label className="flex items-center justify-between gap-2 text-[11px] text-muted-foreground">
      Custom colour
      <input
        type="color"
        value={/^#[0-9a-f]{6}$/i.test(textEl.color || "") ? textEl.color : "#1e293b"}
        onChange={(e) => applyStyle({ color: e.target.value })}
        className="h-6 w-9 cursor-pointer rounded border border-border bg-transparent"
      />
    </label>
  )}

  <div className="space-y-1.5">
    <div className="flex items-center justify-between text-[11px] text-muted-foreground">
      <span>Opacity</span>
      <span className="tabular-nums font-semibold text-foreground">{Math.round(opacity * 100)}%</span>
    </div>
    <input
      type="range"
      min={10}
      max={100}
      step={5}
      value={Math.round(opacity * 100)}
      aria-label="Opacity"
      onChange={(e) => mutateElement({ opacity: Number(e.target.value) / 100 })}
      className="h-2 w-full cursor-pointer appearance-none rounded-lg bg-muted accent-primary"
    />
  </div>
</PopoverContent>
</Popover>

/* More Actions Popover: Transforms, Arrange, Lock, Delete */
<Popover>
  <PopoverTrigger asChild>
    <button className={iconBtn} title="More actions">
      <MoreHorizontal size={15} />
    </button>

```

```

</PopoverTrigger>
<PopoverContent className="w-52 p-1.5 text-xs space-y-1" align="end" onCloseAutoFocus={returnFocus}>
  <div className="px-2 py-1 text-[10px] font-semibold text-muted-foreground uppercase">Case & Clean</div>
  <div className="grid grid-cols-3 gap-1 px-1">
    <button
      onClick={() => transformCase("upper")}
      className="rounded border py-1 text-[10px] hover:bg-muted"
      title="Convert to UPPERCASE"
    >
      UPPER
    </button>
    <button
      onClick={() => transformCase("lower")}
      className="rounded border py-1 text-[10px] hover:bg-muted"
      title="Convert to lowercase"
    >
      lower
    </button>
    <button
      onClick={() => transformCase("title")}
      className="rounded border py-1 text-[10px] hover:bg-muted"
      title="Convert to Title Case"
    >
      Title
    </button>
  </div>
  <button
    onClick={() => transformCase("clean")}
    className="flex w-full items-center gap-2 rounded px-2.5 py-1.5 hover:bg-muted text-[11px]"
  >
    <Sparkles size={13} className="text-primary" /> Clean extra spaces
  </button>

  <div className="my-1 border-t border-border" />
  <div className="px-2 py-0.5 text-[10px] font-semibold text-muted-foreground uppercase">Layering</div>
  <button
    className="flex w-full items-center gap-2 rounded px-2.5 py-1.5 hover:bg-muted text-[11px]"
    onClick={() => bringToFront([id])}
  >
    <ArrowUpToLine size={13} /> Bring to front
  </button>
  <button
    className="flex w-full items-center gap-2 rounded px-2.5 py-1.5 hover:bg-muted text-[11px]"
    onClick={() => sendToBack([id])}
  >
    <ArrowDownToLine size={13} /> Send to back
  </button>
  <button
    className="flex w-full items-center gap-2 rounded px-2.5 py-1.5 hover:bg-muted text-[11px]"
    onClick={() => {
      const [copy] = duplicateElements([id]);
      if (copy) setEditingId(copy);
    }}
  >
    <Copy size={13} /> Duplicate
  </button>
  <button
    className="flex w-full items-center gap-2 rounded px-2.5 py-1.5 hover:bg-muted text-[11px]"
    onClick={() => {
      toggleLock([id]);
      exitEditing();
    }}
  >
    <Lock size={13} /> Lock
  </button>
  <div className="my-1 border-t border-border" />
  <button
    className="flex w-full items-center gap-2 rounded px-2.5 py-1.5 text-red-600 hover:bg-red-500/10 dark:text-red-400 text-[11px]"
    onClick={() => {
      deleteElements([id]);
      exitEditing();
    }}
  >
    <Trash2 size={13} /> Delete
  </button>

```

```

        <div className="mt-1 border-t border-border pt-1 px-2 text-[10px] text-muted-foreground flex justify-between">
          <span>{wordCount} words</span>
          <span>{charCount} chars</span>
        </div>
      </PopoverContent>
    </Popover>

    { /* Done Button */ }
    <button
      onClick={exitEditing}
      className="ml-1 h-8 shrink-0 rounded-xl bg-primary px-3 text-xs font-semibold text-primary-foreground shadow-sm transition-opacity hover:opacity-90 active:scale-95"
    >
      Done
    </button>
  </div>

  { /* 2. Live Dictation Floating Waveform / Status Banner */ }
  {dictating && speechStatus === "requesting-permission" && (
    <div
      className="absolute z-50 flex items-center gap-2 px-3 py-1.5 bg-amber-500/15 border border-amber-500/40 text-amber-700 dark:text-amber-300 rounded-full text-xs font-medium shadow-lg backdrop-blur-md animate-in fade-in slide-in-from-top-1 select-none"
      style={{
        left: barLeft,
        top: Math.max(EDGE, barTop - 36),
      }}
    >
      <Loader2 size={13} className="animate-spin text-amber-600 dark:text-amber-400" />
      <span>Requesting microphone permission...</span>
    </div>
  )}

  {dictating && speechStatus === "listening" && (
    <div
      className="absolute z-50 flex items-center gap-2.5 px-3 py-1.5 bg-red-500/15 border border-red-500/40 text-red-600 rounded-full text-xs font-medium shadow-lg backdrop-blur-md animate-in fade-in slide-in-from-top-1 select-none dark:text-red-400"
      style={{
        left: barLeft,
        top: Math.max(EDGE, barTop - 36),
      }}
    >
      <span className="relative flex h-2.5 w-2.5">
        <span className="animate-ping absolute inline-flex h-full w-full rounded-full bg-red-400 opacity-75" />
        <span className="relative inline-flex rounded-full h-2.5 w-2.5 bg-red-500" />
      </span>
      <span className="max-w-[240px] sm:max-w-md truncate">
        {interimText ? "Hearing you..." : "Listening... speak into note"}
      </span>
      <button
        onClick={() => {
          dictationRef.current?.stop();
          setDictating(false);
          setInterimText("");
          setSpeechStatus("idle");
        }}
        className="ml-1 rounded-full bg-red-500/20 px-2 py-0.5 text-[10px] font-semibold hover:bg-red-500/30 transition-colors"
      >
        Stop
      </button>
    </div>
  )}

  { !dictating && dictationMessage && (
    <div
      className="absolute z-50 flex items-center gap-2 px-3 py-2 bg-amber-500/15 border border-amber-500/40 text-amber-800 dark:text-amber-200 rounded-2xl text-xs shadow-xl backdrop-blur-md animate-in fade-in slide-in-from-top-1 max-w-sm sm:max-w-md"
      style={{
        left: barLeft,
        top: Math.max(EDGE, barTop - 42),
      }}
    >

```

```

<AlertCircle size={14} className="shrink-0 text-amber-600 dark:text-amber-400" />
<span className="flex-1 text-[11px] leading-snug">{dictationMessage}</span>
<button
  onClick={() => setDictationMessage(null)}
  className="shrink-0 rounded p-1 hover:bg-amber-500/20 text-muted-foreground hover:text-foreground"
  aria-label="Dismiss notice"
>
  <X size={12} />
</button>
</div>
)}

{/* 3. The Text Box Area */}
<div
  ref={boxRef}
  className="absolute pointer-events-auto z-40"
  style={{
    left: `${screenPos.x}px`,
    top: `${screenPos.y}px`,
    width: `${screenWidth}px`,
    transformOrigin: "top left",
    transform: activeElement.rotation ? `rotate(${(activeElement.rotation * 180) / Math.PI}deg)` : undefined,
  }}
  onMouseDown={(e) => e.stopPropagation()}
  onPointerDown={(e) => e.stopPropagation()}
  onClick={(e) => e.stopPropagation()}
>
  <textarea
    ref={textareaRef}
    value={text}
    // dir=auto: each line follows its own first strong character, so
    // Hebrew and English lines share one box while editing too.
    dir="auto"
    onChange={handleTextChange}
    onSelect={(e) => {
      const t = e.target as HTMLTextAreaElement;
      setSelRange({ start: t.selectionStart || 0, end: t.selectionEnd || 0 });
    }}
    onKeyDown={handleKeyDown}
    placeholder={
      isSticky
        ? "Write a sticky note..."
        : "Type your thoughts... (use [[WikiLinks]], - [ ] tasks, or tap Dictate to speak)"
    }
  >
    <div
      style={{
        width: "100%",
        minHeight: `${screenHeight}px`,
        backgroundColor: isSticky
          ? stickyStyle?.bg
          : isCard
          ? textEl.backgroundColor || "#ffffff"
          : "transparent",
        color: isSticky ? stickyStyle?.text : textEl.color || "#1e293b",
        opacity,
        border: isSticky
          ? `2px solid ${stickyStyle?.border}`
          : isCard
          ? `2px solid ${textEl.borderColor || "#e2e8f0"}`
          : `2px dashed #6366f1`,
        borderRadius: isSticky ? "8px" : isCard ? `${textEl.borderRadius || 8}px` : "4px",
        padding: isSticky
          ? `${14 * camera.zoom}px`
          : isCard
          ? `${(textEl.padding || 12) * camera.zoom}px`
          : `${4 * camera.zoom}px ${8 * camera.zoom}px`,
        fontSize: `${fontSize * camera.zoom}px`,
        fontFamily: isSticky
          ? "Inter, system-ui, sans-serif"
          : textEl.fontFamily || "Inter, system-ui, sans-serif",
        fontWeight: isSticky ? 400 : textEl.fontWeight || 400,
        textAlign: align,
        lineHeight: textEl.lineHeight || 1.45,
        outline: "none",
        resize: "none",
        overflow: "hidden",
        boxShadow: isSticky
          ? "0 4px 16px raba(0.0.0.0.12)"
          : isCard
          ? textEl.shadow || "0 2px 4px raba(0.0.0.0.12)"
          : "none",
      }}
    >
      {text}
    </div>
  </textarea>
</div>
)
}

```

```

      : isCard
      ? "0 4px 20px rgba(0,0,0,0.12)"
      : "0 0 0 4px rgba(99,102,241,0.15)",
      touchAction: "manipulation",
      boxSizing: "border-box",
    })
  />

  { /* Live [[WikiLink]] autocomplete: stays visible while the query has
      no match yet, so typing never looks dead – it narrows with every
      character instead of blinking out. */ }
  { wikiQuery !== null && (
    <div className="absolute left-0 top-full z-50 mt-1.5 w-64 rounded-xl border border-border bg-white/9
      8 p-1 shadow-2xl backdrop-blur-md animate-in fade-in zoom-in-95 dark:bg-zinc-900/98">
      <div className="px-2 py-1 text-[10px] font-semibold uppercase tracking-wider text-muted-foreground
        ">
        Link to Board
      </div>
      {matchingBoards.length > 0 ? (
        matchingBoards.map((b, idx) => (
          <button
            key={b.id}
            onClick={() => insertWikiLink(b.name)}
            className={cn(
              "flex w-full items-center justify-between rounded-lg px-2.5 py-1.5 text-left text-xs trans
              ition-colors",
              idx === selectedWikiIndex
                ? "bg-primary font-medium text-primary-foreground"
                : "text-foreground hover:bg-muted"
            )}
          >
            <span className="truncate">{b.name}</span>
            <span className="text-[10px] opacity-70">[[{b.name}]]</span>
          </button>
        )}
      ) : (
        <div className="px-2.5 py-1.5 text-xs text-muted-foreground">
          No boards match "{wikiQuery}" – keep typing
        </div>
      )}
    </div>
  )}
</div>
</TooltipProvider>
);
}

```

```

=====
// === END FILE: src/components/whiteboard/InlineTextEditor.tsx ===
=====

```

```

=====
// === START FILE: src/components/whiteboard/InsertPagesModal.tsx ===
// File 58 of 194 | Lines: 438 | Size: 17647 bytes
=====

```

```
"use client";
```

```

/**
 * Insert pages into the notebook you are in.
 *
 * Two sources, one flow, because the question is the same either way: WHICH
 * pages, from WHERE, and WHERE DO THEY GO.
 *
 * - another notebook or whiteboard already in the app, found by name;
 * - a PDF file from the device.
 *
 * The three steps stay on one screen rather than becoming a wizard: picking
 * "pages 2-5 of last year's notes, after page 12" is one thought, and a wizard
 * would make you commit to the source before it tells you how many pages it
 * even has.
 *
 * Everything COPIES. The source notebook is never modified, so a mistake here
 * costs a delete rather than work you cannot get back.
 */

```

```
import React from "react";
```

```

import { FileText, NotebookText, Search, X, Layers, AlertTriangle } from "lucide-react";
import { useWhiteboard, useActiveBoard } from "@store/whiteboard";
import { getPages, isNotebook, pageSizeOf, resolvePageRange } from "@lib/whiteboard/notebook";
import type { Board } from "@lib/whiteboard/types";
import {
  Dialog,
  DialogContent,
  DialogDescription,
  DialogHeader,
  DialogTitle,
} from "@components/ui/dialog";
import { Button } from "@components/ui/button";
import { cn } from "@lib/utils";

interface InsertPagesModalProps {
  open: boolean;
  onClose: () => void;
}

/** A chosen source: a board in the app, or a PDF picked from the device. */
type Source =
  | { kind: "board"; board: Board; pageCount: number }
  | { kind: "pdf"; file: File; pageCount: number };

/** Where the pages go, expressed the way people say it out loud. */
type Placement = "start" | "before" | "after" | "end";

const PLACEMENTS: Array<{ id: Placement; label: string }> = [
  { id: "start", label: "At the beginning" },
  { id: "before", label: "Before this page" },
  { id: "after", label: "After this page" },
  { id: "end", label: "At the end" },
];

/** A number input that tolerates being empty while you type. */
function PageNumberInput({
  value,
  onChange,
  max,
  label,
}: {
  value: string;
  onChange: (v: string) => void;
  max: number;
  label: string;
}) {
  return (
    <input
      type="number"
      inputMode="numeric"
      min={1}
      max={max}
      value={value}
      aria-label={label}
      onChange={(e) => onChange(e.target.value)}
      className="w-16 h-8 rounded-lg border border-border bg-background px-2 text-center text-sm tabular-nums
        focus:border-primary focus:outline-none"
    />
  );
}

export function InsertPagesModal({ open, onClose }: InsertPagesModalProps) {
  const activeBoard = useActiveBoard();
  const boards = useWhiteboard((s) => s.boards);
  const insertPagesFromBoard = useWhiteboard((s) => s.insertPagesFromBoard);
  const insertPdfPages = useWhiteboard((s) => s.insertPdfPages);

  const [query, setQuery] = React.useState("");
  const [source, setSource] = React.useState<Source | null>(null);
  const [from, setFrom] = React.useState("1");
  const [to, setTo] = React.useState("1");
  const [placement, setPlacement] = React.useState<Placement>("after");
  const [busy, setBusy] = React.useState(false);
  const [error, setError] = React.useState<string | null>(null);
  const [done, setDone] = React.useState<string | null>(null);
  const fileInputRef = React.useRef<HTMLInputElement | null>(null);

  const targetPages = getPages(activeBoard);

```

```

const activeIndex = Math.max(
  0,
  targetPages.findIndex((p) => p.id === activeBoard?.activePageId)
);

// A fresh dialog every time: a stale source from last week's insert is far
// more confusing than re-picking it.
React.useEffect(() => {
  if (!open) return;
  setQuery("");
  setSource(null);
  setFrom("1");
  setTo("1");
  setPlacement("after");
  setError(null);
  setDone(null);
  setBusy(false);
}, [open]);

/** Choosing a source pre-fills the range to "all of it" - the common case. */
const chooseSource = (s: Source) => {
  setSource(s);
  setFrom("1");
  setTo(String(s.pageCount));
  setError(null);
  setDone(null);
};

const candidates = React.useMemo(() => {
  const q = query.trim().toLowerCase();
  return boards
    .filter((b) => b.id !== activeBoard?.id)
    .filter((b) => (q ? b.name.toLowerCase().includes(q) : true))
    .sort((a, b) => (b.accessedAt ?? b.updatedAt ?? 0) - (a.accessedAt ?? a.updatedAt ?? 0))
    .slice(0, 40);
}, [boards, query, activeBoard?.id]);

const pickPdf = async (file: File) => {
  setError(null);
  setBusy(true);
  try {
    // The page count has to come from the document itself; a range box that
    // does not know the maximum cannot clamp or validate anything.
    const { readPdfFile, openPdf } = await import("@/lib/whiteboard/pdf");
    const bytes = await readPdfFile(file);
    const doc = await openPdf(bytes);
    chooseSource({ kind: "pdf", file, pageCount: doc.numPages });
  } catch (err) {
    setError((err as Error).message);
  } finally {
    setBusy(false);
  }
};

const range = source
  ? resolvePageRange(source.pageCount, Number(from), Number(to))
  : { from: 0, to: 0, indices: [] as number[] };

const insertIndex =
  placement === "start"
    ? 0
    : placement === "end"
    ? targetPages.length
    : placement === "before"
    ? activeIndex
    : activeIndex + 1;

/**
 * Pages keep their own contents but land on THIS notebook's sheet, so a
 * different shape means the imported page is letterboxed. Worth saying
 * before the insert, not discovering afterwards on 60 pages.
 */
const sizeWarning = React.useMemo(() => {
  if (!source || source.kind !== "board" || !activeBoard) return null;
  const a = pageSizeOf(activeBoard);
  const b = pageSizeOf(source.board);
  const ratio = (s: { width: number; height: number }) => s.width / s.height;
  if (Math.abs(ratio(a) - ratio(b)) < 0.02) return null;

```

```

return `"$${source.board.name}" uses a differently shaped page (${Math.round(b.width)}x${Math.round(
  b.height
)}) against this notebook's ${Math.round(a.width)}x${Math.round(a.height)}. Inserted pages keep their cont
  ents but sit on this notebook's sheet.`;
}, [source, activeBoard]);

const handleInsert = async () => {
  if (!source || range.indices.length === 0) return;
  setBusy(true);
  setError(null);
  try {
    const n =
      source.kind === "board"
        ? insertPagesFromBoard(source.board.id, { from: range.from, to: range.to }, insertIndex)
        : await insertPdfPages(source.file, { from: range.from, to: range.to }, insertIndex);

    if (n === 0) {
      setError("Nothing was inserted. Check that those pages exist.");
      return;
    }
    setDone(`Inserted ${n} page${n === 1 ? "" : "s"}.`);
    onClose();
  } catch (err) {
    setError((err as Error).message);
  } finally {
    setBusy(false);
  }
};

if (!activeBoard) return null;

const notANotebook = !isNotebook(activeBoard);

return (
  <Dialog open={open} onOpenChange={({isOpen}) => !isOpen && onClose()}>
    <DialogContent className="max-w-2xl p-0 gap-0 overflow-hidden bg-white/98 dark:bg-zinc-900/98 backdrop-blur-xl border border-border/80 shadow-2xl rounded-2xl">
      <DialogHeader className="px-6 py-4 border-b border-border/60 bg-muted/20 space-y-0.5">
        <div className="flex items-center gap-2.5">
          <div className="w-8 h-8 rounded-lg bg-primary/10 text-primary flex items-center justify-center">
            <Layers size={18} />
          </div>
          <div>
            <DialogTitle className="text-base font-semibold">Insert pages</DialogTitle>
            <DialogDescription className="text-xs text-muted-foreground">
              Copy pages from another notebook or a PDF into &ldquo;{activeBoard.name}&rdquo;.
            </DialogDescription>
          </div>
        </div>
      </DialogHeader>

      {notANotebook ? (
        <div className="p-6 text-sm text-muted-foreground">
          Pages can only be inserted into a notebook. This board is a whiteboard, which is a
          single infinite canvas rather than a stack of sheets.
        </div>
      ) : (
        <div className="p-6 space-y-5 max-h-[70vh] overflow-y-auto">
          </* ----- Step 1: where from ----- */>
          <section className="space-y-2">
            <h3 className="text-xs font-semibold uppercase tracking-wider text-muted-foreground">
              1. Take pages from
            </h3>

            {source ? (
              <div className="flex items-center gap-2 rounded-xl border border-primary/50 bg-primary/5 px-3 py-2">
                {source.kind === "pdf" ? (
                  <FileText size={16} className="text-primary shrink-0" />
                ) : (
                  <NotebookText size={16} className="text-primary shrink-0" />
                )}
                <div className="min-w-0 flex-1">
                  <div className="truncate text-sm font-medium">
                    {source.kind === "pdf" ? source.file.name : source.board.name}
                  </div>
                  <div className="text-[11px] text-muted-foreground">
                    {source.pageCount} page{source.pageCount === 1 ? "" : "s"}
                  </div>
                </div>
              </div>
            )}
          </section>
        </div>
      )}
    </DialogContent>
  </Dialog>
)

```

```

        {source.kind === "pdf" ? " . PDF file" : ""}
      </div>
    </div>
    <button
      type="button"
      onClick={() => setSource(null)}
      aria-label="Choose a different source"
      className="p-1 rounded-md hover:bg-muted text-muted-foreground"
    >
      <X size={14} />
    </button>
  </div>
) : (
  <>
    <div className="relative">
      <Search
        size={14}
        className="absolute left-3 top-1/2 -translate-y-1/2 text-muted-foreground"
      />
      <input
        autoFocus
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        placeholder="Search your notebooks by name..."
        className="w-full h-9 rounded-lg border border-border bg-background pl-9 pr-3 text-sm focus:outline-none"
      />
    </div>

    <div className="max-h-48 overflow-y-auto rounded-lg border border-border/60 divide-y divide-border/40">
      {candidates.length === 0 && (
        <div className="px-3 py-6 text-center text-xs text-muted-foreground">
          {query ? `No notebook matches "${query}"` : "There is nothing else to copy from yet."}
        </div>
      )}
      {candidates.map((b) => {
        const count = isNotebook(b) ? getPages(b).length : 1;
        return (
          <button
            key={b.id}
            type="button"
            onClick={() => chooseSource({ kind: "board", board: b, pageCount: count })}
            className="w-full flex items-center gap-2.5 px-3 py-2 text-left hover:bg-muted/60 transition-colors"
          >
            <NotebookText size={15} className="text-muted-foreground shrink-0" />
            <span className="flex-1 truncate text-sm">{b.name}</span>
            <span className="text-[11px] text-muted-foreground tabular-nums shrink-0">
              {count} {isNotebook(b) ? "pages" : "canvas"}
            </span>
          </button>
        );
      })}
    </div>

    <div className="flex items-center gap-2">
      <div className="h-px flex-1 bg-border/60" />
      <span className="text-[11px] text-muted-foreground">or</span>
      <div className="h-px flex-1 bg-border/60" />
    </div>

    <input
      ref={fileInputRef}
      type="file"
      accept="application/pdf, .pdf"
      className="hidden"
      onChange={(e) => {
        const f = e.target.files?.[0];
        // Cleared so picking the SAME file twice still fires.
        e.target.value = "";
        if (f) void pickPdf(f);
      }}
    />
    <Button
      variant="outline"
      className="w-full gap-2"

```

```

        disabled={busy}
        onClick={() => fileInputRef.current?.click()}
      >
        <FileText size={15} />
        {busy ? "Reading PDF..." : "Choose a PDF file"}
      </Button>
    </>
  )}
</section>

{/* ----- Step 2: which pages ----- */}
{source && (
  <section className="space-y-2">
    <h3 className="text-xs font-semibold uppercase tracking-wider text-muted-foreground">
      2. Which pages
    </h3>
    <div className="flex flex-wrap items-center gap-2 text-sm">
      <span className="text-muted-foreground">From page</span>
      <PageNumberInput value={from} onChange={setFrom} max={source.pageCount} label="First page" />
    >
      <span className="text-muted-foreground">to</span>
      <PageNumberInput value={to} onChange={setTo} max={source.pageCount} label="Last page" />
      <button
        type="button"
        onClick={() => {
          setFrom("1");
          setTo(String(source.pageCount));
        }}
        className="ml-1 rounded-md border border-border px-2 py-1 text-[11px] font-medium hover:bg-muted"
      >
        All {source.pageCount}
      </button>
    </div>
    <p className="text-[11px] text-muted-foreground">
      {range.indices.length > 0
        ? `${range.indices.length} page${range.indices.length === 1 ? "" : "s"} (${range.from}-${range.to}) will be copied.`
        : "That range is empty."}
    </p>
  </section>
)}

{/* ----- Step 3: where to ----- */}
{source && (
  <section className="space-y-2">
    <h3 className="text-xs font-semibold uppercase tracking-wider text-muted-foreground">
      3. Put them
    </h3>
    <div className="grid grid-cols-2 gap-1.5">
      {PLACEMENTS.map((p) => (
        <button
          key={p.id}
          type="button"
          onClick={() => setPlacement(p.id)}
          className={cn(
            "py-1.5 px-2 rounded-lg border text-xs font-medium transition-all",
            placement === p.id
              ? "border-primary bg-primary/10 text-primary font-semibold"
              : "border-border hover:bg-muted text-foreground"
          )}
        >
          {p.label}
        </button>
      ))}
    </div>
    <p className="text-[11px] text-muted-foreground">
      You are on page {activeIndex + 1} of {targetPages.length}. The pages will become
      page{range.indices.length === 1 ? "" : "s"}{" "}
      {insertIndex + 1}
      {range.indices.length > 1 ? `-${range.insertIndex + range.indices.length}` : ""} of {" "}
      {targetPages.length + range.indices.length}.
    </p>
  </section>
)}

{sizeWarning && (
  <div className="flex gap-2 rounded-lg border border-amber-500/40 bg-amber-500/10 px-3 py-2 text-

```

```

      [11px] leading-snug text-amber-700 dark:text-amber-400">
      <AlertTriangle size={14} className="shrink-0 mt-0.5" />
      <span>{sizeWarning}</span>
    </div>
  )}
  {error && (
    <div
      role="alert"
      className="rounded-lg border border-red-500/40 bg-red-500/10 px-3 py-2 text-[11px] text-red-600 dark:text-red-400"
    >
      {error}
    </div>
  )}
  {done && (
    <div role="status" className="text-[11px] text-emerald-600 dark:text-emerald-400">
      {done}
    </div>
  )}

  <div className="flex gap-2 pt-1">
    <Button variant="outline" className="flex-1" onClick={onClose} disabled={busy}>
      Cancel
    </Button>
    <Button
      className="flex-1 font-semibold"
      disabled={!source || busy || range.indices.length === 0}
      onClick={handleInsert}
    >
      {busy
        ? "Inserting..."
        : `Insert ${range.indices.length} | "${range.start} - ${range.end}"`}
    </Button>
  </div>
</div>
)}
</DialogContent>
</Dialog>
);
}

```

```

=====
// === END FILE: src/components/whiteboard/InsertPagesModal.tsx ===
=====

```

```

=====
// === START FILE: src/components/whiteboard/KeyboardShortcutsModal.tsx ===
// File 59 of 194 | Lines: 258 | Size: 12290 bytes
=====

```

```

"use client";

import React, { useState, useMemo } from "react";
import {
  Keyboard,
  Search,
  Wrench,
  Navigation,
  Edit3,
  BookOpen,
  Tablet,
  X,
} from "lucide-react";
import {
  Dialog,
  DialogContent,
  DialogDescription,
  DialogHeader,
  DialogTitle,
} from "@components/ui/dialog";
import { Input } from "@components/ui/input";
import { ScrollArea } from "@components/ui/scroll-area";
import { cn } from "@lib/utills";

interface ShortcutItem {
  name: string;
  keys: string[];
}

```

```

    description?: string;
  }

interface ShortcutCategory {
  id: string;
  name: string;
  icon: React.ElementType;
  shortcuts: ShortcutItem[];
}

const SHORTCUT_CATEGORIES: ShortcutCategory[] = [
  {
    id: "tools",
    name: "Drawing & Creation Tools",
    icon: Wrench,
    shortcuts: [
      { name: "Select Tool", keys: ["V"], description: "Default selection pointer" },
      { name: "Lasso Select", keys: ["Q"], description: "Freehand lasso multi-selection" },
      { name: "Pan Canvas", keys: ["H"], description: "Hand tool or hold Space" },
      { name: "Laser Pointer", keys: ["K"], description: "Point without writing; hover the pen to aim" },
      { name: "Select Text", keys: ["I"], description: "Select PDF / Word / PowerPoint text to copy or highlight" },
      { name: "Pen / Brush", keys: ["P"], description: "Freehand stroke drawing" },
      { name: "Highlighter", keys: ["Y"], description: "Semi-transparent highlighter" },
      { name: "Eraser", keys: ["E"], description: "Continuous drag or tap eraser" },
      { name: "Rectangle", keys: ["R"], description: "Create box or container" },
      { name: "Ellipse / Circle", keys: ["O"], description: "Create circle or oval" },
      { name: "Diamond", keys: ["D"], description: "Decision node shape" },
      { name: "Line", keys: ["L"], description: "Straight line segment" },
      { name: "Arrow", keys: ["A"], description: "Directional arrow line" },
      { name: "Text Box", keys: ["T"], description: "Click or double-click to type" },
      { name: "Sticky Note", keys: ["N"], description: "Colored markdown note card" },
      { name: "Smart Connector", keys: ["C"], description: "Connect notes with typed relationship" },
      { name: "Upload Image", keys: ["I"], description: "Insert image or paste via Ctrl+V" },
    ],
  },
  {
    id: "navigation",
    name: "Navigation & View",
    icon: Navigation,
    shortcuts: [
      { name: "Pan Canvas", keys: ["Space", "Drag"], description: "Or hold Middle Mouse button" },
      { name: "Zoom In / Out", keys: ["Ctrl", "+ / -"], description: "Or Ctrl + Mouse Wheel" },
      { name: "Reset Zoom to 100%", keys: ["Ctrl", "0"], description: "Default 1:1 camera scale" },
      { name: "Zoom to Fit Content", keys: ["Shift", "1"], description: "Fit all board elements in view" },
      { name: "Toggle Minimap", keys: ["M"], description: "Radar navigator overlay" },
      { name: "Toggle Theme", keys: ["Ctrl", "Shift", "D"], description: "Switch Dark / Light / System theme" },
    ],
  },
  {
    id: "editing",
    name: "Selection & Editing",
    icon: Edit3,
    shortcuts: [
      { name: "Select All", keys: ["Ctrl", "A"], description: "Select all elements on active board" },
      { name: "Delete Selected", keys: ["Delete"], description: "Or Backspace key" },
      { name: "Undo", keys: ["Ctrl", "Z"], description: "Undo last canvas modification" },
      { name: "Redo", keys: ["Ctrl", "Y"], description: "Or Ctrl + Shift + Z" },
      { name: "Duplicate Elements", keys: ["Ctrl", "D"], description: "Clone selected elements" },
      { name: "Lock / Unlock", keys: ["Ctrl", "Shift", "L"], description: "Prevent accidental moving" },
      { name: "Bring to Front", keys: ["Ctrl", "I"], description: "Move element to top layer" },
      { name: "Send to Back", keys: ["Ctrl", "B"], description: "Move element to background layer" },
      { name: "Snap to Grid", keys: ["Ctrl", "Shift", "G"], description: "Or press S to toggle grid alignment" },
    ],
  },
  {
    id: "obsidian",
    name: "Obsidian & Knowledge Graph",
    icon: BookOpen,
    shortcuts: [
      { name: "Search Everywhere", keys: ["Ctrl", "K"], description: "Names, folders, tags, text, stickies, PDF text - nearest first" },
      { name: "Find in This Note", keys: ["Ctrl", "F"], description: "Step through matches with Enter / Shift+Enter" },
      { name: "Next Page", keys: ["Page Down"], description: "Turn to the next page of a notebook" },
    ],
  },
];

```

```

    { name: "Previous Page", keys: ["Page Up"], description: "Turn to the previous page of a notebook" },
    { name: "Knowledge Graph View", keys: ["Ctrl", "G"], description: "Interactive bi-directional network graph" },
    { name: "Toggle Backlinks Panel", keys: ["Ctrl", "B"], description: "Inspect incoming and outgoing links" },
    { name: "Link Files Manager", keys: ["Ctrl", "L"], description: "Batch connect boards and references" },
    { name: "Export Board", keys: ["Ctrl", "E"], description: "Export to high-res PNG or vector SVG" },
    { name: "Obsidian Wikilink", keys: ["[[Board]"]], description: "Syntax to link any board inline" },
    { name: "Tag Classification", keys: ["#tag"], description: "Syntax to categorize notes and thoughts" },
    { name: "Keyboard Shortcuts Help", keys: ["?"], description: "Or Ctrl + / to open this help modal" },
  ],
},
{
  id: "touch",
  name: "Touchscreen & Stylus Gestures",
  icon: Tablet,
  shortcuts: [
    { name: "Pinch to Zoom", keys: ["2-Finger Pinch"], description: "Smooth zoom in and out centered on fingers" },
    { name: "2-Finger Pan", keys: ["2-Finger Drag"], description: "Pan canvas smoothly while drawing" },
    { name: "Quick Undo", keys: ["2-Finger Tap"], description: "Tap two fingers together on screen" },
    { name: "Quick Redo", keys: ["3-Finger Tap"], description: "Tap three fingers together on screen" },
    { name: "Create Quick Note", keys: ["Double Tap"], description: "Double-tap empty canvas with finger or stylus" },
    { name: "Palm Rejection", keys: ["Stylus Mode"], description: "Exempts palm touches from drawing" },
  ],
},
];

interface KeyboardShortcutsModalProps {
  open: boolean;
  onClose: () => void;
}

export function KeyboardShortcutsModal({ open, onClose }: KeyboardShortcutsModalProps) {
  const [searchQuery, setSearchQuery] = useState("");

  const filteredCategories = useMemo(() => {
    const q = searchQuery.toLowerCase().trim();
    if (!q) return SHORTCUT_CATEGORIES;

    return SHORTCUT_CATEGORIES.map((cat) => {
      const filtered = cat.shortcuts.filter(
        (item) =>
          item.name.toLowerCase().includes(q) ||
          item.keys.some((k) => k.toLowerCase().includes(q)) ||
          (item.description && item.description.toLowerCase().includes(q))
      );
      return { ...cat, shortcuts: filtered };
    }).filter((cat) => cat.shortcuts.length > 0);
  }, [searchQuery]);

  return (
    <Dialog open={open} onOpenChange={({isOpen}) => !isOpen && onClose()}>
      <DialogContent className="max-w-3xl max-h-[85vh] p-0 gap-0 overflow-hidden bg-white/98 dark:bg-zinc-900/98 backdrop-blur-xl border border-border/80 shadow-2xl rounded-2xl flex flex-col">
        { /* Header */ }
        <div className="px-6 py-4 border-b border-border/60 bg-muted/20 flex flex-col gap-3">
          <div className="flex items-center justify-between">
            <div className="flex items-center gap-2.5">
              <div className="w-8 h-8 rounded-lg bg-primary/10 text-primary flex items-center justify-center font-bold">
                <Keyboard size={18} />
              </div>
            </div>
            <div>
              <DialogTitle className="text-base font-semibold text-foreground">
                Keyboard Shortcuts & Gestures
              </DialogTitle>
              <DialogDescription className="text-xs text-muted-foreground">
                Pro-grade hotkeys for keyboard, mouse, and multi-touch tablet controls.
              </DialogDescription>
            </div>
          </div>
        </div>
        { /* Live Search Box */ }
        <div className="relative">
          <Search size={14} className="absolute left-3 top-1/2 -translate-y-1/2 text-muted-foreground" />

```

```

<Input
  value={searchQuery}
  onChange={(e) => setSearchQuery(e.target.value)}
  placeholder="Search shortcuts (e.g., zoom, pen, wikilink, export, undo)..."
  className="h-9 pl-9 text-xs bg-background/80"
  autoFocus
/>
{searchQuery && (
  <button
    onClick={() => setSearchQuery("")}
    className="absolute right-3 top-1/2 -translate-y-1/2 text-muted-foreground hover:text-foreground"
  >
    <X size={14} />
  </button>
)}
</div>
</div>

{/* Scrollable Shortcut Cards */}
<ScrollArea className="flex-1 p-6 overflow-y-auto max-h-[60vh]">
  {filteredCategories.length === 0 ? (
    <div className="py-12 text-center text-xs text-muted-foreground">
      No shortcuts found matching &ldquo;{searchQuery}&rdquo;
    </div>
  ) : (
    <div className="space-y-6">
      {filteredCategories.map((category) => {
        const Icon = category.icon;
        return (
          <div key={category.id} className="space-y-2.5">
            <div className="flex items-center gap-2 text-xs font-semibold uppercase tracking-wider text-muted-foreground">
              <Icon size={14} className="text-primary" />
              <span>{category.name}</span>
              <span className="text-[10px] font-normal px-1.5 py-0.2 rounded-full bg-muted">
                {category.shortcuts.length}
              </span>
            </div>

            <div className="grid grid-cols-1 md:grid-cols-2 gap-2">
              {category.shortcuts.map((item, idx) => (
                <div
                  key={idx}
                  className="flex items-center justify-between p-2.5 rounded-xl border border-border/70 bg-card/60 hover:bg-muted/40 transition-colors"
                >
                  <div className="space-y-0.5 pr-2 truncate">
                    <div className="text-xs font-medium text-foreground truncate">
                      {item.name}
                    </div>
                    {item.description && (
                      <div className="text-[11px] text-muted-foreground truncate">
                        {item.description}
                      </div>
                    )}
                  </div>

                  <div className="flex items-center gap-1 shrink-0">
                    {item.keys.map((k, kIdx) => (
                      <React.Fragment key={kIdx}>
                        <kbd className="px-2 py-1 text-[11px] font-mono font-semibold text-foreground bg-muted/80 border border-border/80 rounded-md shadow-xs">
                          {k}
                        </kbd>
                        {kIdx < item.keys.length - 1 && (
                          <span className="text-xs text-muted-foreground">+</span>
                        )}
                      </React.Fragment>
                    ))}
                  </div>
                </div>
              )}
            </div>
          </div>
        )}
      )}
    </div>
  );
}
</div>

```

```

    })
  </ScrollArea>

  { /* Footer */
    <div className="px-6 py-3 border-t border-border/60 bg-muted/20 flex items-center justify-between text
      -xs text-muted-foreground">
      <span>Press <kbd className="px-1.5 py-0.5 font-mono text-[10px] bg-muted border rounded">?</kbd> any
        where on canvas to open help</span>
      <span>ProNotes v3</span>
    </div>
  </DialogContent>
</Dialog>
);
}

```

```

=====
// === END FILE: src/components/whiteboard/KeyboardShortcutsModal.tsx ===
=====

```

```

=====
// === START FILE: src/components/whiteboard/LaserPointer.tsx ===
// File 60 of 194 | Lines: 332 | Size: 10664 bytes
=====

```

```

"use client";

/**
 * Laser pointer
 * =====
 * Points at the page without writing on it. Nothing here touches the board:
 * no element, no undo step, no save. It lives on its own canvas above the
 * board, in screen pixels, and only runs an animation loop while something is
 * visible.
 *
 * Points are kept in WORLD coordinates and projected with the live camera each
 * frame, so a circle you drew stays around the word it circles even if the
 * page scrolls while it fades.
 *
 * The canvas input code drives it through `laser` (down / move / up / hover);
 * the look comes from the laser settings in the style (mode, colour, size).
 */

```

```

import React, { useEffect, useRef } from "react";
import { useWhiteboard } from "@store/whiteboard";
import { DEFAULT_STYLE } from "@lib/whiteboard/types";
import { getActivePageIndex } from "@lib/whiteboard/notebook";

```

```

interface LaserPoint {
  x: number;
  y: number;
  t: number;
}

```

```

/** How long each mode's tail lasts behind the dot. */
const TAIL_MS = { dot: 90, trail: 550 } as const;
/** "draw" lines take this long to fade out once their time is up. */
const DRAW_FADE_OUT_MS = 350;
/**
 * The hover dot disappears when the pen stops reporting for this long (it has
 * left hover range). A mouse stays put without reporting, and with the laser
 * the dot IS its cursor, so a mouse dot never times out - it goes on leave.
 */
const HOVER_TIMEOUT_MS = 1500;

```

```

const state = {
  /** Pointer currently pressing with the laser, if any. */
  pointerId: null as number | null,
  /** Recent points of the live pointer (tail for dot / trail). */
  tail: [] as LaserPoint[],
  /** "draw" mode: finished and live lines, kept until they fade together. */
  lines: [] as LaserPoint[][],
  /** When the last line was lifted; the fade counts from here. */
  liftedAt: 0,
  /** Where the dot is (pressing or hovering), and when it was last seen. */
  head: null as LaserPoint | null,
  /** The dot belongs to a hovering mouse: no timeout. */
  headSticky: false,

```

```

    wake: null as (() => void) | null,
  };

function now() {
  return performance.now();
}

/** Imperative handle used by the canvas input handlers. */
export const laser = {
  isActive(pointerId: number) {
    return state.pointerId === pointerId;
  },
  down(pointerId: number, x: number, y: number) {
    const p = { x, y, t: now() };
    state.pointerId = pointerId;
    state.head = p;
    state.tail = [p];
    // A new line joins the ones still showing; they all fade together later.
    state.lines.push([p]);
    state.wake?();
  },
  move(x: number, y: number) {
    const p = { x, y, t: now() };
    state.head = p;
    state.tail.push(p);
    state.lines[state.lines.length - 1]?.push(p);
    state.wake?();
  },
  up(pointerId: number) {
    if (state.pointerId !== pointerId) return;
    state.pointerId = null;
    state.liftedAt = now();
    state.wake?();
  },
  /** A hovering pen or mouse: the dot follows without drawing. */
  hover(x: number, y: number, pointerType = "pen") {
    if (state.pointerId !== null) return;
    state.head = { x, y, t: now() };
    state.headSticky = pointerType === "mouse";
    state.wake?();
  },
  hideHover() {
    if (state.pointerId === null) state.head = null;
    state.wake?();
  },
  /** Everything off at once (tool changed, pinch started). */
  clear() {
    state.pointerId = null;
    state.tail = [];
    state.lines = [];
    state.head = null;
    state.wake?();
  },
};

/** Draws one polyline as a glowing beam: coloured halo, white-hot core. */
function beam(
  ctx: CanvasRenderingContext2D,
  pts: Array<{ x: number; y: number }>,
  color: string,
  width: number,
  alpha: number
) {
  if (pts.length === 0 || alpha <= 0) return;
  ctx.save();
  ctx.globalAlpha = alpha;
  ctx.lineCap = "round";
  ctx.lineJoin = "round";
  const path = new Path2D();
  path.moveTo(pts[0].x, pts[0].y);
  // A lone point still shows: a zero-length round-capped line is a dot.
  if (pts.length === 1) path.lineTo(pts[0].x + 0.01, pts[0].y);
  for (let i = 1; i < pts.length; i++) {
    const a = pts[i - 1];
    const b = pts[i];
    path.quadraticCurveTo(a.x, a.y, (a.x + b.x) / 2, (a.y + b.y) / 2);
  }
  if (pts.length > 1) path.lineTo(pts[pts.length - 1].x, pts[pts.length - 1].y);
}

```

```

    ctx.shadowColor = color;
    ctx.shadowBlur = width * 1.6;
    ctx.strokeStyle = color;
    ctx.lineWidth = width;
    ctx.stroke(path);
    ctx.shadowBlur = 0;
    ctx.strokeStyle = "rgba(255,255,255,0.9)";
    ctx.lineWidth = Math.max(1, width * 0.34);
    ctx.stroke(path);
    ctx.restore();
}

/** The pointer itself: a glowing disc with a bright centre. */
function dot(ctx: CanvasRenderingContext2D, x: number, y: number, color: string, size: number, alpha = 1) {
    const r = size / 2;
    ctx.save();
    ctx.globalAlpha = alpha;
    const halo = ctx.createRadialGradient(x, y, 0, x, y, r * 2.6);
    halo.addColorStop(0, color);
    halo.addColorStop(0.35, color);
    halo.addColorStop(1, "rgba(0,0,0,0)");
    ctx.globalAlpha = alpha * 0.45;
    ctx.fillStyle = halo;
    ctx.beginPath();
    ctx.arc(x, y, r * 2.6, 0, Math.PI * 2);
    ctx.fill();
    ctx.globalAlpha = alpha;
    ctx.fillStyle = color;
    ctx.beginPath();
    ctx.arc(x, y, r, 0, Math.PI * 2);
    ctx.fill();
    ctx.fillStyle = "rgba(255,255,255,0.92)";
    ctx.beginPath();
    ctx.arc(x, y, r * 0.42, 0, Math.PI * 2);
    ctx.fill();
    ctx.restore();
}

/**
 * The canvas the laser draws on. Mount it inside the board's container, above
 * the board; it never takes a pointer event.
 */
export function LaserLayer() {
    const canvasRef = useRef<HTMLCanvasElement>(null);

    useEffect(() => {
        const canvas = canvasRef.current;
        if (!canvas) return;
        const ctx = canvas.getContext("2d");
        if (!ctx) return;

        let raf = 0;
        let running = false;

        const frame = () => {
            raf = 0;
            const { camera, style } = useWhiteboard.getState();
            const color = style.laserColor ?? DEFAULT_STYLE.laserColor;
            const size = style.laserSize ?? DEFAULT_STYLE.laserSize;
            const mode = style.laserMode ?? DEFAULT_STYLE.laserMode;
            const holdMs = style.laserFadeMs ?? DEFAULT_STYLE.laserFadeMs;

            const dpr = window.devicePixelRatio || 1;
            const w = canvas.clientWidth;
            const h = canvas.clientHeight;
            if (canvas.width !== Math.round(w * dpr) || canvas.height !== Math.round(h * dpr)) {
                canvas.width = Math.round(w * dpr);
                canvas.height = Math.round(h * dpr);
            }
            ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
            ctx.clearRect(0, 0, w, h);

            const toScreen = (p: { x: number; y: number }) => ({
                x: (p.x - camera.x) * camera.zoom + w / 2,
                y: (p.y - camera.y) * camera.zoom + h / 2,
            });

            const t = now();

```

```

const pressing = state.pointerId !== null;
let busy = pressing;

if (mode === "draw") {
  // Lines stay while you keep drawing, then all fade together.
  let alpha = 1;
  if (!pressing && state.lines.length > 0) {
    const past = t - state.liftedAt - holdMs;
    alpha = past <= 0 ? 1 : 1 - past / DRAW_FADE_OUT_MS;
    if (alpha <= 0) {
      state.lines = [];
    } else {
      busy = true;
    }
  }
  for (const line of state.lines) beam(ctx, line.map(toScreen), color, size * 0.6, alpha);
  state.tail = [];
} else {
  state.lines = [];
  // A tail that thins and fades towards its end.
  const life = TAIL_MS[mode];
  state.tail = state.tail.filter((p) => t - p.t < life);
  const tail = state.tail;
  if (tail.length > 1) {
    busy = true;
    const pts = tail.map(toScreen);
    for (let i = 1; i < pts.length; i++) {
      const k = 1 - (t - tail[i].t) / life; // 1 at the head, 0 at the end
      beam(ctx, [pts[i - 1], pts[i]], color, Math.max(1, size * 0.75 * k), k * 0.9);
    }
  }
}

// The dot: while pressing, or while a pen/mouse hovers with the laser.
const head = state.head;
if (head) {
  const stale = !pressing && !state.headSticky && t - head.t > HOVER_TIMEOUT_MS;
  if (stale) {
    state.head = null;
  } else {
    const s = toScreen(head);
    dot(ctx, s.x, s.y, color, pressing ? size : size * 0.8, pressing ? 1 : 0.85);
    busy = true;
  }
}

if (busy) raf = requestAnimationFrame(frame);
else running = false;
};

const wake = () => {
  if (running) return;
  running = true;
  raf = requestAnimationFrame(frame);
};
state.wake = wake;

// Which page of which note the points belong to. Points are in that
// page's coordinates: on another note (split view) or after the notebook
// re-anchors on the next page, they would land in the wrong place.
const placeOf = (s: ReturnType<typeof useWhiteboard.getState>) => {
  const b = s.boards.find((x) => x.id === s.activeBoardId);
  return `${s.activeBoardId}:${b ? getActivePageIndex(b) : 0}`;
};
let place = placeOf(useWhiteboard.getState());
// The camera moves the drawn lines; redraw when it does.
const unsub = useWhiteboard.subscribe((s, prev) => {
  if (s.camera !== prev.camera) wake();
  // Put the laser down when another tool is picked.
  if (s.tool !== prev.tool && prev.tool === "laser") laser.clear();
  if (s.activeBoardId !== prev.activeBoardId || s.boards !== prev.boards) {
    const next = placeOf(s);
    if (next !== place) {
      place = next;
      laser.clear();
    }
  }
});

```

```

    wake();

    return () => {
      unsub();
      if (raf) cancelAnimationFrame(raf);
      if (state.wake === wake) state.wake = null;
    };
  }, []);

  return (
    <canvas
      ref={canvasRef}
      aria-hidden
      className="pointer-events-none absolute inset-0 h-full w-full"
      style={{ zIndex: 5 }}
    />
  );
}

/** The laser's tool-bar icon: a pointer pen with a beam and a lit spot. */
export function LaserIcon({ size = 18, className }: { size?: number; className?: string }) {
  return (
    <svg
      width={size}
      height={size}
      viewBox="0 0 24 24"
      fill="none"
      stroke="currentColor"
      strokeWidth={2}
      strokeLinecap="round"
      strokeLinejoin="round"
      className={className}
      aria-hidden
    >
      {/ body */}
      <path d="M3.5 20.5 12 12" />
      <path d="M2.8 18.4 5.6 21.2" />
      {/ beam */}
      <path d="M13.5 10.5 16 8" strokeDasharray="1.5 2.2" />
      {/ lit spot with rays */}
      <circle cx="18.5" cy="5.5" r="2" fill="currentColor" />
      <path d="M18.5 1.2v1.2M22.8 5.5h-1.2M21.6 2.4l-.85.85" />
    </svg>
  );
}

```

```

=====
// === END FILE: src/components/whiteboard/LaserPointer.tsx ===
=====

```

```

=====
// === START FILE: src/components/whiteboard/LinkLegend.tsx ===
// File 61 of 194 | Lines: 73 | Size: 2795 bytes
=====

```

```

"use client";

/**
 * Link Legend
 * =====
 * A floating panel (bottom-right) showing all link types and their colors –
 * like a map key/legend. Shows:
 * - Built-in types (Reference, Dependency, Related, Flow)
 * - Custom types created by the user
 * - Parent-child (folder→board) relationship
 *
 * Toggle visibility with the "Legend" button in the header.
 */

```

```

import React from "react";
import { useWhiteboard } from "@store/whiteboard";
import {
  BUILTIN_LINK_TYPES,
  LINK_TYPE_COLORS,
  LINK_TYPE_LABELS,
  getLinkTypeColor,
  getLinkTypeLabel,
}

```

```

} from "@lib/whiteboard/types";
import { Map } from "lucide-react";
import { cn } from "@lib/utls";

export function LinkLegend({ visible, onToggle }: { visible: boolean; onToggle: () => void }) {
  const customLinkTypes = useWhiteboard((s) => s.customLinkTypes);

  if (!visible) return null;

  return (
    // Clear of the bottom docks. The toolbar and the zoom/undo cluster both sit
    // at `bottom-6` (24px) with z-30 and are ~52px tall, so they occupy roughly
    // 24-76px; the legend used to sit at bottom-16 (64px) with z-20 and was
    // drawn underneath them. bottom-28 clears the docks, right-6 lines it up
    // with the right-hand cluster, and max-h keeps a long custom-type list from
    // growing up into the header.
    <div className="absolute bottom-28 right-6 z-30 bg-white/95 dark:bg-zinc-900/95 backdrop-blur-md shadow-lg
      border border-border rounded-xl p-3 w-56 max-h-[45vh] overflow-y-auto">
      <div className="flex items-center gap-1.5 mb-2">
        <Map size={13} className="text-muted-foreground" />
        <span className="text-xs font-semibold">Link Legend</span>
      </div>
      <div className="space-y-1.5">
        { /* Built-in types */ }
        { BUILTIN_LINK_TYPES.map((lt) => (
          <div key={lt} className="flex items-center gap-2 text-xs">
            <span
              className="w-4 h-1.5 rounded-full shrink-0"
              style={{ backgroundColor: LINK_TYPE_COLORS[lt] }}
            />
            <span>{LINK_TYPE_LABELS[lt]}</span>
          </div>
        )) }
        { /* Custom types */ }
        { customLinkTypes.map((ct) => (
          <div key={ct.id} className="flex items-center gap-2 text-xs">
            <span
              className="w-4 h-1.5 rounded-full shrink-0"
              style={{ backgroundColor: ct.color }}
            />
            <span>{ct.name}</span>
          </div>
        )) }
        { /* Parent-child (folder→board) */ }
        <div className="flex items-center gap-2 text-xs pt-1 border-t border-border">
          <span className="w-4 h-1.5 shrink-0 border-t-2 border-dashed border-gray-400" />
          <span className="text-muted-foreground">contains (folder→board)</span>
        </div>
      </div>
    </div>
  );
}

```

```

=====
// === END FILE: src/components/whiteboard/LinkLegend.tsx ===
=====

```

```

=====
// === START FILE: src/components/whiteboard/LinkManager.tsx ===
// File 62 of 194 | Lines: 447 | Size: 18380 bytes
=====

```

```

"use client";

/**
 * Link Manager Modal
 * =====
 * The primary UI for creating links between files (boards) and folders.
 *
 * How it works:
 * 1. User clicks "Link Files" button in the sidebar (next to +Board/+Folder)
 * 2. This modal opens with two columns: "From" (source) and "To" (target)
 * 3. User selects a board or folder from each column
 * 4. User picks a link type (Reference, Dependency, Related, Flow, or custom)
 * 5. User can optionally enter a custom label
 * 6. Click "Create Link" → a ConnectionElement is created on the source's board
 *
 */

```

```

* The connection is stored as a ConnectionElement on the source board, with
* source.elementId = source board/folder id and target.elementId = target id.
* For folders (which have no elements), we store a special pseudo-element.
*
* Existing links are listed at the bottom with their type colors and a delete button.
*/

import React, { useState, useMemo } from "react";
import { useWhiteboard } from "@store/whiteboard";
import { allElements } from "@lib/whiteboard/notebook";
import {
  BUILTIN_LINK_TYPES,
  LINK_TYPE_COLORS,
  LINK_TYPE_LABELS,
  getLinkTypeColor,
  getLinkTypeLabel,
  type Folder,
  type Board,
  type ConnectionElement,
  type WhiteboardElement,
} from "@lib/whiteboard/types";
import { nanoid } from "nanoid";
import { X, Link2, Folder as FolderIcon, FileText, Trash2, Plus } from "lucide-react";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { ScrollArea } from "@components/ui/scroll-area";
import { cn } from "@lib/utls";

interface LinkManagerProps {
  open: boolean;
  onClose: () => void;
}

type SelectableItem =
  | { kind: "board"; id: string; name: string; color: string; folderId: string | null }
  | { kind: "folder"; id: string; name: string; color: string };

export function LinkManager({ open, onClose }: LinkManagerProps) {
  const folders = useWhiteboard((s) => s.folders);
  const boards = useWhiteboard((s) => s.boards);
  const customLinkTypes = useWhiteboard((s) => s.customLinkTypes);
  const addElementToBoard = useWhiteboard((s) => s.addElementToBoard);
  const deleteElementFromBoard = useWhiteboard((s) => s.deleteElementFromBoard);

  const [sourceId, setSourceId] = useState<string | null>(null);
  const [sourceKind, setSourceKind] = useState<"board" | "folder">("board");
  const [targetId, setTargetId] = useState<string | null>(null);
  const [targetKind, setTargetKind] = useState<"board" | "folder">("board");
  const [linkType, setLinkType] = useState<string>("relates");
  const [customLabel, setCustomLabel] = useState("");
  const [sourceSearch, setSourceSearch] = useState("");
  const [targetSearch, setTargetSearch] = useState("");

  // Build a flat list of all selectable items (boards + folders)
  const allItems: SelectableItem[] = useMemo(() => {
    const boardItems: SelectableItem[] = boards.map((b) => ({
      kind: "board" as const,
      id: b.id,
      name: b.name,
      color: b.color || "#3b82f6",
      folderId: b.folderId,
    }));
    const folderItems: SelectableItem[] = folders.map((f) => ({
      kind: "folder" as const,
      id: f.id,
      name: f.name,
      color: f.color || "#f59e0b",
    }));
    return [...folderItems, ...boardItems];
  }, [boards, folders]);

  // Filtered items for source and target columns based on search
  const sourceItems = useMemo(() => {
    if (!sourceSearch.trim()) return allItems;
    const q = sourceSearch.toLowerCase();
    return allItems.filter((i) => i.name.toLowerCase().includes(q));
  }, [allItems, sourceSearch]);

```

```

const targetItems = useMemo(() => {
  if (!targetSearch.trim()) return allItems.filter((i) => i.id !== sourceId);
  const q = targetSearch.toLowerCase();
  return allItems.filter((i) => i.id !== sourceId && i.name.toLowerCase().includes(q));
}, [allItems, sourceId, targetSearch]);

// Collect all existing connection elements across all boards
const existingLinks = useMemo(() => {
  const links: Array<{
    connectionId: string;
    boardId: string;
    sourceName: string;
    sourceKind: "board" | "folder";
    targetName: string;
    targetKind: "board" | "folder";
    linkType: string;
    label?: string;
  }> = [];
  for (const board of boards) {
    for (const el of allElements(board)) {
      if (el.type === "connection") {
        const conn = el as ConnectionElement;
        const sourceItem = allItems.find((i) => i.id === conn.source.elementId);
        const targetItem = allItems.find((i) => i.id === conn.target.elementId);
        if (sourceItem && targetItem) {
          links.push({
            connectionId: conn.id,
            boardId: board.id,
            sourceName: sourceItem.name,
            sourceKind: sourceItem.kind,
            targetName: targetItem.name,
            targetKind: targetItem.kind,
            linkType: conn.linkType,
            label: conn.label,
          });
        }
      }
    }
  }
  return links;
}, [boards, allItems]);

if (!open) return null;

const handleCreateLink = () => {
  if (!sourceId || !targetId || sourceId === targetId) return;

  // For connections between boards/folders, we need to store the connection
  // on a board. If source is a board, store it there. If source is a folder,
  // store it on the active board (or first board).
  let storageBoardId: string | null = null;
  if (sourceKind === "board") {
    storageBoardId = sourceId;
  } else {
    // Source is a folder – find the first board in that folder, or use active board
    const boardInFolder = boards.find((b) => b.folderId === sourceId);
    storageBoardId = boardInFolder?.id || boards[0]?.id || null;
  }
  if (!storageBoardId) return;

  // Check for duplicate
  const storageBoard = boards.find((b) => b.id === storageBoardId);
  const exists = (storageBoard ? allElements(storageBoard) : []).some(
    (el) =>
      el.type === "connection" &&
      (el as ConnectionElement).source.elementId === sourceId &&
      (el as ConnectionElement).target.elementId === targetId
  );
  if (exists) {
    alert("This link already exists!");
    return;
  }

  // Create the connection element
  const conn: ConnectionElement = {
    id: nanoid(),
    type: "connection",
    x: 0,

```

```

    y: 0,
    rotation: 0,
    opacity: 1,
    source: { boardId: storageBoardId, elementId: sourceId, anchor: "auto" },
    target: { boardId: targetKind === "board" ? targetId : storageBoardId, elementId: targetId, anchor: "auto" },
    linkType,
    routing: "curved",
    dashStyle: "solid",
    startArrowhead: "none",
    endArrowhead: "arrow",
    strokeWidth: 2,
    label: customLabel.trim() || undefined,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  };

  // Write directly to the storage board — switching boards here would wipe
  // the user's camera/selection/undo-history on the board they are viewing.
  addElementToBoard(storageBoardId, conn as unknown as WhiteboardElement, true);

  // Reset selection
  setSourceId(null);
  setTargetId(null);
  setCustomLabel("");
};

const handleDeleteLink = (connectionId: string, boardId: string) => {
  deleteElementFromBoard(boardId, [connectionId], true);
};

const sourceItem = allItems.find((i) => i.id === sourceId);
const targetItem = allItems.find((i) => i.id === targetId);
const canCreate = sourceId && targetId && sourceId !== targetId;

return (
  <div
    data-wb-modal
    className="fixed inset-0 z-[90] flex items-center justify-center bg-black/50 backdrop-blur-sm"
    onClick={onClose}
  >
    <div
      className="w-full max-w-3xl bg-white dark:bg-zinc-900 rounded-2xl shadow-2xl border border-border over
        flow-hidden max-h-[85vh] flex flex-col"
      onClick={(e) => e.stopPropagation()}
    >
      {/* Header */}
      <div className="flex items-center justify-between px-5 py-3 border-b border-border">
        <div className="flex items-center gap-2">
          <Link2 size={18} className="text-primary" />
          <h2 className="text-base font-semibold">Link Files & Folders</h2>
        </div>
        <Button variant="ghost" size="icon" className="h-7 w-7" onClick={onClose}>
          <X size={16} />
        </Button>
      </div>

      <ScrollArea className="flex-1">
        <div className="p-5 space-y-4">
          {/* Instructions */}
          <div className="text-xs text-muted-foreground bg-muted/50 rounded-lg p-3">
            Select a source (file or folder) on the left, a target on the right,
            choose a link type, then click "Create Link". The link is saved
            with the source board and shows as an edge in the Knowledge Graph
            (Ctrl+G) and in the list below.
          </div>

          {/* Source & Target selectors */}
          <div className="grid grid-cols-2 gap-4">
            {/* Source */}
            <div className="space-y-2">
              <div className="text-xs font-medium text-muted-foreground flex items-center gap-1">
                <span className="w-2 h-2 rounded-full bg-primary" />
                From (source)
              </div>
              <Input
                value={sourceSearch}
                onChange={(e) => setSourceSearch(e.target.value)}
              />
            </div>

```

```

        placeholder="Search name..."
        className="h-7 text-xs"
      />
    <div className="border border-border rounded-lg max-h-48 overflow-y-auto">
      {sourceItems.length === 0 ? (
        <div className="text-xs text-muted-foreground italic py-3 text-center">No matches</div>
      ) : (
        sourceItems.map((item) => (
          <button
            key={` ${item.kind}-${item.id}`}
            onClick={() => {
              setSourceId(item.id);
              setSourceKind(item.kind);
            }}
            className={cn(
              "flex items-center gap-2 w-full px-3 py-2 text-left text-sm hover:bg-muted transiti
n-colors border-b border-border last:border-0",
              sourceId === item.id && "bg-primary/10 border-l-2 border-l-primary"
            )}
          >
            {item.kind === "folder" ? (
              <FolderIcon size={13} style={{ color: item.color }} />
            ) : (
              <FileText size={13} style={{ color: item.color }} />
            )}
            <span className="flex-1 truncate">{item.name}</span>
            <span className="text-[10px] text-muted-foreground capitalize">{item.kind}</span>
          </button>
        ))
      )}
    </div>
  </div>

  /* Target */
  <div className="space-y-2">
    <div className="text-xs font-medium text-muted-foreground flex items-center gap-1">
      <span className="w-2 h-2 rounded-full bg-emerald-500" />
      To (target)
    </div>
    <Input
      value={targetSearch}
      onChange={(e) => setTargetSearch(e.target.value)}
      placeholder="Search name..."
      className="h-7 text-xs"
    />
    <div className="border border-border rounded-lg max-h-48 overflow-y-auto">
      {targetItems.length === 0 ? (
        <div className="text-xs text-muted-foreground italic py-3 text-center">No matches</div>
      ) : (
        targetItems.map((item) => (
          <button
            key={` ${item.kind}-${item.id}`}
            onClick={() => {
              setTargetId(item.id);
              setTargetKind(item.kind);
            }}
            className={cn(
              "flex items-center gap-2 w-full px-3 py-2 text-left text-sm hover:bg-muted transiti
n-colors border-b border-border last:border-0",
              targetId === item.id && "bg-emerald-500/10 border-l-2 border-l-emerald-500"
            )}
          >
            {item.kind === "folder" ? (
              <FolderIcon size={13} style={{ color: item.color }} />
            ) : (
              <FileText size={13} style={{ color: item.color }} />
            )}
            <span className="flex-1 truncate">{item.name}</span>
            <span className="text-[10px] text-muted-foreground capitalize">{item.kind}</span>
          </button>
        ))
      )}
    </div>
  </div>
</div>

  /* Link type selector */
  <div className="space-y-2">

```

```

<div className="text-xs font-medium text-muted-foreground">Link type</div>
<div className="flex flex-wrap gap-1.5">
  {BUILTIN_LINK_TYPES.map((lt) => (
    <button
      key={lt}
      className={cn(
        "flex items-center gap-1.5 px-2.5 py-1.5 rounded-md text-xs border transition-colors",
        linkType === lt
          ? "bg-primary/10 border-primary"
          : "border-border hover:bg-muted"
      )}
      onClick={() => setLinkType(lt)}
    >
      <span
        className="w-2.5 h-2.5 rounded-full"
        style={{ backgroundColor: LINK_TYPE_COLORS[lt] }}
      />
      <span>{LINK_TYPE_LABELS[lt]}</span>
    </button>
  ))}
  {customLinkTypes.map((ct) => (
    <button
      key={ct.id}
      className={cn(
        "flex items-center gap-1.5 px-2.5 py-1.5 rounded-md text-xs border transition-colors",
        linkType === ct.id
          ? "bg-primary/10 border-primary"
          : "border-border hover:bg-muted"
      )}
      onClick={() => setLinkType(ct.id)}
    >
      <span
        className="w-2.5 h-2.5 rounded-full"
        style={{ backgroundColor: ct.color }}
      />
      <span>{ct.name}</span>
    </button>
  ))}
</div>
</div>

{/* Custom label */}
<div className="space-y-2">
  <div className="text-xs font-medium text-muted-foreground">
    Custom label (optional – overrides the type name on the arrow)
  </div>
  <Input
    value={customLabel}
    onChange={(e) => setCustomLabel(e.target.value)}
    placeholder="e.g., depends on, blocks, references..."
    className="h-8 text-sm"
  />
</div>

{/* Preview + Create */}
<div className="flex items-center justify-between bg-muted/30 rounded-lg p-3">
  <div className="text-xs">
    {sourceItem && targetItem ? (
      <span>
        <strong>{sourceItem.name}</strong>
        <span className="text-muted-foreground mx-2">→</span>
        <strong>{targetItem.name}</strong>
      </span>
    ) : (
      <span className="text-muted-foreground">Select source and target</span>
    )}
  </div>
  <Button
    size="sm"
    disabled={!canCreate}
    onClick={handleCreateLink}
  >
    <Plus size={14} className="mr-1" />
    Create Link
  </Button>
</div>

{/* Existing links */}

```

```
<div className="space-y-2">
  <div className="text-xs font-medium text-muted-foreground">
    Existing links ({existingLinks.length})
  </div>
  {existingLinks.length === 0 ? (
    <div className="text-xs text-muted-foreground italic py-4 text-center">
      No links yet. Create one above.
    </div>
  ) : (
    <div className="space-y-1 max-h-40 overflow-y-auto">
      {existingLinks.map((link) => (
        <div
          key={link.connectionId}
          className="flex items-center gap-2 px-3 py-2 rounded-lg border border-border bg-background
nd text-xs"
        >
          <span
            className="w-3 h-1.5 rounded-full shrink-0"
            style={{ backgroundColor: getLinkTypeColor(link.linkType, customLinkTypes) }}
          />
          <span className="flex-1 truncate">
            <span className="font-medium">{link.sourceName}</span>
            <span className="text-muted-foreground mx-1.5">-></span>
            <span className="font-medium">{link.targetName}</span>
          </span>
          <span className="text-muted-foreground text-[10px]">
            {link.label || getLinkTypeLabel(link.linkType, customLinkTypes)}
          </span>
          <Button
            variant="ghost"
            size="icon"
            className="h-6 w-6 text-destructive hover:text-destructive"
            onClick={() => handleDeleteLink(link.connectionId, link.boardId)}
          >
            <Trash2 size={12} />
          </Button>
        </div>
      ))}
    </div>
  )}
</div>
</div>
</ScrollArea>
</div>
</div>
);
}
```

```
// === END FILE: src/components/whiteboard/LinkManager.tsx ===
```

```
// === START FILE: src/components/whiteboard/LockedModeBar.tsx ===
// File 63 of 194 | Lines: 134 | Size: 5696 bytes
```

```
"use client";
```

```

/**
 * Locked-items mode.
 *
 * A locked thing ignores every touch – that is the point of locking it – and
 * that used to leave no way back: you could not select it, so you could not
 * unlock it. This mode flips the rule for the Select tool: ONLY locked things
 * can be picked (tap, lasso or box), each wears an amber padlock so you can
 * see what is locked, and this bar unlocks the picked ones or all of them.
 * Unlocked things can't be picked meanwhile, so nothing else gets moved by
 * accident. Any other tool, Done, or Esc ends the mode.
 */

```

```
import React from "react";
import { Lock, LockOpen, MousePointerSquareDashed } from "lucide-react";
import { useWhiteboard, useActiveBoard } from "@store/whiteboard";
import { getBoardElements } from "@lib/whiteboard/notebook";
import { cn } from "@lib/utls";
```

```

export function LockedModeBar() {
  const lockedMode = useWhiteboard((s) => s.lockedMode);
  const selectedIds = useWhiteboard((s) => s.selectedIds);
  const board = useActiveBoard();
  const [confirmAll, setConfirmAll] = React.useState(false);
  const [flash, setFlash] = React.useState<string | null>(null);

  React.useEffect(() => {
    if (!lockedMode) return;
    const onKey = (e: KeyboardEvent) => {
      if (e.key === "Escape") useWhiteboard.getState().setLockedMode(false);
    };
    window.addEventListener("keydown", onKey);
    return () => window.removeEventListener("keydown", onKey);
  }, [lockedMode]);

  React.useEffect(() => {
    if (!confirmAll) return;
    const t = setTimeout(() => setConfirmAll(false), 3000);
    return () => clearTimeout(t);
  }, [confirmAll]);

  React.useEffect(() => {
    if (!flash) return;
    const t = setTimeout(() => setFlash(null), 2200);
    return () => clearTimeout(t);
  }, [flash]);

  if (!lockedMode || !board) return null;
  const locked = getBoardElements(board).filter((el) => el.locked && el.type !== "connection");
  const picked = selectedIds.filter((id) => locked.some((el) => el.id === id));

  const unlock = (ids: string[]) => {
    const n = useWhiteboard.getState().unlockElements(ids);
    useWhiteboard.getState().clearSelection();
    setConfirmAll(false);
    if (n) setFlash(n === 1 ? "Unlocked – it moves again with the Select tool" : `${n} items unlocked`);
  };

  const pill = "flex h-9 items-center gap-1.5 rounded-full px-3 text-[12px] font-medium transition active:scale-95";

  return (
    <div
      className="absolute left-1/2 top-3 z-40 flex max-w-[calc(100%-24px)] -translate-x-1/2 flex-col items-center gap-1.5 animate-in fade-in slide-in-from-top-2"
      onPointerDown={(e) => e.stopPropagation()}
    >
      <div className="flex flex-wrap items-center justify-center gap-1 rounded-[22px] border border-amber-400/40 bg-white/95 p-1 pl-3 shadow-xl backdrop-blur-md dark:bg-zinc-900/95">
        <span className="flex items-center gap-1.5 pr-1 text-[12px] font-semibold text-amber-700 dark:text-amber-300">
          <span className="flex h-6 w-6 items-center justify-center rounded-full bg-amber-500 text-white">
            <Lock size={13} />
          </span>
          Locked items
          <span className="font-normal text-muted-foreground">
            · {locked.length === 0 ? "none here" : `${locked.length} on this page`}
          </span>
        </span>

        {picked.length > 0 ? (
          <button onClick={() => unlock(picked)} className={cn(pill, "bg-amber-500 text-white hover:bg-amber-400")}>
            <LockOpen size={14} /> Unlock {picked.length > 1 ? picked.length : ""}
          </button>
        ) : (
          locked.length > 0 && (
            <button onClick={() => useWhiteboard.getState().selectAll()} className={cn(pill, "text-foreground hover:bg-muted")}>
              <MousePointerSquareDashed size={14} /> Select all
            </button>
          )
        )}
      </div>

      {locked.length > 1 && (
        <button
          onClick={() => (confirmAll ? unlock(locked.map((el) => el.id)) : setConfirmAll(true))}
          className={cn(pill, confirmAll ? "bg-rose-500 text-white hover:bg-rose-400" : "text-foreground hov

```

```

        er:bg-muted"))}
      >
      <LockOpen size={14} /> {confirmAll ? `Unlock all ${locked.length}?` : "Unlock all"}
    </button>
  )}
  <button
    onClick={() => useWhiteboard.getState().setLockedMode(false)}
    className={cn(pill, "bg-foreground text-background hover:opacity-90")}
  >
    Done
  </button>
</div>
<div className="rounded-full bg-black/60 px-3 py-1 text-[11px] text-white backdrop-blur">
  {flash ?? "Tap, lasso or drag a box over a locked item"}
</div>
</div>
);
}

/** The Select tool menu's switch into (and out of) locked-items mode. */
export function LockedModeMenuItem({ onDone }: { onDone: () => void }) {
  const lockedMode = useWhiteboard((s) => s.lockedMode);
  const count = useWhiteboard((s) => {
    const b = s.boards.find((x) => x.id === s.activeBoardId);
    return b ? getBoardElements(b).filter((el) => el.locked && el.type !== "connection").length : 0;
  });
  return (
    <button
      onClick={() => {
        useWhiteboard.getState().setLockedMode(!lockedMode);
        onDone();
      }}
      className={cn(
        "mt-0.5 flex w-full items-center gap-2 rounded-lg px-2 py-1.5 text-[12px]",
        lockedMode ? "bg-amber-500/15 text-amber-700 dark:text-amber-300" : "hover:bg-muted"
      )}
    >
      {lockedMode ? <LockOpen size={15} /> : <Lock size={15} />}
      <span className="flex-1 text-left">{lockedMode ? "Leave locked items" : "Locked items..."}</span>
      <span className="text-[10px] text-muted-foreground">{count}</span>
    </button>
  );
}

=====
// === END FILE: src/components/whiteboard/LockedModeBar.tsx ===
=====

=====
// === START FILE: src/components/whiteboard/Minimap.tsx ===
// File 64 of 194 | Lines: 223 | Size: 8008 bytes
=====

"use client";

import React, { useRef, useCallback, useState, useMemo } from "react";
import { Map, X, Maximize2, Minimize2, Navigation } from "lucide-react";
import { useWhiteboard, useActiveBoard } from "@store/whiteboard";
import { getBoardElements } from "@lib/whiteboard/notebook";
import {
  getElementBounds,
  getUnionBounds,
  screenToWorld,
  type Viewport,
  type BoundingBox,
} from "@lib/whiteboard/geometry";
import { STICKY_COLORS } from "@lib/whiteboard/types";
import { cn } from "@lib/utils";

interface MinimapProps {
  viewport?: Viewport;
}

export function Minimap({ viewport = { width: 1200, height: 800 } }: MinimapProps) {
  const minimapOpen = useWhiteboard((s) => s.minimapOpen);
  const toggleMinimap = useWhiteboard((s) => s.toggleMinimap);
  const camera = useWhiteboard((s) => s.camera);

```

```

const setCamera = useWhiteboard((s) => s.setCamera);
const activeBoard = useActiveBoard();

const [isMinimized, setIsMinimized] = useState(false);
const [isDragging, setIsDragging] = useState(false);
const containerRef = useRef<HTMLDivElement>(null);

const MINIMAP_WIDTH = 200;
const MINIMAP_HEIGHT = 130;
const PADDING = 200; // World units margin around content

const elements = getBoardElements(activeBoard);

// 1. Calculate the active viewport bounding box in World coordinates
const vpWorldW = viewport.width / Math.max(camera.zoom, 0.001);
const vpWorldH = viewport.height / Math.max(camera.zoom, 0.001);
const vpWorldX = camera.x - vpWorldW / 2;
const vpWorldY = camera.y - vpWorldH / 2;

// 2. Compute composite total bounds covering all elements AND the current camera viewport
const totalBounds: BoundingBox = useMemo(() => {
  const rawBounds = getUnionBounds(elements);
  let minX = vpWorldX;
  let minY = vpWorldY;
  let maxX = vpWorldX + vpWorldW;
  let maxY = vpWorldY + vpWorldH;

  if (rawBounds) {
    minX = Math.min(minX, rawBounds.x);
    minY = Math.min(minY, rawBounds.y);
    maxX = Math.max(maxX, rawBounds.x + rawBounds.width);
    maxY = Math.max(maxY, rawBounds.y + rawBounds.height);
  }

  const paddedW = Math.max(maxX - minX + PADDING * 2, 800);
  const paddedH = Math.max(maxY - minY + PADDING * 2, 600);

  return {
    x: minX - PADDING,
    y: minY - PADDING,
    width: paddedW,
    height: paddedH,
  };
}, [elements, vpWorldX, vpWorldY, vpWorldW, vpWorldH]);

// 3. Mapping scale and centering
const scale = useMemo(() => {
  const sX = MINIMAP_WIDTH / totalBounds.width;
  const sY = MINIMAP_HEIGHT / totalBounds.height;
  return Math.min(sX, sY);
}, [totalBounds.width, totalBounds.height]);

const offsetX = (MINIMAP_WIDTH - totalBounds.width * scale) / 2;
const offsetY = (MINIMAP_HEIGHT - totalBounds.height * scale) / 2;

// Transform functions: World <-> Minimap
const worldToMini = useCallback(
  (wx: number, wy: number) => {
    const mx = (wx - totalBounds.x) * scale + offsetX;
    const my = (wy - totalBounds.y) * scale + offsetY;
    return { x: mx, y: my };
  },
  [totalBounds.x, totalBounds.y, scale, offsetX, offsetY]
);

const miniToWorld = useCallback(
  (mx: number, my: number) => {
    const wx = (mx - offsetX) / scale + totalBounds.x;
    const wy = (my - offsetY) / scale + totalBounds.y;
    return { x: wx, y: wy };
  },
  [totalBounds.x, totalBounds.y, scale, offsetX, offsetY]
);

// Viewport box in minimap coordinates
const vpMiniPos = worldToMini(vpWorldX, vpWorldY);
const vpMiniW = Math.max(vpWorldW * scale, 8);
const vpMiniH = Math.max(vpWorldH * scale, 6);

```

```

// Handle pointer down / drag to pan camera
const handlePointerAction = (e: React.PointerEvent<HTMLDivElement>) => {
  if (!containerRef.current) return;
  const rect = containerRef.current.getBoundingClientRect();
  const mx = e.clientX - rect.left;
  const my = e.clientY - rect.top;
  const worldPt = miniToWorld(mx, my);
  setCamera({ x: worldPt.x, y: worldPt.y });
};

const onPointerDown = (e: React.PointerEvent<HTMLDivElement>) => {
  e.stopPropagation();
  setIsDragging(true);
  (e.target as Element).setPointerCapture?.(e.pointerId);
  handlePointerAction(e);
};

const onPointerMove = (e: React.PointerEvent<HTMLDivElement>) => {
  if (!isDragging) return;
  e.stopPropagation();
  handlePointerAction(e);
};

const onPointerUp = (e: React.PointerEvent<HTMLDivElement>) => {
  if (isDragging) {
    e.stopPropagation();
    setIsDragging(false);
  }
};

if (!minimapOpen) return null;

return (
  <div className="absolute bottom-6 right-6 z-30 flex flex-col bg-white/95 dark:bg-zinc-900/95 backdrop-blur
    -xl border border-border/80 shadow-2xl rounded-2xl overflow-hidden select-none animate-in fade-in slid
    e-in-from-bottom-2">
    {/* Header */}
    <div className="flex items-center justify-between px-3 py-1.5 border-b border-border/60 bg-muted/30">
      <div className="flex items-center gap-1.5 text-xs font-semibold text-foreground">
        <Navigation size={12} className="text-primary" />
        <span>Minimap</span>
      </div>
      <div className="flex items-center gap-0.5">
        <button
          onClick={() => setIsMinimized(!isMinimized)}
          className="p-1 rounded-md hover:bg-muted text-muted-foreground hover:text-foreground"
          title={isMinimized ? "Expand minimap" : "Minimize minimap"}
        >
          {isMinimized ? <Maximize2 size={12} /> : <Minimize2 size={12} />}
        </button>
        <button
          onClick={toggleMinimap}
          className="p-1 rounded-md hover:bg-muted text-muted-foreground hover:text-foreground"
          title="Close minimap (M)"
        >
          <X size={12} />
        </button>
      </div>
    </div>
    </div>

    {/* Minimap Body */}
    {!isMinimized && (
      <div
        ref={containerRef}
        onPointerDown={onPointerDown}
        onPointerMove={onPointerMove}
        onPointerUp={onPointerUp}
        onPointerCancel={onPointerUp}
        className="relative bg-muted/10 cursor-crosshair overflow-hidden touch-none"
        style={{ width: MINIMAP_WIDTH, height: MINIMAP_HEIGHT }}
      >
        {/* Miniature Elements SVG */}
        <svg className="w-full h-full absolute inset-0 pointer-events-none">
          {elements.map((el) => {
            if (el.type === "connection") return null;
            const b = getElementBounds(el);
            const miniPos = worldToMini(b.x, b.y);

```

```

const miniW = Math.max(b.width * scale, 2);
const miniH = Math.max(b.height * scale, 2);

let fill = "#94a3b8";
if (el.type === "sticky") {
  const colors = STICKY_COLORS[(el as { color?: string }).color as keyof typeof STICKY_COLORS];
  fill = colors?.bg || "#fef08a";
} else if (el.type === "text") {
  fill = "#6366f1";
} else if ((el as { strokeColor?: string }).strokeColor) {
  fill = (el as { strokeColor: string }).strokeColor;
} else if ((el as { color?: string }).color) {
  fill = (el as { color: string }).color;
}

return (
  <rect
    key={el.id}
    x={miniPos.x}
    y={miniPos.y}
    width={miniW}
    height={miniH}
    rx={1}
    fill={fill}
    opacity={0.75}
  />
);
}}
}

{/* Active Camera Viewport Box */}
<rect
  x={vpMiniPos.x}
  y={vpMiniPos.y}
  width={vpMiniW}
  height={vpMiniH}
  rx={2}
  className="fill-primary/20 stroke-primary stroke-[1.5]"
  style={{ filter: "drop-shadow(0 1px 3px rgba(0,0,0,0.15))" }}
/>
</svg>
</div>
)}
</div>
);
}

```

```

=====
// === END FILE: src/components/whiteboard/Minimap.tsx ===
=====

```

```

=====
// === START FILE: src/components/whiteboard/MoveToDialog.tsx ===
// File 65 of 194 | Lines: 549 | Size: 19666 bytes
=====

```

```
"use client";
```

```
/**
```

```

 * Move-to dialog: file a board/notebook or folder anywhere by browsing.
 *
 * Why a dialog and not only drag-and-drop: drag needs a visible target, but a
 * destination five levels deep (or a 200-board sidebar) is faster reached by
 * drilling down a path – or by typing a name. The dialog shows top-level
 * folders, tapping one walks into it (breadcrumb across the top walks back
 * out), and "Move here" files the item where you stand. Search jumps straight
 * to any folder. The store still rejects cycles, self-drops and ghosts, so the
 * dialog only ever disables what would be a no-op.
 */

```

```

import React from "react";
import {
  Check,
  ChevronRight,
  Folder as FolderIcon,
  FolderInput,
  House,
  NotebookText,

```

```

    Search,
    X,
  } from "lucide-react";
import { useWhiteboard } from "@store/whiteboard";
import { getPages, isNotebook } from "@lib/whiteboard/notebook";
import type { Board, Folder } from "@lib/whiteboard/types";
import { isDescendantFolder } from "../Sidebar";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import {
  Dialog,
  DialogContent,
  DialogDescription,
  DialogFooter,
  DialogHeader,
  DialogTitle,
} from "@components/ui/dialog";
import { cn } from "@lib/utils";

export interface MoveToItem {
  kind: "board" | "folder";
  id: string;
  name: string;
}

export type DestinationTarget =
  | { kind: "folder"; folderId: string | null; name: string }
  | { kind: "notebook"; boardId: string; name: string; pageCount: number };

/** Folder ids from the top level down to `id` (inclusive). Empty = top level. */
export function chainTo(folders: Folder[], id: string | null): string[] {
  if (id === null) return [];
  const chain: string[] = [];
  const seen = new Set<string>();
  let cursor: string | null = id;
  while (cursor && !seen.has(cursor)) {
    seen.add(cursor);
    chain.unshift(cursor);
    cursor = folders.find((f) => f.id === cursor)?.parentId ?? null;
  }
  return chain;
}

export function folderName(folders: Folder[], id: string): string {
  return folders.find((f) => f.id === id)?.name ?? "...";
}

export interface DestinationBrowserProps {
  selectedTarget: DestinationTarget;
  onSelectTarget: (target: DestinationTarget) => void;
  allowNotebookTarget?: boolean;
  excludeBoardId?: string;
  moveItem?: MoveToItem | null;
  className?: string;
}

export function DestinationBrowser({
  selectedTarget,
  onSelectTarget,
  allowNotebookTarget = false,
  excludeBoardId,
  moveItem,
  className,
}: DestinationBrowserProps) {
  const folders = useWhiteboard((s) => s.folders);
  const boards = useWhiteboard((s) => s.boards);

  const [path, setPath] = React.useState<string[]>([]);
  const [query, setQuery] = React.useState("");

  const destId = path.length > 0 ? path[path.length - 1] : null;
  const destLabel = destId === null ? "Top level" : folderName(folders, destId);

  const childrenFolders = folders.filter((f) => (f.parentId ?? null) === destId);
  const notebooksHere = allowNotebookTarget
    ? boards.filter(
        (b) =>
          isNotebook(b) &&

```

```

        (b.folderId ?? null) === destId &&
        (!excludeBoardId || b.id !== excludeBoardId)
    )
    : [];

const containedCount = (folderId: string) =>
    folders.filter((f) => f.parentId === folderId).length +
    boards.filter((b) => b.folderId === folderId).length;

const q = query.trim().toLowerCase();
const searchFolderHits = q
    ? folders.filter((f) => f.name.toLowerCase().includes(q))
    : [];
const searchNotebookHits =
    q && allowNotebookTarget
    ? boards.filter(
        (b) =>
            isNotebook(b) &&
            b.name.toLowerCase().includes(q) &&
            (!excludeBoardId || b.id !== excludeBoardId)
        )
    : [];

const selectFolder = (folderId: string | null) => {
    onSelectTarget({
        kind: "folder",
        folderId,
        name: folderId === null ? "Top level" : folderName(folders, folderId),
    });
};

const selectNotebook = (b: Board) => {
    onSelectTarget({
        kind: "notebook",
        boardId: b.id,
        name: b.name,
        pageCount: getPages(b).length,
    });
};

return (
    <div className={cn("space-y-2", className)}>
        {/* Search box */}
        <div className="relative">
            <Search
                size={13}
                className="absolute left-2.5 top-1/2 -translate-y-1/2 text-muted-foreground"
            />
            <Input
                value={query}
                onChange={(e) => setQuery(e.target.value)}
                placeholder={allowNotebookTarget ? "Search folders & notebooks..." : "Search folders..."}
                aria-label={allowNotebookTarget ? "Search folders and notebooks" : "Search folders"}
                className="h-8 pl-8 pr-7 text-xs"
            />
            {query && (
                <button
                    type="button"
                    onClick={() => setQuery("")}
                    aria-label="Clear search"
                    className="absolute right-1.5 top-1/2 -translate-y-1/2 rounded p-0.5 text-muted-foreground hover:text-foreground"
                >
                    <X size={13} />
                </button>
            )}
        </div>

        {q ? (
            {/* Flat hits across folders and notebooks */}
            <div className="max-h-56 overflow-y-auto space-y-1 pr-1">
                {searchFolderHits.length === 0 && searchNotebookHits.length === 0 ? (
                    <p className="px-2 py-6 text-center text-xs text-muted-foreground">
                        Nothing matches "{query}"
                    </p>
                ) : (
                    <>
                        {searchFolderHits.map((f) => (

```

```

<button
  key={f.id}
  type="button"
  onClick={() => {
    setPath(chainTo(folders, f.id));
    setQuery("");
    selectFolder(f.id);
  }}
  className="flex w-full items-center gap-2 rounded-lg px-2 py-1.5 text-left transition-colors
  hover:bg-muted"
>
  <FolderIcon
    size={15}
    className="shrink-0"
    style={{ color: f.color || "#f59e0b" }}
  />
  <span className="min-w-0 flex-1">
    <span className="block truncate text-xs font-medium">{f.name}</span>
    <span className="block truncate text-[10px] text-muted-foreground">
      {chainTo(folders, f.parentId ?? null)
        .map((id) => folderName(folders, id))
        .join(" / ") || "Top level"}
    </span>
  </span>
  <ChevronRight size={14} className="shrink-0 text-muted-foreground" />
</button>
)))
{searchNotebookHits.map((nb) => {
  const isSelected =
    selectedTarget.kind === "notebook" && selectedTarget.boardId === nb.id;
  return (
    <button
      key={nb.id}
      type="button"
      onClick={() => {
        selectNotebook(nb);
        setQuery("");
      }}
      className={cn(
        "flex w-full items-center gap-2 rounded-lg px-2 py-1.5 text-left transition-colors",
        isSelected
          ? "bg-primary/10 text-primary border border-primary/40 font-medium"
          : "hover:bg-muted"
      )}
    >
      <NotebookText size={15} className="shrink-0 text-primary" />
      <span className="min-w-0 flex-1">
        <span className="block truncate text-xs font-medium">{nb.name}</span>
        <span className="block truncate text-[10px] text-muted-foreground">
          {chainTo(folders, nb.folderId ?? null)
            .map((id) => folderName(folders, id))
            .join(" / ") || "Top level"}
          </span>
      </span>
      <span className="shrink-0 text-[10px] text-muted-foreground tabular-nums">
        {getPages(nb).length} pages
      </span>
    </button>
  );
});
</div>
) : (
  <>
    {/* Breadcrumb back out */}
    <div className="flex items-center gap-1 overflow-x-auto py-1">
      <button
        type="button"
        onClick={() => {
          setPath([]);
          selectFolder(null);
        }}
        title="Top level"
        aria-label="Go to top level"
        className={cn(
          "flex shrink-0 items-center gap-1 rounded-md px-1.5 py-1 text-[11px] transition-colors",
          path.length === 0 && selectedTarget.kind === "folder"

```

```

      ? "bg-primary/10 font-semibold text-primary"
      : "text-muted-foreground hover:bg-muted"
    )}
  >
  <House size={12} />
</button>
{path.map((id, i) => (
  <React.Fragment key={id}>
    <ChevronRight size={12} className="shrink-0 text-muted-foreground/60" />
    <button
      type="button"
      onClick={() => {
        setPath(path.slice(0, i + 1));
        selectFolder(id);
      }}
      className={cn(
        "shrink-0 truncate rounded-md px-1.5 py-1 text-[11px] transition-colors max-w-[120px]",
        i === path.length - 1 && selectedTarget.kind === "folder"
          ? "bg-primary/10 font-semibold text-primary"
          : "text-muted-foreground hover:bg-muted"
      )}
    >
      {folderName(folders, id)}
    </button>
  </React.Fragment>
)}}
</div>

{/* Folder & notebook items list */}
<div className="max-h-56 overflow-y-auto space-y-1 pr-1 border border-border/60 rounded-lg p-1">
  {/* When notebook target is allowed and user is in a folder, give option to stay on folder */}
  {allowNotebookTarget && (
    <button
      type="button"
      onClick={() => selectFolder(destId)}
      className={cn(
        "flex w-full items-center gap-2 rounded-md px-2 py-1.5 text-left transition-colors",
        selectedTarget.kind === "folder" && selectedTarget.folderId === destId
          ? "bg-primary/10 text-primary border border-primary/30 font-medium"
          : "hover:bg-muted/70 text-muted-foreground"
      )}
    >
      <FolderIcon size={14} className="shrink-0" />
      <span className="flex-1 text-xs truncate">
        {destId === null ? "Top level (no folder)" : `This folder (${destLabel})`}
      </span>
      {selectedTarget.kind === "folder" && selectedTarget.folderId === destId && (
        <Check size={13} className="text-primary shrink-0" />
      )}
    </button>
  )}
  {childrenFolders.length === 0 && notebooksHere.length === 0 ? (
    <p className="px-2 py-4 text-center text-xs text-muted-foreground">
      {destId === null && folders.length === 0
        ? "No folders yet."
        : "No subfolders or notebooks here."}
    </p>
  ) : null}

  {childrenFolders.map((f) => {
    const invalid =
      moveItem?.kind === "folder" &&
      (f.id === moveItem.id || isDescendantFolder(folders, moveItem.id, f.id));
    return (
      <button
        key={f.id}
        type="button"
        disabled={invalid}
        onClick={() => {
          setPath([...path, f.id]);
          selectFolder(f.id);
        }}
        title={invalid ? "Cannot move a folder into itself" : `Open ${f.name}`}
        className={cn(
          "flex w-full items-center gap-2 rounded-md px-2 py-1.5 text-left transition-colors",
          invalid ? "opacity-40" : "hover:bg-muted"
        )}
      >

```

```

    >
    <FolderIcon
      size={15}
      className="shrink-0"
      style={{ color: f.color || "#f59e0b" }}
    />
    <span className="min-w-0 flex-1 truncate text-xs font-medium">
      {f.name}
    </span>
    <span className="shrink-0 text-[10px] tabular-nums text-muted-foreground">
      {containedCount(f.id) > 0 ? containedCount(f.id) : ""}
    </span>
    <ChevronRight size={14} className="shrink-0 text-muted-foreground" />
  </button>
);
}})

{notebooksHere.map((nb) => {
  const isSelected =
    selectedTarget.kind === "notebook" && selectedTarget.boardId === nb.id;
  return (
    <button
      key={nb.id}
      type="button"
      onClick={() => selectNotebook(nb)}
      className={cn(
        "flex w-full items-center gap-2 rounded-md px-2 py-1.5 text-left transition-colors",
        isSelected
          ? "bg-primary/10 text-primary border border-primary/40 font-medium"
          : "hover:bg-muted"
      )}
    >
      <NotebookText size={15} className="shrink-0 text-primary" />
      <span className="min-w-0 flex-1 truncate text-xs font-medium">
        {nb.name}
      </span>
      <span className="shrink-0 text-[10px] tabular-nums text-muted-foreground">
        {getPages(nb).length} pages
      </span>
      {isSelected && <Check size={13} className="text-primary shrink-0 ml-1" />}
    </button>
  );
});
</div>
</>
)
</div>
);
}

export function MoveToDialog({
  open,
  onClose,
  item,
  currentParentId,
}): {
  open: boolean;
  onClose: () => void;
  item: MoveToItem | null;
  currentParentId: string | null;
} {
  const folders = useWhiteboard((s) => s.folders);
  const moveBoard = useWhiteboard((s) => s.moveBoard);
  const moveFolder = useWhiteboard((s) => s.moveFolder);

  const [target, setTarget] = React.useState<DestinationTarget>({
    kind: "folder",
    folderId: null,
    name: "Top level",
  });

  React.useEffect(() => {
    if (open) {
      setTarget({ kind: "folder", folderId: null, name: "Top level" });
    }
  }, [open, item?.id]);

  if (!item) return null;

```

```

const destId = target.kind === "folder" ? target.folderId : null;

// A destination is useless (button disabled, never applied) when it is where
// the item already is, the item itself, or – for folders – its own subtree.
const destInvalid =
  destId === currentParentId ||
  (item.kind === "folder" &&
    destId !== null &&
    (destId === item.id || isDescendantFolder(folders, item.id, destId)));

const applyMove = () => {
  if (destInvalid) return;
  if (item.kind === "board") moveBoard(item.id, destId);
  else moveFolder(item.id, destId);
  onClose();
};

const destLabel = destId === null ? "Top level" : folderName(folders, destId);

return (
  <Dialog open={open} onOpenChange={{o} => !o && onClose()}>
    <DialogContent className="max-w-sm p-0 gap-0 overflow-hidden">
      <DialogHeader className="px-4 pb-2 pt-4 text-left">
        <DialogTitle className="flex items-center gap-2 text-base">
          <FolderInput size={17} className="shrink-0 text-primary" />
          Move "{item.name}"
        </DialogTitle>
        <DialogDescription className="text-xs">
          {item.kind === "board"
            ? "Pick the folder it should live in."
            : "Pick the folder it should move into."}
        </DialogDescription>
      </DialogHeader>

      <div className="px-4 pb-2">
        <DestinationBrowser
          selectedTarget={target}
          onSelectTarget={setTarget}
          allowNotebookTarget={false}
          moveItem={item}
        />
      </div>

      <DialogFooter className="flex-row items-center justify-between gap-2 border-t border-border px-4 py-3">
        >
        <Button variant="ghost" size="sm" onClick={onClose}>
          Cancel
        </Button>
        <Button size="sm" disabled={destInvalid} onClick={applyMove} className="gap-1.5">
          <Check size={14} />
          {destId === currentParentId
            ? "Already here"
            : destId === null
              ? "Move to top level"
              : `Move to ${destLabel}`}
        </Button>
      </DialogFooter>
    </DialogContent>
  </Dialog>
);
}

export function BatchMoveDialog({
  open,
  onClose,
  boardIds,
  folderIds = [],
  onMoved,
}): {
  open: boolean;
  onClose: () => void;
  boardIds: string[];
  /** Folders move one by one; the store rejects cycles, so no bad move applies. */
  folderIds?: string[];
  onMoved?: () => void;
} {
  const folders = useWhiteboard((s) => s.folders);

```

```

const batchMoveBoards = useWhiteboard((s) => s.batchMoveBoards);
const moveFolder = useWhiteboard((s) => s.moveFolder);

const [target, setTarget] = React.useState<DestinationTarget>({
  kind: "folder",
  folderId: null,
  name: "Top level",
});

React.useEffect(() => {
  if (open) {
    setTarget({ kind: "folder", folderId: null, name: "Top level" });
  }
}, [open]);

const total = boardIds.length + folderIds.length;
if (!open || total === 0) return null;

const destId = target.kind === "folder" ? target.folderId : null;
const destLabel = destId === null ? "Top level" : folderName(folders, destId);

const applyMove = () => {
  batchMoveBoards(boardIds, destId);
  for (const id of folderIds) moveFolder(id, destId);
  onMoved?();
  onClose();
};

return (
  <Dialog open={open} onOpenChange={(o) => !o && onClose()}>
    <DialogContent className="max-w-sm p-0 gap-0 overflow-hidden">
      <DialogHeader className="px-4 pb-2 pt-4 text-left">
        <DialogTitle className="flex items-center gap-2 text-base">
          <FolderInput size={17} className="shrink-0 text-primary" />
          Move {total} {total === 1 ? "Item" : "Items"}
        </DialogTitle>
        <DialogDescription className="text-xs">
          Pick the destination folder for selected items.
        </DialogDescription>
      </DialogHeader>

      <div className="px-4 pb-2">
        <DestinationBrowser
          selectedTarget={target}
          onSelectTarget={setTarget}
          allowNotebookTarget={false}
        />
      </div>

      <DialogFooter className="flex-row items-center justify-between gap-2 border-t border-border px-4 py-3">
        <Button variant="ghost" size="sm" onClick={onClose}>
          Cancel
        </Button>
        <Button size="sm" onClick={applyMove} className="gap-1.5">
          <Check size={14} />
          {destId === null ? "Move to top level" : `Move to ${destLabel}`}
        </Button>
      </DialogFooter>
    </DialogContent>
  </Dialog>
);
}

=====
// === END FILE: src/components/whiteboard/MoveToDialog.tsx ===
=====

=====
// === START FILE: src/components/whiteboard/NotebookPages.tsx ===
// File 66 of 194 | Lines: 910 | Size: 35396 bytes
=====

"use client";

/**
 * Notebook page navigation

```

```

* =====
* Two pieces: a compact page bar that is always visible in a notebook, and a
* page panel you can open beside it.
*
* The panel was rebuilt for touch after it was tested on a Galaxy Tab S9:
*
* - Most page actions only appeared on mouse HOVER (`opacity-0
*   group-hover:opacity-100`). A tablet has no hover, so bookmark, insert and
*   save-as-template were invisible, and the two that showed were 11 px icons.
*   Every action now lives in an always-visible ■ menu with full-size items.
*
* - Reordering was HTML5 drag-and-drop, which a finger cannot drive in an
*   Android WebView, and its drop handler called `movePage(target, from)` -
*   moving the page you dropped ONTO back to where you started. It only looked
*   right for neighbours. Reorder is now a pointer drag on a grip, and it moves
*   the page you are holding to where you let go.
*
* - Thumbnails were grey bounding boxes, so every page of a handwritten or PDF
*   notebook looked the same. They now draw the page's own paper, real ink
*   and the PDF page.
*
* - Deleting a page was one unconfirmed tap next to Duplicate, and page
*   deletion is not undoable. It now asks, unless the page is empty.
*
* - Going to a page kept the scroll offset of the page you left, so you often
*   arrived half way down. It now shows the top of the page.
*/

```

```

import React from "react";
import { useTheme } from "next-themes";
import {
  ArrowDownToLine,
  ArrowUpToLine,
  Check,
  ChevronLeft,
  ChevronRight,
  Copy,
  GripVertical,
  ImagePlus,
  LayoutGrid,
  ListOrdered,
  MoreHorizontal,
  Plus,
  Save,
  Star,
  Trash2,
  X,
} from "lucide-react";
import { useWhiteboard, useActiveBoard } from "@/store/whiteboard";
import {
  bookmarkedPages,
  getActivePageIndex,
  getPages,
  isNotebook,
  pageSizeOf,
  paperForPage,
} from "@/lib/whiteboard/notebook";
import { PaperTiles, paperColorFor } from "@/PaperPattern";
import {
  BOARD_PATTERNS,
  BOARD_PATTERN_DEFAULT_SIZE,
  BOARD_PATTERN_MAX_SIZE,
  BOARD_PATTERN_MIN_SIZE,
  type BoardPattern,
  type NotebookPage,
} from "@/lib/whiteboard/types";
import { StepperRow } from "@/StepperControl";
import { storeImage } from "@/lib/whiteboard/imagestore";
import { ElementRenderer } from "@/ElementRenderer";
import { PdfPageLayer } from "@/PdfPageLayer";
import { Button } from "@/components/ui/button";
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuLabel,
  DropdownMenuSeparator,
  DropdownMenuTrigger,
}

```

```

} from "@components/ui/dropdown-menu";
import {
  Dialog,
  DialogContent,
  DialogDescription,
  DialogHeader,
  DialogTitle,
} from "@components/ui/dialog";
import { cn } from "@lib/utils";

/** Width of the page panel. The page bar sits just to its right when open. */
export const PANEL_W = 212;

/** Press-and-hold this long (without moving) to pick a page card up for dragging. */
const HOLD_TO_DRAG_MS = 300;
/**
 * Moving further than this before the hold fires means a scroll, not a
 * pickup. Generous on purpose: a hand holding a tablet trembles, and every
 * cancelled hold is a drag that never starts.
 */
const HOLD_MOVE_PX = 24;
/** Dragging within this far of the panel's top/bottom edge scrolls it. */
const DRAG_SCROLL_EDGE_PX = 64;
const DRAG_SCROLL_STEP_PX = 24;

/** The canvas fills the window height closely enough to aim "page top" at. */
const viewportH = () => (typeof window !== "undefined" ? window.innerHeight : 800);

// -----
// Page bar
// -----

/**
 * The page panel's mount point. The compact controls that used to live here
 * too are now `NotebookPageControls`, rendered inside the tool dock.
 */
export function NotebookPagePanelHost() {
  const panelOpen = useWhiteboard((s) => s.pagePanelOpen);
  const setPagePanelOpen = useWhiteboard((s) => s.setPagePanelOpen);
  return panelOpen ? <PagePanel onClose={() => setPagePanelOpen(false)} /> : null;
}

/**
 * Which page you are on, and how to get to another one – sized and shaped to
 * sit INSIDE the tool dock, immediately beside the tools.
 *
 * It used to be a floating bar pinned to the bottom-left corner, which meant
 * it drifted into the dock on a narrow pane and needed a ResizeObserver that
 * measured both and lifted the bar out of the way. Living in the dock's own
 * row, it cannot collide with the dock: flexbox simply puts it alongside.
 *
 * There is no "add page" here. Adding is a deliberate, occasional act that
 * belongs with the other page management in the panel; what you want at the
 * edge of the canvas, constantly, is to know where you are and jump.
 */
export function NotebookPageControls() {
  const activeBoard = useActiveBoard();
  const goToPage = useWhiteboard((s) => s.goToPage);
  const panelOpen = useWhiteboard((s) => s.pagePanelOpen);
  const togglePagePanel = useWhiteboard((s) => s.togglePagePanel);

  const pages = getPages(activeBoard);
  const index = getActivePageIndex(activeBoard);

  // Typing a page number beats tapping "next" forty times in a long notebook.
  const [typing, setTyping] = React.useState(false);
  const [draft, setDraft] = React.useState("");
  const inputRef = React.useRef<HTMLInputElement>(null);
  React.useEffect(() => {
    if (typing) inputRef.current?.select();
  }, [typing]);

  if (!isNotebook(activeBoard) || pages.length === 0) return null;

  const atFirst = index <= 0;
  const atLast = index >= pages.length - 1;

  const go = (i: number) => {

```

```

    const target = pages[i];
    if (target) goToPage(target.id, viewportH());
  };

  const commit = () => {
    setTyping(false);
    const n = Number.parseInt(draft, 10);
    if (!Number.isFinite(n)) return;
    // Out-of-range is clamped rather than refused: "500" in a 30-page notebook
    // plainly means the end, and an error for that would be pedantry.
    go(Math.min(pages.length, Math.max(1, n)) - 1);
  };

  // Compact: it shares the dock's line with the tools, on their left. In a
  // narrow pane the arrows step aside (the number is tap-to-type, the grid
  // opens every page), so the tools keep the width.
  const arrowBtn =
    "flex h-8 w-6 @max-[40rem]:hidden items-center justify-center rounded-lg text-muted-foreground transition-
    colors hover:bg-muted hover:text-foreground disabled:pointer-events-none disabled:opacity-40";

  return (
    <div className="flex shrink-0 items-center gap-0 rounded-2xl border border-border/80 bg-white/95 p-1 shado
    w-2xl backdrop-blur-xl dark:bg-zinc-900/95">
      <button
        onClick={togglePagePanel}
        title="All pages"
        aria-label="All pages"
        aria-pressed={panelOpen}
        className={cn(
          "flex h-8 w-8 items-center justify-center rounded-lg transition-colors",
          panelOpen ? "bg-primary text-primary-foreground" : "text-muted-foreground hover:bg-muted"
        )}
      >
        <LayoutGrid size={15} />
      </button>

      <span className="mx-0.5 h-5 w-px shrink-0 bg-border" />

      <button
        onClick={() => go(index - 1)}
        disabled={atFirst}
        title="Previous page"
        aria-label="Previous page"
        className={arrowBtn}
      >
        <ChevronLeft size={15} />
      </button>

      {typing ? (
        <input
          ref={inputRef}
          value={draft}
          onChange={(e) => setDraft(e.target.value.replace(/^[0-9]/g, ""))}
          onKeyDown={(e) => {
            e.stopPropagation(); // the canvas owns single-key shortcuts
            if (e.key === "Enter") commit();
            else if (e.key === "Escape") setTyping(false);
          }}
          onBlur={commit}
          type="text"
          inputMode="numeric"
          aria-label={`Page number, 1 to ${pages.length}`}
          className="h-8 w-[46px] rounded-lg border border-primary bg-transparent px-0.5 text-center text-[11p
            x] font-semibold tabular-nums outline-none"
        />
      ) : (
        <button
          onClick={() => {
            setDraft(String(index + 1));
            setTyping(true);
          }}
          title="Go to page..."
          aria-label={`Page ${index + 1} of ${pages.length}. Go to page...`}
          className="h-8 min-w-[46px] rounded-lg px-0.5 text-center text-[11px] font-semibold tabular-nums tex
            t-muted-foreground transition-colors hover:bg-muted hover:text-foreground"
        >
          {index + 1} / {pages.length}
        </button>
      )}
    </div>
  );

```

```

    })
    <button
      onClick={() => go(index + 1)}
      disabled={atLast}
      title="Next page"
      aria-label="Next page"
      className={arrowBtn}
    >
      <ChevronRight size={15} />
    </button>
  </div>
);
}

// -----
// Page panel
// -----

function PagePanel({ onClose }: { onClose: () => void }) {
  const activeBoard = useActiveBoard();
  const goToPage = useWhiteboard((s) => s.goToPage);
  const insertPageAt = useWhiteboard((s) => s.insertPageAt);
  const pageTemplates = useWhiteboard((s) => s.pageTemplates);
  const addPageFromTemplate = useWhiteboard((s) => s.addPageFromTemplate);
  const deletePageTemplate = useWhiteboard((s) => s.deletePageTemplate);
  const createImageTemplate = useWhiteboard((s) => s.createImageTemplate);
  const movePage = useWhiteboard((s) => s.movePage);
  const [tab, setTab] = React.useState<"all" | "fav">("all");
  const imageInputRef = React.useRef<HTMLInputElement>(null);

  const pages = getPages(activeBoard);
  const favourites = bookmarkedPages(activeBoard);
  const activeIndex = getActivePageIndex(activeBoard);
  const dim = pageSizeOf(activeBoard);
  const { resolvedTheme } = useTheme();

  // --- pointer-drag reorder -----
  const cardRefs = React.useRef<Array<HTMLDivElement | null>>([]);
  const [drag, setDrag] = React.useState<{ from: number; to: number } | null>(null);

  const targetIndexFor = (clientY: number, from: number) => {
    // New position = how many OTHER cards have their middle above the pointer.
    let to = 0;
    cardRefs.current.forEach((el, i) => {
      if (!el || i === from) return;
      const r = el.getBoundingClientRect();
      if (r.top + r.height / 2 < clientY) to++;
    });
    return to;
  };

  const startDrag = (e: React.PointerEvent, from: number) => {
    e.preventDefault();
    try {
      (e.currentTarget as Element).setPointerCapture?.(e.pointerId);
    } catch {
      // Throws if the pointer is no longer active - a real race when a finger
      // lifts before the handler runs. Unguarded it aborted the drag before
      // it started; losing capture only costs moves outside the panel.
    }
    setDrag({ from, to: from });
  };

  const moveDrag = (e: React.PointerEvent) => {
    if (!drag) return;
    // Auto-scroll near the edges so a page can be dragged past the viewport:
    // without this a drag dies at the visible top/bottom and far positions
    // are unreachable. Nudged per move event; moves fire continuously mid-drag.
    const scroller = e.currentTarget as HTMLElement;
    const r = scroller.getBoundingClientRect();
    if (e.clientY < r.top + DRAG_SCROLL_EDGE_PX) scroller.scrollTop -= DRAG_SCROLL_STEP_PX;
    else if (e.clientY > r.bottom - DRAG_SCROLL_EDGE_PX)
      scroller.scrollTop += DRAG_SCROLL_STEP_PX;
    const to = targetIndexFor(e.clientY, drag.from);
    if (to !== drag.to) setDrag({ ...drag, to });
  };

  const endDrag = () => {
    if (drag && drag.to !== drag.from) movePage(pages[drag.from].id, drag.to);
  };

```

```

    setDrag(null);
  };

  if (!activeBoard) return null;

  // Thumbnails keep the page's real proportions, so landscape looks landscape.
  const thumbW = PANEL_W - 40;
  const thumbH = Math.round((dim.height / dim.width) * thumbW);

  return (
    <div
      className="pointer-events-auto absolute bottom-0 left-0 top-0 z-50 flex flex-col border-r border-border/
        80 bg-white/95 shadow-2xl backdrop-blur-xl dark:bg-zinc-900/95"
      style={{ width: PANEL_W }}
    >
      <div className="flex items-center justify-between border-b border-border/60 px-3 py-2">
        <span className="text-[12px] font-semibold text-muted-foreground">
          {pages.length} page{pages.length === 1 ? "" : "s"}
        </span>
        <Button variant="ghost" size="icon" className="h-8 w-8" onClick={onClose} title="Close">
          <X size={16} />
        </Button>
      </div>

      <div className="flex gap-1 border-b border-border/60 p-1.5">
        {[
          { id: "all", label: "Pages" },
          { id: "fav", label: `★ Favourites ${favourites.length}` },
        ] as const}.map((t) => (
          <button
            key={t.id}
            onClick={() => setTab(t.id)}
            className={cn(
              "flex-1 rounded-lg px-1 py-1.5 text-[11px] font-medium transition-colors",
              tab === t.id ? "bg-primary text-primary-foreground" : "text-muted-foreground hover:bg-muted"
            )}
          >
            {t.label}
          </button>
        ))}
      </div>

      {tab === "fav" && (
        <div className="flex-1 space-y-3 overflow-y-auto p-3">
          {favourites.length === 0 ? (
            <p className="px-1 py-4 text-center text-[12px] leading-snug text-muted-foreground">
              No favourite pages yet. Open a page's menu and choose "Add to favourites".
            </p>
          ) : (
            favourites.map(({ page, index }) => {
              const sheet = paperForPage(activeBoard, page);
              const sheetFill =
                paperColorFor(page.background ?? activeBoard?.background, resolvedTheme) ||
                (resolvedTheme === "dark" ? "#18181b" : "#ffffff");
              return (
                <button
                  key={page.id}
                  onClick={() => goToPage(page.id, viewportH())}
                  title={`Go to page ${index + 1}`}
                  className="block w-full text-left"
                >
                  <div
                    className={cn(
                      "overflow-hidden rounded-lg border-2 bg-white dark:bg-zinc-800",
                      index === activeIndex ? "border-primary" : "border-border"
                    )}
                    style={{ height: thumbH }}
                  >
                    <PageThumb
                      page={page}
                      dim={dim}
                      sheetPattern={sheet.pattern}
                      sheetSize={sheet.size}
                      sheetFill={sheetFill}
                    />
                  </div>
                  <span className="mt-1 block text-[11px] tabular-nums text-muted-foreground">
                    Page {index + 1}
                  </span>
                </button>
              )
            })
          )}
        </div>
      )}
    </div>
  );

```

```

        </span>
      </button>
    );
  })
}
</div>
})

{tab === "all" && (
  <div
    className="flex-1 overflow-y-auto p-3"
    onPointerMove={moveDrag}
    onPointerUp={endDrag}
    onPointerCancel={() => setDrag(null)}
  >
    {pages.map((page, i) => {
      const dropBefore = drag && drag.to === i && drag.from > i;
      const dropAfter = drag && drag.to === i && drag.from < i;
      return (
        <div key={page.id}>
          {dropBefore && <DropLine />}
          <PageCard
            refCb={(el) => (cardRefs.current[i] = el)}
            page={page}
            index={i}
            count={pages.length}
            active={i === activeIndex}
            dragging={drag?.from === i}
            dim={dim}
            thumbH={thumbH}
            onOpen={() => goToPage(page.id, viewportH())}
            onGripDown={(e) => startDrag(e, i)}
            onHoldPickup={(i) => setDrag({ from: i, to: i })}
          />
          {dropAfter && <DropLine />}
        </div>
      );
    })}

    <button
      onClick={() => {
        insertPageAt(pages.length);
        const b = useWhiteboard.getState().boards.find((x) => x.id === activeBoard.id);
        if (b?.activePageId) goToPage(b.activePageId, viewportH());
      }}
      className="mt-1 flex w-full items-center justify-center gap-1.5 rounded-lg border-2 border-dashed
        border-border py-3.5 text-[12px] font-medium text-muted-foreground transition-colors hover:bor-
        der-primary hover:text-primary"
    >
      <Plus size={16} /> Add page
    </button>

    /* Template library: any saved page, or an imported image, becomes a
    sheet you can add again - the GoodNotes planner/worksheet pattern. */
    <div className="mt-4 border-t border-border/60 pt-3">
      <div className="mb-2 flex items-center justify-between">
        <span className="text-[11px] font-semibold uppercase tracking-wider text-muted-foreground">
          Templates
        </span>
        <button
          onClick={() => imageInputRef.current?.click()}
          title="Use an image as a page template"
          aria-label="Import image as template"
          className="flex h-8 w-8 items-center justify-center rounded-lg text-muted-foreground hover:bg-
            muted hover:text-foreground"
        >
          <ImagePlus size={16} />
        </button>
      </div>
      <input
        ref={imageInputRef}
        type="file"
        accept="image/*"
        className="hidden"
        onChange={(e) => {
          const file = e.target.files?.[0];
          if (!file) return;

```

```

const reader = new FileReader();
reader.onload = () => {
  const src = String(reader.result);
  const img = new Image();
  img.onload = () => {
    // Pixels to IndexedDB; the template keeps only a reference,
    // since one template is reused on many pages.
    storeImage(src).then((stored) =>
      createImageTemplate(
        file.name.replace(/\.(\.[^.]*)$/, ""),
        stored,
        img.naturalWidth,
        img.naturalHeight
      )
    );
  };
  img.src = src;
};
reader.readAsDataURL(file);
e.target.value = "";
}}
/>

{pageTemplates.length === 0 ? (
  <p className="text-t-11px leading-snug text-muted-foreground">
    Choose "Save as template" in a page's menu, or import an image, to reuse it as a sheet.
  </p>
) : (
  <div className="space-y-1.5">
    {pageTemplates.map((t) => (
      <div key={t.id} className="flex items-center gap-1">
        <button
          onClick={() => {
            addPageFromTemplate(t.id, pages[activeIndex]?.id);
            const b = useWhiteboard.getState().boards.find((x) => x.id === activeBoard.id);
            if (b?.activePageId) goToPage(b.activePageId, viewportH());
          }}
          title={`Add a page from "${t.name}"`}
          className="flex-1 truncate rounded-lg border border-border px-2 py-2 text-left text-[12px] transition-colors hover:border-primary hover:text-primary"
        >
          {t.name}
        </button>
        <button
          onClick={() => {
            if (window.confirm(`Delete the template "${t.name}"?`)) deletePageTemplate(t.id);
          }}
          title="Delete template"
          aria-label={`Delete template ${t.name}`}
          className="flex h-8 w-8 items-center justify-center rounded-lg text-muted-foreground hover:bg-muted hover:text-destructive"
        >
          <Trash2 size={14} />
        </button>
      </div>
    ))}
  </div>
)}
</div>
</div>
)}
</div>
);
}

function DropLine() {
  return <div className="my-1 h-1 rounded-full bg-primary" />;
}

// -----
// One page in the panel
// -----

function PageCard({
  refCb,
  page,
  index,
  count,

```

```

    active,
    dragging,
    dim,
    thumbH,
    onOpen,
    onGripDown,
    onHoldPickup,
  }: {
    refCb: (el: HTMLDivElement | null) => void;
    page: NotebookPage;
    index: number;
    count: number;
    active: boolean;
    dragging: boolean;
    dim: { width: number; height: number };
    thumbH: number;
    onOpen: () => void;
    onGripDown: (e: React.PointerEvent) => void;
    onHoldPickup: (index: number) => void;
  }) {
    const activeBoard = useActiveBoard();
    const insertPageAt = useWhiteboard((s) => s.insertPageAt);
    const goToPage = useWhiteboard((s) => s.goToPage);
    const duplicatePage = useWhiteboard((s) => s.duplicatePage);
    const deletePage = useWhiteboard((s) => s.deletePage);
    const movePage = useWhiteboard((s) => s.movePage);
    const togglePageBookmark = useWhiteboard((s) => s.togglePageBookmark);
    const setPageBackground = useWhiteboard((s) => s.setPageBackground);
    const savePageAsTemplate = useWhiteboard((s) => s.savePageAsTemplate);

    const pattern = page.background?.pattern ?? activeBoard?.background?.pattern ?? "dots";
    const rulingSize =
      page.background?.size ?? activeBoard?.background?.size ?? BOARD_PATTERN_DEFAULT_SIZE;

    // The Set-order dialog overlays the panel without unmounting it, so the
    // page list stays exactly where it was when the dialog closes.
    const [orderOpen, setOrderOpen] = React.useState(false);

    // The thumbnail wears the same paper as the canvas sheet (primitives, so
    // the memoised PageThumb below still skips re-renders when nothing changed).
    const { resolvedTheme } = useTheme();
    const sheet = paperForPage(activeBoard, page);
    const sheetFill =
      paperColorFor(page.background ?? activeBoard?.background, resolvedTheme) ||
      (resolvedTheme === "dark" ? "#18181b" : "#ffffff");

    /** Insert, then show the new page's top. */
    const insertAndShow = (at: number) => {
      insertPageAt(at);
      const b = useWhiteboard.getState().boards.find((x) => x.id === activeBoard?.id);
      if (b?.activePageId) goToPage(b.activePageId, viewportH());
    };

    const confirmDelete = () => {
      const n = page.elements.length + (page.pdf ? 1 : 0);
      // Page deletion is not undoable, so anything with content asks first.
      if (n === 0 || window.confirm(`Delete page ${index + 1}? It has content, and this can't be undone.`)) {
        deletePage(page.id);
      }
    };

    // Hold-to-pick-up: press and hold anywhere on the card, then drag to
    // reorder – the touch-friendly alternative to the small grip below. Moving
    // before the hold fires means a scroll, so the panel still scrolls from the
    // card. The grip and the ■ menu keep their own gestures (data-hold-ignore).
    const cardRef = React.useRef<HTMLDivElement | null>(null);
    const holdTimer = React.useRef<ReturnType<typeof setTimeout> | null>(null);
    const holdStart = React.useRef<{ x: number; y: number; pointerId: number } | null>(null);
    const suppressClick = React.useRef(false);

    const cancelHold = React.useCallback(() => {
      if (holdTimer.current) clearTimeout(holdTimer.current);
      holdTimer.current = null;
      holdStart.current = null;
    }, []);
    React.useEffect(() => cancelHold, [cancelHold]);

    return (

```

```

<div
  ref={el => {
    cardRef.current = el;
    refCb(el);
  }}
  className={cn("mb-3", dragging && "opacity-40")}
  onPointerDown={e => {
    if (!e.isPrimary) return;
    if ((e.target as Element).closest?("[data-hold-ignore]")) return;
    suppressClick.current = false;
    holdStart.current = { x: e.clientX, y: e.clientY, pointerId: e.pointerId };
    holdTimer.current = setTimeout(() => {
      holdTimer.current = null;
      const s = holdStart.current;
      holdStart.current = null;
      if (!s) return;
      // Capture so the finger drags the page instead of scrolling the
      // panel from here on; moves still bubble to the panel's move handler.
      try {
        cardRef.current?.setPointerCapture?.(s.pointerId);
      } catch {
        /* capture only helps; the drop works without it */
      }
      suppressClick.current = true;
      try {
        navigator.vibrate?.(15);
      } catch {
        /* haptic is a courtesy, never required */
      }
      onHoldPickup(index);
    }, HOLD_TO_DRAG_MS);
  }}
  onPointerMove={e => {
    const s = holdStart.current;
    if (s && Math.hypot(e.clientX - s.x, e.clientY - s.y) > HOLD_MOVE_PX) cancelHold();
  }}
  onPointerUp={cancelHold}
  onPointerCancel={cancelHold}
>
  <button
    onClick={() => {
      // A hold that picked the page up is not a tap: swallow the click
      // that follows the release (a real drag also ends here, and must
      // not navigate away after reordering).
      if (suppressClick.current) {
        suppressClick.current = false;
        return;
      }
      onOpen();
    }}
    title={`Go to page ${index + 1}`}
    className={cn(
      "relative block w-full overflow-hidden rounded-lg border-2 bg-white transition-colors dark:bg-zinc-800",
      active ? "border-primary shadow-md" : "border-border hover:border-muted-foreground/50"
    )}
    style={{ height: thumbH }}
  >
    <PageThumb
      page={page}
      dim={dim}
      sheetPattern={sheet.pattern}
      sheetSize={sheet.size}
      sheetFill={sheet.fill}
    />
    {page.bookmarked && (
      <Star size={14} className="absolute right-1.5 top-1.5 text-amber-500" fill="currentColor" />
    )}
  </button>

  <div className="mt-1 flex items-center gap-0.5">
    /* Grip: press and drag to reorder. touch-action none so the finger
       drags the page instead of scrolling the panel. (Or press-and-hold
       anywhere on the card, then drag.) */
    <span
      data-hold-ignore
      onPointerDown={onGripDown}
      title="Drag to reorder"

```

```

        aria-label={`Drag page ${index + 1} to reorder`}
        className="flex h-8 w-7 cursor-grab touch-none items-center justify-center rounded-md text-muted-foreground hover:bg-muted"
    >
    <GripVertical size={16} />
</span>

<span
    className={cn(
        "flex-1 text-[12px] tabular-nums",
        active ? "font-bold text-primary" : "text-muted-foreground"
    )}
>
    {index + 1}
</span>

<DropdownMenu>
    <DropdownMenuTrigger asChild>
        <button
            data-hold-ignore
            title="Page options"
            aria-label={`Options for page ${index + 1}`}
            className="flex h-8 w-8 items-center justify-center rounded-md text-muted-foreground hover:bg-muted hover:text-foreground"
        >
            <MoreHorizontal size={18} />
        </button>
    </DropdownMenuTrigger>
    <DropdownMenuContent align="end" className="w-56">
        <DropdownMenuLabel>Page {index + 1}</DropdownMenuLabel>
        <DropdownMenuItem className="py-2.5" onSelect={() => insertAndShow(index)}>
            <ArrowUpToLine size={16} className="mr-2" /> Insert page above
        </DropdownMenuItem>
        <DropdownMenuItem className="py-2.5" onSelect={() => insertAndShow(index + 1)}>
            <ArrowDownToLine size={16} className="mr-2" /> Insert page below
        </DropdownMenuItem>
        <DropdownMenuItem className="py-2.5" onSelect={() => duplicatePage(page.id)}>
            <Copy size={16} className="mr-2" /> Duplicate
        </DropdownMenuItem>
        <DropdownMenuSeparator />
        <DropdownMenuItem className="py-2.5" onSelect={() => setOrderOpen(true)}>
            <ListOrdered size={16} className="mr-2" /> Set order...
        </DropdownMenuItem>
        <DropdownMenuSeparator />
        <DropdownMenuItem className="py-2.5" onSelect={() => togglePageBookmark(page.id)}>
            <Star
                size={16}
                className={cn("mr-2", page.bookmarked && "text-amber-500")}
                fill={page.bookmarked ? "currentColor" : "none"}
            />
            {page.bookmarked ? "Remove from favourites" : "Add to favourites"}
        </DropdownMenuItem>
        <DropdownMenuSeparator />
        {/* Paper directly in this menu – NOT a nested submenu radio group:
           submenu radios swallow taps in the Android WebView, so picking
           Grid there never reached the store while everything around it
           worked. Plain items fire onSelect reliably on touch; the check
           shows the page's current ruling. */}
        <DropdownMenuLabel>Paper</DropdownMenuLabel>
        {BOARD_PATTERNS.map((pt) => (
            <DropdownMenuItem
                key={pt.id}
                className="py-2.5"
                onSelect={() => setPageBackground(page.id, { pattern: pt.id })}
            >
                {pt.id === pattern ? (
                    <Check size={16} className="mr-2 text-primary" />
                ) : (
                    <span className="mr-2 inline-block w-4" aria-hidden />
                )}
                {pt.label}
            </DropdownMenuItem>
        ))}
        <DropdownMenuSeparator />
        <div className="flex items-center justify-between px-2 pb-1 pt-1.5">
            <span className="text-[11px] font-semibold uppercase tracking-wider text-muted-foreground">
                Ruling size
            </span>

```

```

    <span className="rounded-md bg-muted px-1.5 py-0.5 text-[11px] font-semibold tabular-nums">
      {rulingSize}px
    </span>
  </div>
  {/* Slider plus +/- steppers, like the font-size control: drag for
    roughly, hold a stepper for exact. Plain controls, not menu
    items, so the menu stays open while you adjust. */}
  <div className="px-2 pb-1.5">
    <StepperRow
      value={rulingSize}
      min={BOARD_PATTERN_MIN_SIZE}
      max={BOARD_PATTERN_MAX_SIZE}
      onChange={(size) => setPageBackground(page.id, { size })}
    >
      <input
        type="range"
        min={BOARD_PATTERN_MIN_SIZE}
        max={BOARD_PATTERN_MAX_SIZE}
        value={rulingSize}
        aria-label="Ruling size"
        onChange={(e) => setPageBackground(page.id, { size: Number(e.target.value) })}
        className="h-1.5 w-full cursor-pointer appearance-none rounded-lg bg-muted accent-primary"
      />
    </StepperRow>
  </div>
  <DropdownMenuSeparator />
  <DropdownMenuItem
    className="py-2.5"
    onSelect={() => savePageAsTemplate(page.id, `Page ${index + 1} template`)}
  >
    <Save size={16} className="mr-2" /> Save as template
  </DropdownMenuItem>
  <DropdownMenuSeparator />
  <DropdownMenuItem
    className="py-2.5 text-destructive focus:text-destructive"
    onSelect={confirmDelete}
  >
    <Trash2 size={16} className="mr-2" /> Delete page...
  </DropdownMenuItem>
</DropdownMenuContent>
</DropdownMenu>
</div>
{/* Move-to-position: a tap-friendly number grid instead of repeated
  Move up/down taps (and of the fiddly drag on a tablet). The panel
  stays mounted behind the dialog. */}
<Dialog open={orderOpen} onOpenChange={setOrderOpen}>
  <DialogContent className="max-w-xs">
    <DialogHeader>
      <DialogTitle>Move page {index + 1} to...</DialogTitle>
      <DialogDescription className="text-xs">
        Pick the position this page should move to.
      </DialogDescription>
    </DialogHeader>
    <div className="grid grid-cols-4 gap-2 py-1">
      {Array.from({ length: count }, (_, k) => (
        <button
          key={k}
          disabled={k === index}
          onClick={() => {
            movePage(page.id, k);
            setOrderOpen(false);
          }}
          aria-label={`Move to position ${k + 1}`}
          className={cn(
            "h-11 rounded-lg border text-sm font-medium tabular-nums transition-colors",
            k === index
              ? "border-primary bg-primary/10 text-primary"
              : "border-border hover:border-primary/50 hover:bg-muted"
          )}
        >
          {k + 1}
        </button>
      )})}
    </div>
  </DialogContent>
</Dialog>
</div>
);

```

```

}

/**
 * A real thumbnail: the page's own paper, ink and PDF page, scaled down.
 *
 * Memoised on the page object, which the store replaces whenever that page is
 * edited - so drawing on page 3 redraws page 3's thumbnail and no other. The
 * PDF raster is requested at the page width, the same key the canvas uses, so
 * it comes straight from the cache rather than being rendered twice.
 */
const PageThumb = React.memo(function PageThumb({
  page,
  dim,
  sheetPattern,
  sheetSize,
  sheetFill,
}): {
  page: NotebookPage;
  dim: { width: number; height: number };
  sheetPattern: BoardPattern;
  sheetSize: number;
  sheetFill: string;
}) {
  const thumbPatternId = `thumb-paper-${page.id}`;
  return (
    <svg
      width="100%"
      height="100%"
      viewBox={`0 0 ${dim.width} ${dim.height}`}
      preserveAspectRatio="xMidYMid meet"
      style={{ pointerEvents: "none" }}
    >
      <defs>
        <pattern
          id={thumbPatternId}
          x={0}
          y={0}
          width={sheetSize}
          height={sheetSize}
          patternUnits="userSpaceOnUse"
        >
          <PaperTiles pattern={sheetPattern} size={sheetSize} />
        </pattern>
      </defs>
      <rect x={0} y={0} width={dim.width} height={dim.height} fill={sheetFill} />
      {page.pdf && <PdfPageLayer pdf={page.pdf} width={dim.width} height={dim.height} />}
      {sheetPattern !== "plain" && (
        <rect
          x={0}
          y={0}
          width={dim.width}
          height={dim.height}
          fill={`url(#${thumbPatternId})`}
        />
      )}
    </svg>
  );
});

=====
// === END FILE: src/components/whiteboard/NotebookPages.tsx ===
=====

=====
// === START FILE: src/components/whiteboard/PaperPattern.tsx ===
// File 67 of 194 | Lines: 57 | Size: 1816 bytes
=====

/**
 * Notebook paper, shared by every renderer.
 *
 * A sheet's ruling comes from `paperForPage` (the page's own paper, falling
 * back to the notebook's) – never from the ACTIVE page's paper. Each visible
 * sheet gets its own `<pattern>` (unique id per sheet) because different pages

```

```

*/ in one document can carry different rulings and spacings.
*/

import { BOARD_BACKGROUNDS, type BoardBackground, type BoardPattern } from "@lib/whiteboard/types";

/** Resolve a stored paper colour for the current theme ("" = follow theme). */
export function paperColorFor(
  bg: BoardBackground | undefined,
  resolvedTheme: string | undefined
): string {
  const preset = BOARD_BACKGROUNDS.find((b) => b.id === bg?.color);
  if (preset) return resolvedTheme === "dark" ? preset.dark : preset.light;
  // A custom colour is stored literally; presets are stored by id so that
  // they can flip between their light and dark variants with the theme.
  return bg?.color || "";
}

/** Ruling tiles in WORLD units (no zoom pre-multiplied – the zoomed group does it). */
export function PaperTiles({ pattern, size }: { pattern: BoardPattern; size: number }) {
  if (pattern === "dots") {
    return (
      <circle
        cx={size / 2}
        cy={size / 2}
        r={Math.max(0.5, size / 20)}
        className="fill-zinc-300 dark:fill-zinc-700"
      />
    );
  }
  if (pattern === "grid") {
    return (
      <path
        d={`M ${size} 0 L 0 0 0 0 ${size}`}
        fill="none"
        strokeWidth={0.8}
        className="stroke-zinc-300 dark:stroke-zinc-700"
      />
    );
  }
  if (pattern === "lines") {
    return (
      <path
        d={`M 0 ${size} L ${size} ${size}`}
        fill="none"
        strokeWidth={0.8}
        className="stroke-zinc-300 dark:stroke-zinc-700"
      />
    );
  }
  return null;
}

=====
// === END FILE: src/components/whiteboard/PaperPattern.tsx ===
=====

=====
// === START FILE: src/components/whiteboard/PdfPageLayer.tsx ===
// File 68 of 194 | Lines: 118 | Size: 3643 bytes
=====

"use client";

/**
 * Draws the PDF page that sits underneath a notebook page's ink.
 *
 * Pages are rasterised on demand and cached in memory, keyed by document,
 * page and render width. A 90-page lecture PDF must never be rendered all at
 * once – only the sheets the viewport can reach are asked for, and the cache
 * stops a page being re-rendered every time you scroll past it.
 *
 * The cache is memory-only on purpose. Persisting the rasters is exactly the
 * mistake this whole design avoids: it is the thing that would blow the
 * storage budget. Re-rendering after a restart costs a few milliseconds.
 */

import React from "react";

```

```

import type { PDFDocumentProxy } from "pdfjs-dist";
import type { PdfPageRef } from "@lib/whiteboard/types";

/** Rendered page images, keyed `docId:pageNumber:width`. */
const rasterCache = new Map<string, string>();
/** In-flight renders, so scrolling quickly cannot start the same one twice. */
const inFlight = new Map<string, Promise<string>>();

/** Open documents are shared with the text layer (see `openStoredPdf`). */
async function docFor(docId: string): Promise<PDFDocumentProxy> {
  const { openStoredPdf } = await import("@lib/whiteboard/pdf");
  return openStoredPdf(docId);
}

async function rasterFor(ref: PdfPageRef, width: number): Promise<string> {
  const key = `${ref.docId}:${ref.pageNumber}:${Math.round(width)}`;
  const cached = rasterCache.get(key);
  if (cached) return cached;

  let pending = inFlight.get(key);
  if (!pending) {
    pending = (async () => {
      const { renderPdfPage } = await import("@lib/whiteboard/pdf");
      const doc = await docFor(ref.docId);
      const url = await renderPdfPage(doc, ref.pageNumber, width);
      rasterCache.set(key, url);
      return url;
    })();
    inFlight.set(key, pending);
    pending.finally(() => inFlight.delete(key));
  }
  return pending;
}

export function PdfPageLayer({
  pdf,
  width,
  height,
}: {
  pdf: PdfPageRef;
  width: number;
  height: number;
}) {
  const [href, setHref] = React.useState<string | null>(null);
  const [failed, setFailed] = React.useState<string | null>(null);

  React.useEffect(() => {
    let alive = true;
    setFailed(null);
    rasterFor(pdf, width).then(
      (url) => {
        if (alive) setHref(url);
      },
      (err: Error) => {
        if (alive) setFailed(err.message);
      }
    );
    return () => {
      alive = false;
    };
  }, [pdf.docId, pdf.pageNumber, width]);

  if (failed) {
    return (
      <g pointerEvents="none">
        <rect x={0} y={0} width={width} height={height} fill="#fff" />
        <text
          x={width / 2}
          y={height / 2}
          textAnchor="middle"
          fontSize={Math.max(12, width / 40)}
          fill="#b91c1c">
          {failed}
        </text>
      </g>
    );
  }
}

```

```

if (!href) {
  // A plain sheet while the raster arrives – never a spinner that would
  // redraw on every scroll frame.
  return <rect x={0} y={0} width={width} height={height} fill="#fff" pointerEvents="none" />;
}

// "meet", not "none". A notebook made FROM a PDF has sheets the exact shape
// of its pages, so either setting looks identical there - but pages can now
// be inserted into a notebook of a different shape, and stretching someone's
// lecture slide to fit A4 is a silent corruption of what they imported.
// Letterboxing keeps the page's own proportions and centres it on the sheet.
return (
  <image
    href={href}
    x={0}
    y={0}
    width={width}
    height={height}
    preserveAspectRatio="xMidYMid meet"
    pointerEvents="none"
  />
);
}

=====
// === END FILE: src/components/whiteboard/PdfPageLayer.tsx ===
=====

=====
// === START FILE: src/components/whiteboard/PdfTextSelect.tsx ===
// File 69 of 194 | Lines: 772 | Size: 29912 bytes
=====

"use client";

/**
 * Select text on PDF / Word / PowerPoint pages
 * =====
 * The "Select text" tool: drag over a page's text with the pen, a finger or
 * the mouse to select it, double-tap for a word (triple-tap for the line), then
 * Copy, Highlight, Underline or Strike it from the floating bar.
 *
 * How it stays out of the way of writing
 * -----
 * There is no DOM text layer over the page. The page's text is geometry (runs
 * with boxes, see lib/whiteboard/pdfText.ts), and this tool hit-tests it from
 * the canvas's OWN pointer handlers - only while it is the active tool. So
 * nothing extra ever sits between the S Pen and the page: pens, the eraser,
 * the lasso, palm rejection, pinch and scrolling are exactly as before, and
 * the Android WebView never gets a chance to start its own text selection
 * (with its own handles and Copy / Share bar) over the ink.
 *
 * The selection and its handles are drawn in world space, inside the camera
 * group, so they sit on the text at every zoom and scroll - in whichever split
 * pane is live, since the live pane IS this canvas. The whole layer is
 * pointer-events: none and lives only in the canvas, never in an export.
 *
 * Highlight / underline / strike make ordinary strokes - the highlighter's and
 * the fineliner's - laid along the selected lines: saved, undoable in one step,
 * erasable, and exported exactly like a line you drew yourself.
 *
 * It also listens for `wb:pdf-find` (sent by the search panel after it opens a
 * page) and briefly lights up every match on that page, as PDF viewers do.
 */

import React, { useCallback, useEffect, useMemo, useRef, useState } from "react";
import { Check, Copy, Search, Strikethrough, Underline } from "lucide-react";
import { nanoid } from "nanoid";
import type { Board, Camera, NotebookPage, ToolType, WhiteboardElement } from "@lib/whiteboard/types";
import {
  caretAt,
  findAll,
  lineSegments,
  markStrokes,
  offsetAt,
  pageBoxOnSheet,

```

```

    queryTerms,
    rangeRects,
    selectedText,
    setCharMeasurer,
    wordRangeAt,
    type LineRect,
    type MarkKind,
    type PageLayout,
    type Rect,
  } from "@lib/whiteboard/pdfText";
import { cn } from "@lib/utils";

/** The I-beam over a marked line: "select the text", not "type text". */
export function TextSelectIcon({ size = 18, className }: { size?: number; className?: string }) {
  return (
    <svg
      width={size}
      height={size}
      viewBox="0 0 24 24"
      fill="none"
      stroke="currentColor"
      strokeWidth={2}
      strokeLinecap="round"
      strokeLinejoin="round"
      className={className}
      aria-hidden="true"
    >
      <rect x="3" y="9" width="12" height="6" rx="1.5" fill="currentColor" opacity="0.28" stroke="none" />
      <path d="M3 5h9M3 19h7" />
      <path d="M18 4v16M15.5 4h5M15.5 20h5" />
    </svg>
  );
}

// -----
// Colours
// -----

/** Highlighter tints, each with the ink its underline / strike is drawn in. */
const MARK_COLORS = [
  { name: "Yellow", fill: "#fde047", ink: "#ca8a04" },
  { name: "Green", fill: "#86efac", ink: "#16a34a" },
  { name: "Pink", fill: "#f9a8d4", ink: "#db2777" },
  { name: "Blue", fill: "#93c5fd", ink: "#2563eb" },
] as const;

const COLOR_KEY = "pronotes:textMarkColor";

function loadColorIndex(): number {
  try {
    const n = Number(localStorage.getItem(COLOR_KEY));
    return Number.isInteger(n) && n >= 0 && n < MARK_COLORS.length ? n : 0;
  } catch {
    return 0;
  }
}

// -----
// Find highlight (wb:pdf-find)
// -----

interface FindRequest {
  boardId: string;
  pageId?: string;
  pageIndex: number;
  query: string;
  /** Which match on the page to bring into view (0 = the first). */
  ordinal: number;
  at: number;
}

/** A request waits this long for its page to be the one on screen. */
const FIND_WAIT_MS = 4000;
const FLASH_MS = 2600;

// -----
// Helpers
// -----

```

```

function installMeasurer() {
  if (typeof document === "undefined") return;
  const ctx = document.createElement("canvas").getContext("2d");
  if (!ctx) return;
  ctx.font = "16px sans-serif";
  setCharMeasurer((s) => Array.from(s, (ch) => ctx.measureText(ch).width));
}
let measurerInstalled = false;

async function copyToClipboard(text: string): Promise<boolean> {
  try {
    await navigator.clipboard.writeText(text);
    return true;
  } catch {
    // Older WebViews: the textarea route still reaches the system clipboard.
    const ta = document.createElement("textarea");
    ta.value = text;
    ta.setAttribute("readonly", "");
    ta.style.cssText = "position:fixed;left:-9999px;top:0;opacity:0";
    document.body.appendChild(ta);
    ta.select();
    let ok = false;
    try {
      ok = document.execCommand("copy");
    } catch {
      ok = false;
    }
    ta.remove();
    return ok;
  }
}

// -----
// The hook
// -----

interface Params {
  tool: ToolType;
  board: Board | null;
  isNb: boolean;
  activePage: NotebookPage | null;
  activeIndex: number;
  pageDim: { width: number; height: number };
  camera: Camera;
  viewport: { width: number; height: number };
  getWorldPoint: (clientX: number, clientY: number) => { x: number; y: number };
  addElements: (els: WhiteboardElement[], commit?: boolean) => void;
  setCamera: (camera: Partial<Camera>) => void;
}

interface Drag {
  pointerId: number;
  /** "select": `anchor` stays put and the pointer moves the other end. */
  anchor: number;
  startX: number;
  startY: number;
  moved: boolean;
  /** The selection before this drag, put back if the drag is cancelled. */
  prev: Selection | null;
}

interface Selection {
  start: number;
  end: number;
}

export function usePdfTextSelect(p: Params) {
  const { tool, board, isNb, activePage, activeIndex, pageDim, camera, viewport } = p;
  const active = tool === "textselect";
  const ref = activePage?.pdf ?? null;
  const pageKey = `${board?.id ?? ""}:${activePage?.id ?? ""}:${ref?.docId ?? ""}:${ref?.pageNumber ?? 0}`;

  // undefined = still reading; null = the page has no text. Kept with the page
  // it belongs to: right after the active page changes (the canvas replays a
  // press on the sheet you touched) the old page's text must not be hit-tested
  // against the new page.
  const [layoutState, setLayoutState] = useState<{ key: string; value: PageLayout | null | undefined }>({

```

```

    key: "",
    value: undefined,
  });
  const layout = layoutState.key === pageKey ? layoutState.value : undefined;
  const setLayout = useCallback(
    (value: PageLayout | null | undefined) => setLayoutState({ key: pageKey, value }),
    [pageKey]
  );
  /** A press that arrived while this page's text was still loading. */
  const pendingRef = useRef<{ pointerId: number; x0: number; y0: number; x: number; y: number } | null>(null);
  const [legacyOffice, setLegacyOffice] = useState(false);
  const [sel, setSel] = useState<Selection | null>(null);
  const [dragging, setDragging] = useState(false);
  const [copied, setCopied] = useState(false);
  /** The action bar's measured width, to keep it inside the screen. */
  const [barW, setBarW] = useState(380);
  const [colorIndex, setColorIndex] = useState(0);
  const [flash, setFlash] = useState<{ ranges: Array<[number, number]>; key: number } | null>(null);
  const [findTick, setFindTick] = useState(0);
  const findRef = useRef<FindRequest | null>(null);
  const dragRef = useRef<Drag | null>(null);
  const tapsRef = useRef<{ at: number; x: number; y: number; type: string; count: number }>({ at: 0, x: 0, y: 0, type: "", count: 0 });

  useEffect(() => {
    if (!measurerInstalled) {
      measurerInstalled = true;
      installMeasurer();
    }
    setColorIndex(loadColorIndex());
  }, []);

  // A find request that is waiting for THIS page (the search panel navigates
  // first and asks second, so it may arrive a render early).
  const findForPage = (() => {
    const f = findRef.current;
    if (!f || !board || f.boardId !== board.id || Date.now() - f.at > FIND_WAIT_MS) return null;
    const onPage = f.pageId ? f.pageId === activePage?.id : f.pageIndex === activeIndex;
    return onPage ? f : null;
  })();

  // Text is read only when something needs it: the tool is in hand, or a find
  // is waiting. Scrolling a 90-page PDF with a pen never reads any.
  const wanted = Boolean(ref) && (active || findForPage !== null || flash !== null);
  useEffect(() => {
    setSel(null);
    // Match boxes belong to the page they were found on.
    setFlash(null);
    setLegacyOffice(false);
    dragRef.current = null;
    setDragging(false);
    if (!ref || !wanted) {
      setLayout(ref ? undefined : null);
      return;
    }
    let alive = true;
    setLayout(undefined);
    (async () => {
      const { getPageLayout, isImageOnlyOfficeImport } = await import("@/lib/whiteboard/pdfTextIndex");
      const l = await getPageLayout(ref);
      if (!alive) return;
      setLayout(l);
      if (!l && (await isImageOnlyOfficeImport(ref.docId)) && alive) setLegacyOffice(true);
    }).catch(() => alive && setLayout(null));
    return () => {
      alive = false;
    };
  }, [ref]);
  // `ref` is identified by pageKey.
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [pageKey, wanted]);

// Putting the tool down drops the selection.
useEffect(() => {
  if (!active) {
    setSel(null);
    dragRef.current = null;
    setDragging(false);
  }
});

```

```

}, [active]));

useEffect(() => {
  const onFind = (e: Event) => {
    const d = (e as CustomEvent).detail as Partial<FindRequest> | undefined;
    if (!d || typeof d.boardId !== "string" || typeof d.query !== "string" || !d.query.trim()) return;
    findRef.current = {
      boardId: d.boardId,
      pageId: typeof d.pageId === "string" ? d.pageId : undefined,
      pageIndex: typeof d.pageIndex === "number" ? d.pageIndex : -1,
      query: d.query,
      ordinal: typeof (d as { ordinal?: unknown }).ordinal === "number" ? Math.max(0, (d as { ordinal: number }).ordinal) : 0,
      at: Date.now(),
    };
    setFindTick((n) => n + 1);
  };
  window.addEventListener("wb:pdf-find", onFind);
  return () => window.removeEventListener("wb:pdf-find", onFind);
}, []);

const box = useMemo(
  () => (layout ? pageBoxOnSheet(layout.page, pageDim.width, pageDim.height) : null),
  [layout, pageDim.width, pageDim.height]
);
const toWorld = useCallback(
  (r: Rect): Rect => (box ? { x: box.x + r.x * box.w, y: box.y + r.y * box.h, w: r.w * box.w, h: r.h * box.h } : r),
  [box]
);
const toScreen = useCallback(
  (x: number, y: number) => ({
    x: (x - camera.x) * camera.zoom + viewport.width / 2,
    y: (y - camera.y) * camera.zoom + viewport.height / 2,
  }),
  [camera.x, camera.y, camera.zoom, viewport.width, viewport.height]
);

// Serve a waiting find once this page's text is in.
useEffect(() => {
  const f = findForPage;
  if (!f || !layout || !box) return;
  findRef.current = null;
  // Every word of the query lights up (search matches them in any order);
  // the view goes to the match the search stepped to.
  const terms = queryTerms(f.query);
  const ranges = terms.flatMap((t) => findAll(layout.text, t, 200)).sort((a, b) => a[0] - b[0]);
  if (!ranges.length) return;
  const key = Date.now();
  setFlash({ ranges, key });
  const lead = findAll(layout.text, terms[0] ?? f.query, 200);
  const target = lead[Math.min(f.ordinal, lead.length - 1)] ?? ranges[0];
  const r = rangeRects(layout, target[0], target[1]).map(toWorld)[0];
  if (r) {
    const top = (r.y - camera.y) * camera.zoom + viewport.height / 2;
    if (top < 40 || top > viewport.height - 60) {
      p.setCamera({ y: r.y + viewport.height * 0.15 / camera.zoom });
    }
  }
  const t = setTimeout(() => setFlash((cur) => (cur?.key === key ? null : cur)), FLASH_MS + 200);
  return () => clearTimeout(t);
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [findTick, layout, box, pageKey]);

// A press held while the text loaded starts now, from where it landed.
useEffect(() => {
  const pend = pendingRef.current;
  if (!pend || layout === undefined) return;
  pendingRef.current = null;
  if (!layout || !box) return;
  const w0 = p.getWorldPoint(pend.x0, pend.y0);
  if (w0.x < box.x || w0.x > box.x + box.w || w0.y < box.y || w0.y > box.y + box.h) return;
  const at = offsetAt(layout, (w0.x - box.x) / box.w, (w0.y - box.y) / box.h);
  if (at === null) return;
  const moved = Math.hypot(pend.x - pend.x0, pend.y - pend.y0) >= 4;
  dragRef.current = { pointerId: pend.pointerId, anchor: at, startX: pend.x0, startY: pend.y0, moved, prev: null };
  setDragging(true);

```

```

    if (moved) {
      const w = p.getWorldPoint(pend.x, pend.y);
      const now = offsetAt(layout, (w.x - box.x) / box.w, (w.y - box.y) / box.h);
      if (now !== null && now !== at) setSel({ start: Math.min(now, at), end: Math.max(now, at) });
    }
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [layout, box]);

const offsetAtClient = useCallback(
  (clientX: number, clientY: number): number | null => {
    if (!layout || !box) return null;
    const w = p.getWorldPoint(clientX, clientY);
    return offsetAt(layout, (w.x - box.x) / box.w, (w.y - box.y) / box.h);
  },
  // eslint-disable-next-line react-hooks/exhaustive-deps
  [layout, box, p.getWorldPoint]
);

const selWorldRects = useMemo(
  () => (layout && sel ? rangeRects(layout, sel.start, sel.end).map((r) => ({ ...toWorld(r), line: r.line } )
    ) : []),
  [layout, sel, toWorld]
);

const carets = useMemo(() => {
  if (!layout || !sel || sel.end <= sel.start) return null;
  const a = caretAt(layout, sel.start, "start");
  const b = caretAt(layout, sel.end, "end");
  if (!a || !b) return null;
  const wa = toWorld({ x: a.x, y: a.y, w: 0, h: a.h });
  const wb = toWorld({ x: b.x, y: b.y, w: 0, h: b.h });
  return { start: { x: wa.x, y: wa.y, h: wa.h }, end: { x: wb.x, y: wb.y, h: wb.h } };
}, [layout, sel, toWorld]);

/** Knob radius in screen pixels; the grab area is wider for a finger. */
const KNOB = 7;
const handleAt = (clientX: number, clientY: number, pointerType: string, svgRect: DOMRect | null): "start" |
  "end" | null => {
  if (!carets || !svgRect) return null;
  const reach = pointerType === "touch" ? 30 : 18;
  for (const which of ["start", "end"] as const) {
    const c = carets[which];
    const s = toScreen(c.x, c.y + c.h);
    const dx = clientX - svgRect.left - s.x;
    const dy = clientY - svgRect.top - (s.y + KNOB);
    // The knob hangs BELOW the line: a press on the text itself (small
    // text, zoomed out) is aimed at the text, not at the knob under it.
    if (Math.hypot(dx, dy) <= reach && clientY - svgRect.top >= s.y - 4) return which;
  }
  return null;
};

/**
 * A press with the tool in hand. False means "not on any text": the canvas
 * pans instead, so a drag in the margin (or on a page with no text) still
 * scrolls the notebook.
 */
const pointerDown = (e: React.PointerEvent<Element>): boolean => {
  if (!active) return false;
  if (e.pointerType === "mouse" && e.button !== 0) return false;
  const svgRect = (e.currentTarget as Element).getBoundingClientRect()?.() ?? null;

  // Second and third taps of a multi-tap land on the word just selected -
  // right beside its knobs - and must keep counting, not grab a knob.
  const t = tapsRef.current;
  const now = Date.now();
  const again =
    now - t.at < 380 && t.type === e.pointerType && Math.hypot(e.clientX - t.x, e.clientY - t.y) < 24;

  const grab = sel && !again ? handleAt(e.clientX, e.clientY, e.pointerType, svgRect) : null;
  if (grab && sel) {
    dragRef.current = {
      pointerId: e.pointerId,
      anchor: grab === "start" ? sel.end : sel.start,
      startX: e.clientX,
      startY: e.clientY,
      moved: true,
      prev: sel,
    };
  }
};

```